

Zero to Google: DSA, System Design and Test Engineering for QA Engineers

Volume 2 - System Design, from zero to expert

Written for Sahil Sharma - SDET preparing for Google SWE-Test / SDET (India). Study it with the web app <https://sahil804-ps.github.io/google-prep/> and Google NotebookLM.

Contents

1. How the Internet Works: What Happens When You Type a URL
2. Clients, Servers and APIs
3. Databases from Zero
4. Operating System and Concurrency Basics
5. Numbers and Back-of-the-Envelope Estimation
6. Building block 1: How to approach a design interview
7. Building block 2: Scalability basics
8. Building block 3: Load balancing
9. Building block 4: Caching
10. Building block 5: Databases: SQL vs NoSQL
11. Building block 6: Sharding and partitioning
12. Building block 7: Message queues and async processing
13. Building block 8: CAP theorem and consistency
14. Building block 9: APIs and communication
15. Building block 10: Reliability and observability
16. Building block 11: Designing test infrastructure
17. Case study 1: Design a URL shortener (like bit.ly)
18. Case study 2: Design a rate limiter
19. Case study 3: Design a key-value store
20. Case study 4: Design a news feed
21. Case study 5: Design a chat system
22. Case study 6: Design a notification system
23. Case study 7: Design a web crawler
24. Case study 8: Design Google Drive / file storage
25. Case study 9: Design a CI system that runs tests for every commit
26. Case study 10: Design YouTube (upload + watch)
27. Case study 11: Design a device lab for testing Android apps at scale
28. Distributed Systems Deep Dive: Replication, Consensus, Clocks and Testing
29. Google's Classic Papers Explained: GFS, MapReduce, Bigtable, Chubby, Dapper, Spanner, Borg
30. Reliability, SRE and Observability: SLOs, Alerts, Incidents and Safe Releases

31. Designing Test Infrastructure at Scale: CI, Distributed Test Execution, Flaky Tests, Analytics and Device Labs

How the Internet Works: What Happens When You Type a URL

In this chapter:

- Follow one web request step by step, from your keyboard to a server and back
- Understand IP addresses, ports, DNS, TCP and UDP in simple words
- See how the TCP 3-way handshake and TLS (HTTPS) make a safe connection
- Learn what an HTTP request and response look like, and why CDNs make websites fast

Time: ~35 minutes | **Level:** Zero

Why start here

Every system design question at Google, from "design a URL shortener" to "design a test results dashboard", sits on top of the internet. If you do not know how a request travels, words like "load balancer", "CDN" or "latency" will feel like magic.

The good news: you already use all of this every day. As an SDET, you have opened browser DevTools, seen a `404`, and debugged a slow API. This chapter gives names and order to the things you have already seen.

A classic interview warm-up question is: "**What happens when you type `https://www.google.com` into your browser and press Enter?**" By the end of this chapter, you can answer it in a clear, step-by-step way.

The big picture in one story

Think of ordering a parcel from a shop in another city using India Post.

1. You know the shop's **name**, but India Post needs the **full address with PIN code**. You look it up in a directory. (This is **DNS**.)
2. You write the address on the envelope. (This is the **IP address**.)
3. You also write which **department** inside the building should get it, like "Accounts, Room 443". (This is the **port**.)
4. Before sending valuable items, you call the shop to confirm they are open and ready. (This is the **TCP handshake**.)
5. You put the items in a locked box that only the shop can open. (This is **TLS / HTTPS**.)
6. Inside the box is your letter: "Please send me your catalogue." (This is the **HTTP request**.)
7. The shop replies with the catalogue. (This is the **HTTP response**.)

8. If the shop has a local branch in your own city, you get the parcel much faster. (This is a **CDN**.)

Keep this story in mind. Now let us look at each step properly.

IP addresses: the house address of a computer

An **IP address** (Internet Protocol address) is a number that identifies a device on a network. Every phone, laptop and server that talks on the internet has one.

There are two versions:

- **IPv4** looks like `142.250.183.4`. It has four numbers from 0 to 255, separated by dots. That is 32 bits, so there are about 4.3 billion possible addresses. That is not enough for the whole world.
- **IPv6** looks like `2404:6800:4009:82b::2004`. It is 128 bits long, so there are more addresses than we will ever need.

Some addresses are **private**. Your home Wi-Fi router gives your laptop an address like `192.168.1.5`. This address only works inside your home. The router uses **NAT** (Network Address Translation) to share one public IP address among all your home devices. It is like a big office with one reception desk: outside letters come to the reception, and the receptionist passes them to the right person.

A special address is `127.0.0.1`, also called **localhost**. It always means "this same machine". When you run a test server on your laptop at `http://localhost:8080`, you are using it.

Ports: the room number inside the building

One server can run many programs at the same time: a web server, a database, an SSH server. The IP address finds the machine. The **port** finds the program on that machine.

A port is a number from 0 to 65535. Some common ones:

Port	Used for
22	SSH (remote login)
53	DNS
80	HTTP (plain web)
443	HTTPS (secure web)
5432	PostgreSQL database
6379	Redis cache

When you type `https://example.com`, the browser uses port 443 by default. When you type `http://localhost:3000`, you are telling the browser "use port 3000 on my own machine".

The combination of IP address + port (for example `142.250.183.4:443`) is called a **socket address**. It is the full "building + room" address.

DNS: the phonebook of the internet

Humans remember names like `www.google.com`. Computers need IP addresses. **DNS** (Domain Name System) converts names into IP addresses. It is like the Contacts app on your phone: you tap "Mom", and the phone dials the number.

DNS is not one big computer. It is a tree of servers spread around the world. Here is the order of steps when your browser needs the IP for `www.google.com`:

1. **Browser cache.** Did the browser look this up recently? If yes, use it.
2. **Operating system cache.** The OS also keeps a small cache. It also checks the `hosts` file (you may have edited this file to point a test domain to a staging server).
3. **Recursive resolver.** If still not found, the OS asks a **recursive resolver**. This is usually run by your internet provider (Jio, Airtel) or a public service like Google Public DNS (`8.8.8.8`). The resolver does the hard work for you.
4. **Root server.** The resolver asks a root server: "Who handles `.com`?"
5. **TLD server.** The root points to the `.com` **TLD** (Top-Level Domain) server. The resolver asks it: "Who handles `google.com`?"
6. **Authoritative server.** The TLD server points to Google's own **authoritative** name server. This server has the final answer: "`www.google.com` is `142.250.183.4`."
7. The resolver gives the answer back to your OS, and everyone caches it.

```
Browser --> OS cache --> Recursive resolver
                               |
                               |--1--> Root server      ("ask .com servers")
                               |--2--> .com TLD server  ("ask google.com servers")
                               |--3--> Authoritative   ("142.250.183.4")
                               |
Browser <-- OS <----- answer (cached for TTL seconds)
```

Each answer comes with a **TTL** (Time To Live), in seconds. TTL says how long the answer can be cached. A TTL of 300 means "you can reuse this answer for 5 minutes". This is why, after a company changes its DNS records, some users still reach the old server for a while.

Common DNS record types:

- **A record:** name to IPv4 address.
- **AAAA record:** name to IPv6 address.
- **CNAME:** name to another name (an alias). For example, `www.shop.com` points to `shop.cdnprovider.net`.
- **MX:** which server receives email for the domain.

You can try this yourself in Python:

```
import socket
```

```
# Ask the OS to resolve a name to an IP address (uses DNS under the hood)
ip = socket.gethostbyname("example.com")
print(ip)
# Expected output: an IPv4 address string, for example "93.184.215.14"
# (the exact number can change over time)
```

TCP and UDP: two ways to send data

Data on the internet travels in small pieces called **packets**. A big web page is split into many packets. Packets can arrive late, out of order, or not at all. Two main **transport protocols** handle this differently.

TCP: the registered post

TCP (Transmission Control Protocol) is **reliable**:

- Every packet has a sequence number, so the receiver can put them back in order.
- The receiver sends an **acknowledgement** (ACK) for data it gets.
- If the sender does not get an ACK in time, it sends the packet again.
- It controls speed so it does not flood the network (**congestion control**).

TCP is like registered post with tracking. It is slower, but nothing is lost. The web (HTTP/1.1 and HTTP/2), email, file transfer and database connections use TCP.

UDP: the loudspeaker announcement

UDP (User Datagram Protocol) just sends packets. No handshake, no ACKs, no resending. If a packet is lost, it is lost.

UDP is like a railway station announcement. It is fast, and if you miss one word, the announcement continues. It is good when **speed matters more than perfection**: video calls, live cricket streaming, online games, and DNS lookups (small, quick questions).

Feature	TCP	UDP
Connection setup	Yes (handshake)	No
Delivery guaranteed	Yes	No
Order guaranteed	Yes	No
Speed	Slower	Faster
Examples	Web, email, databases	Video calls, games, DNS

Fun fact: **HTTP/3** runs on **QUIC**, a protocol built on top of UDP. QUIC adds its own reliability and encryption, but avoids some of TCP's delays. QUIC was first developed at Google and is now an IETF standard (RFC 9000).

The TCP 3-way handshake

Before TCP sends any real data, the two sides must agree to talk. This is the **3-way handshake**. Think of a phone call:

- You: "Hello, can you hear me?" (**SYN**, short for synchronise)
- Friend: "Yes, I can hear you. Can you hear me?" (**SYN-ACK**)
- You: "Yes, I can." (**ACK**)

Now the real conversation begins.

```
Client                                Server
| ----- SYN (seq=x) -----> |
| <----- SYN-ACK (seq=y, ack=x+1) - |
| ----- ACK (ack=y+1) -----> |
|                                     |
| ===== connection is open ===== |
```

Each arrow takes time to travel. The time for a message to go to the server and come back is called **RTT** (Round-Trip Time). The handshake costs one RTT before any data can be sent. If the server is in the USA and you are in Bengaluru, one RTT can be around 200 milliseconds. This is one big reason why companies put servers close to users.

When the conversation is over, TCP closes the connection with a similar exchange using **FIN** messages.

TLS and HTTPS: the locked box

Plain HTTP sends everything as readable text. Anyone on the same café Wi-Fi could read your password. **TLS** (Transport Layer Security) fixes this. **HTTPS** is simply HTTP running inside a TLS connection.

TLS gives three things:

1. **Encryption**: nobody in the middle can read the data.
2. **Integrity**: nobody can change the data without being detected.
3. **Authentication**: you know you are really talking to `yourbank.com`, not a fake site.

How does the browser know the server is real? The server shows a **certificate**. A certificate is like an Aadhaar card for a website. It says "this public key belongs to `yourbank.com`", and it is signed by a trusted **Certificate Authority** (CA). Your browser and OS come with a list of CAs they trust.

A simplified TLS 1.3 handshake, after the TCP handshake:

```
Client                                Server
| -- ClientHello (ciphers, key share) --> |
| <-- ServerHello + certificate + ----- |
|      key share + Finished                |
| -- Finished -----> |
|                                     |
| ===== encrypted HTTP traffic now ===== |
```

Both sides use the "key shares" to calculate the same secret key without ever sending that key over the network. This uses public-key math (key exchange). After that, the fast **symmetric** key encrypts all data. In TLS 1.3, this handshake adds about one RTT. Older TLS 1.2 needed about two.

Testers see TLS problems often: expired certificates on staging, "self-signed certificate" errors in automation, and tests that pass `verify=False` to hide the issue. Hiding the error in tests can also hide a real production bug.

HTTP: the actual letter

Now the connection is open and secure. The browser sends an **HTTP request** (HyperText Transfer Protocol). It is plain structured text (in HTTP/1.1):

```
GET /search?q=cricket HTTP/1.1
Host: www.google.com
User-Agent: Mozilla/5.0
Accept: text/html
Cookie: session=abc123
```

- The first line has the **method** (GET), the **path** (/search?q=cricket) and the version.
- Then come **headers**: extra information as `Name: value` lines.
- Some requests (like POST) also have a **body** after a blank line.

The server replies with an **HTTP response**:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Content-Length: 5120
Cache-Control: private, max-age=0

<!doctype html><html> ... </html>
```

The first line has a **status code** (200 OK). Then headers, a blank line, and the body (here, HTML). Chapter 2 covers methods, status codes and headers in detail.

HTTP versions in one line each:

- **HTTP/1.1**: one request at a time per connection; browsers open several connections.
- **HTTP/2**: many requests share one connection at the same time (**multiplexing**); headers are compressed.
- **HTTP/3**: like HTTP/2, but over QUIC (UDP), so a lost packet does not block all other streams.

What the browser does with the response

The browser's job is not over when the HTML arrives.

1. It **parses** the HTML and builds the **DOM** (Document Object Model), the tree of elements that Selenium and Playwright query.
2. It finds more resources: CSS, JavaScript, images, fonts. Each one may need its own request (often to the same connection, sometimes to other domains, which means more DNS lookups and handshakes).
3. It builds the CSS rules, runs JavaScript, calculates the layout, and **paints** pixels on the screen.
4. JavaScript may call APIs (using `fetch`) to load more data after the page appears.

This is why "page loaded" is not one moment. It is why Playwright has auto-waiting, and why `time.sleep(5)` in Selenium tests causes flaky tests. We return to this in Chapter 4.

CDNs: a local branch of the shop

A **CDN** (Content Delivery Network) is a group of servers spread across many cities. They keep copies of files that do not change often: images, videos, JavaScript and CSS files.

Analogy: Amazon does not ship every order from one warehouse in Mumbai. It has warehouses near every big city, so delivery is fast. A CDN is a set of warehouses for web files.

How it works:

1. The website's DNS points `static.shop.com` to the CDN (often using a CNAME).
2. The CDN's DNS gives you the IP of the **edge server** closest to you, for example in Chennai.
3. If the edge server has the file (a **cache hit**), it returns it immediately.
4. If not (a **cache miss**), it fetches the file from the main server (the **origin**), saves a copy, and returns it.

```
User in Chennai --> CDN edge (Chennai) --hit--> file returned in ~10 ms
                        |
                        +--miss--> Origin server (USA) --> copy saved at edge
```

Benefits: lower latency, less load on the origin server, and protection against traffic spikes (like a big sale or an IPL final).

The main risk is **stale content**. If you deploy a new `app.js` but the CDN still serves the old one, users get a broken page. Teams fix this by putting a version or hash in file names (`app.3f9a2c.js`) or by **purging** the CDN cache. The `Cache-Control` header tells the CDN and browser how long to keep a file.

Putting it all together

Here is the full answer to "what happens when you type a URL":

1. Parse URL: `https://www.google.com/search?q=cricket`
`scheme=https host=www.google.com port=443 path=/search`
2. DNS: `browser cache -> OS cache -> resolver -> root -> TLD -> auth`
`result: 142.250.183.4`
3. TCP: `SYN -> SYN-ACK -> ACK` (1 RTT)
4. TLS: `ClientHello -> ServerHello+cert -> Finished` (about 1 RTT)
5. HTTP: `GET /search?q=cricket --> 200 OK + HTML`
6. Browser: `parse HTML -> fetch CSS/JS/images (often from a CDN)`
`-> run JS -> layout -> paint`

In a real system, the request on the server side may also pass through a **load balancer**, many **application servers**, **caches** and **databases**. Those building blocks are covered in the later system design lessons. For now, you know the road the request travels to get there.

Tester's corner

- **Use DevTools Network tab like a timeline.** It shows DNS, initial connection (TCP), SSL (TLS), waiting (server time, called TTFB: Time To First Byte) and content download for each request. When a page is slow, this tells you *which step* is slow.
- **DNS caching causes confusing test results.** After a deploy that changes DNS, some test machines may hit the old server until the TTL expires. Note the TTL when you debug "works on my machine" issues.
- **Do not hide TLS errors.** Using `verify=False` in `requests` or ignoring certificate errors in Playwright is fine for a known local setup, but add a separate test that checks the real certificate is valid and not close to expiry.
- **Test with network conditions.** Playwright and Chrome DevTools can throttle the network or go offline. Slow 3G behaviour often finds missing loading states and timeout bugs.
- **CDN bugs look like "random" failures.** If some users see an old version, check caching headers and whether file names are versioned. Check the response headers (many CDNs add a header showing hit or miss).
- **Know your ports and hosts in test environments.** Many environment bugs are just the wrong host, wrong port, or `localhost` used inside a Docker container where it means something different.

Key takeaways

- An **IP address** finds the machine; a **port** finds the program on that machine.
- **DNS** turns names into IP addresses, using a chain of caches and servers, and answers are cached for their **TTL**.
- **TCP** is reliable and ordered; **UDP** is fast with no guarantees. HTTP/3 uses QUIC over UDP.
- The **TCP 3-way handshake** (SYN, SYN-ACK, ACK) costs one round trip before data flows.
- **TLS** gives encryption, integrity and authentication using certificates; **HTTPS** is HTTP inside TLS.
- An **HTTP request** has a method, path, headers and optional body; a **response** has a status code, headers and body.
- **CDNs** keep copies of static files close to users to cut latency, but can serve stale content.

Quiz

1. What does DNS do?
A) Encrypts web traffic B) Converts domain names to IP addresses C) Splits data into packets D) Balances load between servers
2. True or false: UDP guarantees that packets arrive in order.
3. Which port does HTTPS use by default?
A) 22 B) 53 C) 80 D) 443
4. What are the three messages of the TCP handshake, in order?
5. Which protocol is a better fit for a live video call?
A) TCP, because nothing can be lost B) UDP, because low delay matters more than a few lost packets C) DNS D) FTP
6. What does the TTL on a DNS record control?
7. Which of these is NOT something TLS provides?
A) Encryption B) Integrity C) Authentication of the server D) Faster page rendering
8. True or false: A CDN cache miss means the edge server fetches the file from the origin server.
9. What would you do? After a release, some users report an old version of the page with broken buttons, but your tests pass on a fresh machine.
10. What would you do? A Playwright test on the staging server fails with "certificate has expired". A teammate suggests ignoring HTTPS errors in the config.

Answer key

1. **B** - DNS is the internet's phonebook: it maps names like `www.google.com` to IP addresses.
2. **False** - UDP gives no ordering or delivery guarantees. TCP is the one that guarantees order.
3. **D** - HTTPS uses port 443 by default. Port 80 is plain HTTP.
4. **SYN, SYN-ACK, ACK** - The client asks to connect, the server agrees and asks back, and the client confirms.
5. **B** - In a live call, old data is useless, so resending lost packets (as TCP does) only adds delay.
6. **How long the answer can be cached** - Resolvers and clients can reuse the answer for TTL seconds before asking again.
7. **D** - TLS secures the connection. It does not make the browser render faster; it actually adds a small handshake cost.
8. **True** - On a miss, the edge gets the file from the origin, stores a copy, and serves it.
9. **Suspect caching** - Check the CDN and browser caching headers, whether JS/CSS file names include a version hash, and whether the CDN needs a purge. Add a test that checks the deployed asset version.
10. **Do not just hide it** - Ignoring errors may be OK as a short-term unblock for a known staging setup, but raise a bug to renew the certificate and add a check that alerts before certificates expire. The same bug in production would block real users.

Flashcards

- **Q:** What is an IP address? — **A:** A number that identifies a device on a network, like a house address.
- **Q:** What is a port? — **A:** A number (0-65535) that identifies which program on a machine should receive the data.
- **Q:** What does DNS do? — **A:** It converts a domain name into an IP address using caches, resolvers, root, TLD and authoritative servers.
- **Q:** What is DNS TTL? — **A:** The number of seconds a DNS answer may be cached before asking again.
- **Q:** TCP vs UDP in one line? — **A:** TCP is reliable and ordered with a handshake; UDP is fast with no guarantees.
- **Q:** What is the TCP 3-way handshake? — **A:** SYN, SYN-ACK, ACK; it opens a connection and costs one round trip.
- **Q:** What is RTT? — **A:** Round-Trip Time, the time for a message to reach the other side and a reply to come back.
- **Q:** What three things does TLS provide? — **A:** Encryption, integrity and server authentication (through certificates).
- **Q:** What is HTTPS? — **A:** HTTP sent inside an encrypted TLS connection, by default on port 443.
- **Q:** What is a CDN? — **A:** A network of edge servers in many cities that cache static files close to users.
- **Q:** What protocol does HTTP/3 use? — **A:** QUIC, which runs on top of UDP.

Clients, Servers and APIs

In this chapter:

- Understand the client/server model and what an API really is
- Learn HTTP methods, status codes, headers, cookies, sessions and JWT tokens
- Design clean REST resources with JSON, idempotency and pagination
- Compare polling, long polling, Server-Sent Events and WebSockets, and see what an API gateway does
- Know how a test engineer tests an API properly

Time: ~40 minutes | **Level:** Zero

The client/server model

A **client** is a program that asks for something. A **server** is a program that waits for requests and answers them.

Analogy: at a restaurant, you (the client) give your order to the kitchen (the server) through a waiter. You do not walk into the kitchen. You do not care how the chef cooks. You only care that you ask in a known way ("one masala dosa") and get a known result.

- Your browser is a client of `google.com`.
- The Swiggy app on your phone is a client of Swiggy's servers.
- Swiggy's order service may itself be a client of the payment service. A program can be a server for one thing and a client for another.

The server is usually **always on**, has a fixed address, and serves many clients at once. Clients come and go.

What is an API

An **API** (Application Programming Interface) is the menu and the rules of the restaurant. It says what you can ask for, how to ask, and what you get back. It hides the kitchen.

In web systems, "API" usually means an **HTTP API**: the server exposes URLs, and clients send HTTP requests to them. For example:

```
GET https://api.shop.com/v1/products/42
```

returns the details of product 42. The client does not know if the server uses Python or Java, PostgreSQL or Spanner. This hiding is what lets teams change the inside without breaking the outside, as long as the

API **contract** stays the same.

An API **contract** is the agreed shape of requests and responses: URLs, fields, types, status codes and error formats. It is often written in an **OpenAPI** (Swagger) file. As a tester, the contract is your main source of truth.

HTTP methods (verbs)

The **method** tells the server what kind of action you want.

Method	Meaning	Example	Safe?	Idempotent?
GET	Read a resource	GET /orders/7	Yes	Yes
POST	Create a resource or run an action	POST /orders	No	No
PUT	Replace a resource fully	PUT /orders/7	No	Yes
PATCH	Change part of a resource	PATCH /orders/7	No	Not always
DELETE	Remove a resource	DELETE /orders/7	No	Yes
HEAD	Like GET but only headers	HEAD /files/9	Yes	Yes
OPTIONS	Ask what is allowed (used by CORS)	OPTIONS /orders	Yes	Yes

Two important words:

- **Safe** means the request does not change anything on the server. GET should only read. A GET that deletes data is a serious bug.
- **Idempotent** means sending the same request once or ten times leaves the server in the same final state. We will look at this closely soon, because it is a favourite interview topic.

Status codes

The **status code** is a 3-digit number in the response. The first digit gives the family.

Code	Name	When it is used
200	OK	Request worked, body has the result
201	Created	A new resource was created (often after POST)
204	No Content	Worked, but no body (often after DELETE)
301	Moved Permanently	Resource has a new URL forever
302	Found	Temporary redirect
304	Not Modified	Your cached copy is still good
400	Bad Request	The request is malformed or invalid

Code	Name	When it is used
401	Unauthorized	You are not logged in (no or bad credentials)
403	Forbidden	You are logged in, but not allowed to do this
404	Not Found	The resource does not exist
405	Method Not Allowed	For example, DELETE on a read-only URL
409	Conflict	Clashes with current state (duplicate, version conflict)
422	Unprocessable Content	Well-formed but fails validation
429	Too Many Requests	You hit a rate limit
500	Internal Server Error	Bug or crash on the server
502	Bad Gateway	A proxy got a bad answer from the server behind it
503	Service Unavailable	Server is overloaded or down for maintenance
504	Gateway Timeout	A proxy waited too long for the server behind it

An easy way to remember:

- **2xx**: success.
- **3xx**: go somewhere else.
- **4xx**: **you** (the client) made a mistake.
- **5xx**: **we** (the server) made a mistake.

Common confusion: **401 vs 403**. 401 means "who are you?" (log in first). 403 means "I know who you are, and the answer is no."

Headers

Headers are extra information sent with requests and responses, as `Name: value` pairs. Useful ones:

- `Content-Type: application/json` - the format of the body you are sending.
- `Accept: application/json` - the format you want back.
- `Authorization: Bearer <token>` - your credentials.
- `Cache-Control: max-age=60` - how long the response can be cached.
- `Location: /orders/7` - where the new resource lives (with 201) or where to go (with 3xx).
- `Retry-After: 30` - with 429 or 503, how many seconds to wait before retrying.
- `X-Request-ID` or `traceparent` - an ID that follows the request through many services, very helpful when debugging logs.

Remembering who you are: cookies, sessions and tokens

HTTP is **stateless**. Each request stands alone. The server does not automatically remember that you logged in one second ago. So how does a website keep you logged in?

Sessions with cookies

1. You log in with username and password.
2. The server creates a **session** (a record like "session abc123 belongs to user 42") and stores it in memory, a database or a cache.
3. The server sends back a **cookie**: `Set-Cookie: session=abc123; HttpOnly; Secure`.
4. The browser automatically sends `Cookie: session=abc123` with every later request to that site.
5. The server looks up abc123 and knows it is you.

Analogy: a cloakroom token at a wedding hall. You give your bag, you get token number 57. The token itself means nothing; the counter keeps the real list.

Important cookie flags:

- `HttpOnly` - JavaScript cannot read it (protects against stolen cookies via XSS).
- `Secure` - only sent over HTTPS.
- `SameSite` - limits sending the cookie from other sites (helps against CSRF).

Tokens and JWT

With **tokens**, the server does not need to store sessions. A popular format is the **JWT** (JSON Web Token, pronounced "jot"). It has three parts separated by dots:

```
header.payload.signature  
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiI0MiIsImV4cCI6MTcwMDAwMDAwMH0.k3Jx...
```

- **Header**: which algorithm signed it.
- **Payload: claims** such as user ID (`sub`), roles, and expiry time (`exp`).
- **Signature**: created with the server's secret key. If anyone changes the payload, the signature no longer matches.

Analogy: a train ticket with a hologram. The ticket conductor does not need to call the booking office. He checks the hologram (signature) and the date (expiry) on the spot.

Two key facts that testers must know:

1. A JWT payload is only **encoded** (Base64URL), **not encrypted**. Anyone can read it. Never put passwords or secrets in it.
2. Because the server does not store it, a JWT is hard to **revoke** before it expires. That is why access tokens are usually short-lived (minutes), with a longer-lived **refresh token** to get new ones.

```
import base64, json

# Decode the payload (middle part) of a JWT. No secret needed to READ it.
token = "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiI0MiIsInJvbmGUiOiJ1c2VyIn0.sig"
payload = token.split(".")[1]
payload += "=" * (-len(payload) % 4)      # add Base64 padding back
print(json.loads(base64.urlsafe_b64decode(payload)))
# Expected output: {'sub': '42', 'role': 'user'}
```

A good security test: change `"role": "user"` to `"role": "admin"`, re-encode, and send it. The server must reject it with 401, because the signature no longer matches.

REST resource design

REST (Representational State Transfer) is a style for designing HTTP APIs. The main idea: your API is a set of **resources** (nouns), and you act on them with HTTP methods (verbs).

Good REST design rules:

- Use **nouns, in plural**, not verbs: `/orders`, not `/getOrders` or `/createOrder`.
- Use the **ID in the path** for one item: `/orders/7`.
- Use **nesting** for clear ownership, but not too deep: `/users/42/orders`.
- Use **query parameters** for filtering, sorting and paging: `/orders?status=paid&sort=-created_at&limit=20`.
- **Version** the API: `/v1/orders`, so you can change things later without breaking old clients.
- Return the right status code and a consistent error format.

Example for a food-delivery app:

Action	Request	Success response
List my orders	GET <code>/v1/users/42/orders</code>	200 + list
Get one order	GET <code>/v1/orders/7</code>	200 + order
Place an order	POST <code>/v1/orders</code>	201 + Location: <code>/v1/orders/7</code>
Change delivery address	PATCH <code>/v1/orders/7</code>	200 + updated order
Cancel an order	POST <code>/v1/orders/7/cancel</code> or DELETE <code>/v1/orders/7</code>	200 or 204

Note that "cancel" is a real action, not just deleting data (you still want a record of cancelled orders). Many teams use a small action sub-resource like `/cancel` for this. Being pragmatic is fine; being consistent is what matters.

Other API styles exist, such as **gRPC** (fast binary calls, used a lot inside companies, including Google, which created it) and **GraphQL** (the client asks for exactly the fields it needs). The later lesson on API design

compares these styles.

JSON

JSON (JavaScript Object Notation) is the most common body format. It is plain text with objects `{}`, arrays `[]`, strings, numbers, booleans (`true` / `false`) and `null`.

```
{
  "id": 7,
  "status": "PAID",
  "items": [{"sku": "DOS-01", "qty": 2, "price_paise": 12000}],
  "delivery_note": null
}
```

Tips you will appreciate as a tester:

- Store money as integers in the smallest unit (paise), not floats. `0.1 + 0.2` is not exactly `0.3` in floating point.
- Decide and document whether missing fields, `null` and empty strings mean different things.
- Use a fixed date format, usually ISO 8601 in UTC: `"2026-10-05T09:30:00Z"`.

A good error body is also JSON and consistent:

```
{"error": {"code": "INVALID_PINCODE", "message": "Pincode must be 6 digits", "field": "pincode"}}
```

Idempotency: safe to retry

Networks fail. Your app sends "pay Rs 500", and the connection drops before the reply arrives. Did the payment happen? The app does not know. If it retries, the user may be charged twice.

An operation is **idempotent** if doing it many times has the same effect as doing it once.

- `GET /orders/7` - idempotent, it only reads.
- `PUT /orders/7 {"address": "MG Road"}` - idempotent. Set it to "MG Road" ten times; it is still "MG Road".
- `DELETE /orders/7` - idempotent in effect. After the first call it is gone; later calls change nothing (they may return 404, but the state is the same).
- `POST /payments {"amount": 500}` - **not** idempotent by default. Each call may create a new payment.

The standard fix is an **idempotency key**. The client creates a unique ID (for example a UUID) for each logical action and sends it in a header. The server remembers keys it has processed.

```

processed = {} # idempotency_key -> saved response (a database in real life)

def create_payment(idem_key, amount):
    if idem_key in processed: # same key seen before: do NOT charge again
        return processed[idem_key]
    result = {"payment_id": len(processed) + 1, "amount": amount}
    processed[idem_key] = result # save before returning
    return result

print(create_payment("key-abc", 500)) # {'payment_id': 1, 'amount': 500}
print(create_payment("key-abc", 500)) # {'payment_id': 1, 'amount': 500} (retry, same result)
print(create_payment("key-xyz", 500)) # {'payment_id': 2, 'amount': 500} (new action)

```

UPI and card payment systems use similar ideas (a unique transaction reference) so retries do not cause double debits. In a real system, the check-and-save must be done atomically (for example with a unique database constraint), or two retries arriving at the same moment can both pass the check. Chapter 4 explains this kind of race condition.

Pagination

If a user has 50,000 orders, you cannot return them all in one response. **Pagination** returns data in pages.

Offset pagination: `GET /orders?limit=20&offset=40` means "skip 40, give me the next 20".

- Simple, and you can jump to page 50.
- Slow for big offsets (the database still walks past the skipped rows).
- If new items are added while the user is paging, items can be **skipped or shown twice**.

Cursor pagination (also called keyset): `GET /orders?limit=20&after=ord_9f3a`. The server returns a `next_cursor` that points to the last item seen.

- Stable and fast, even for huge lists.
- You cannot jump to an exact page number.

```

{"data": [ ...20 orders... ], "next_cursor": "ord_8b21", "has_more": true}

```

Testers should check: the first page, a middle page, the last page, an empty result, a `limit` of 0, a negative or huge `limit`, an invalid cursor, and adding or deleting items between page requests.

Real-time updates: polling, long polling, SSE, WebSockets

Normal HTTP is "client asks, server answers". But what if the server has news, like "your Swiggy rider is 2 minutes away" or a new chat message? There are four common ways.

1. Short polling. The client asks every few seconds: "Any update?" Simple, but wasteful. Most answers are "no".

- 2. Long polling.** The client asks, and the server **holds the request open** until there is news or a timeout (say 30 seconds). Then the client immediately asks again. Fewer empty answers, works everywhere.
- 3. Server-Sent Events (SSE).** The client opens one HTTP connection, and the server keeps sending events down it, one way only (server to client). Good for live scores, notifications, and streaming AI responses. The browser reconnects automatically.
- 4. WebSockets.** The connection starts as HTTP, then "upgrades" to a **two-way**, always-open channel. Both sides can send messages at any time with very little overhead. Good for chat, multiplayer games, and live collaborative editing.

```

Short polling:  C->S "any?"   S->C "no"    ...  C->S "any?"   S->C "yes!"
Long polling:  C->S "any?"   .....(server waits).....  S->C "yes!"   C->S "any?"
SSE:           C->S open     S->C event  S->C event  S->C event   (one way)
WebSocket:     C<->S open   C->S msg  S->C msg  S->C msg  C->S msg   (two way)

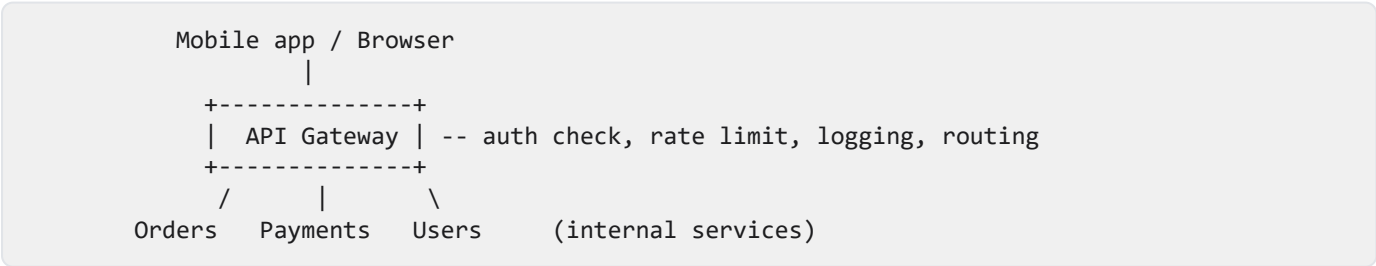
```

Method	Direction	Cost	Good for
Short polling	Client asks	High (many empty calls)	Rare updates, simple systems
Long polling	Client asks, server waits	Medium	Fallback when others are blocked
SSE	Server to client	Low	Live feeds, notifications
WebSocket	Both ways	Low per message, but keeps connections open	Chat, games, collaboration

Trade-off to remember: WebSockets and SSE keep a connection open per user. A million online users means a million open connections, which affects how servers and load balancers are designed.

API gateways

When a company has many backend services (orders, payments, users, search), clients should not need to know all of them. An **API gateway** is a single front door.



Common jobs of an API gateway:

- **Routing:** `/v1/orders/*` goes to the orders service.
- **Authentication:** check the token once, at the door.
- **Rate limiting:** return 429 if a client sends too many requests.

- **TLS termination:** handle HTTPS so internal services do not have to.
- **Logging, metrics and request IDs.**
- Sometimes **response combining** (one client call fans out to several services).

Analogy: the security gate and reception at a tech park. You show your ID once at the gate, and the reception tells you which building to go to.

How testers test APIs

API tests are faster and more stable than UI tests, so they are a big part of a good test strategy. A complete API test plan covers:

1. **Contract tests:** does the response match the schema (fields, types, required fields)? Tools: JSON Schema, OpenAPI validators, consumer-driven contract tools like Pact.
2. **Functional tests:** correct data for valid input, the right status code (201 for create, not 200), and correct `Location` header.
3. **Negative tests:** missing fields, wrong types, too-long strings, invalid IDs, wrong method (expect 405), bad JSON (expect 400).
4. **Auth tests:** no token (401), expired token (401), tampered token (401), valid token but wrong user or role (403). Try to read another user's order by changing the ID (this bug is called **IDOR**, Insecure Direct Object Reference).
5. **Idempotency and retry tests:** send the same POST twice with the same idempotency key; confirm only one resource is created.
6. **Pagination and filtering tests:** edges, empty pages, and changes during paging.
7. **Performance tests:** latency under load and behaviour at the rate limit (429 with `Retry-After`).

A small example with Python `requests` and `pytest` style:

```
import requests

BASE = "https://api.example.test/v1"

def test_create_order_returns_201_and_location():
    body = {"items": [{"sku": "DOS-01", "qty": 2}]}
    r = requests.post(f"{BASE}/orders", json=body,
                     headers={"Authorization": "Bearer test-token"}, timeout=5)
    assert r.status_code == 201  # created, not just 200
    assert r.headers["Location"].startswith("/v1/orders/")
    assert r.json()["status"] == "PENDING"

def test_missing_token_is_401():
    r = requests.get(f"{BASE}/orders/7", timeout=5)
    assert r.status_code == 401
```

Notice the `timeout=5`. Without a timeout, a hung server can make your test suite hang forever.

Tester's corner

- **Status codes are part of the contract.** A create that returns 200 instead of 201, or a validation error that returns 500, is a real bug. 5xx for bad input means the server crashed on something it should have rejected.
- **Always test authorisation, not just authentication.** Logging in is not enough; user A must never read or change user B's data by changing an ID in the URL.
- **Decode JWTs in your tests.** Check the expiry, the claims, and that tampered tokens are rejected.
- **Retry the unsafe calls.** Send POSTs twice, drop the connection in the middle, and confirm no duplicate orders or double payments.
- **Pagination hides bugs at the edges.** Test the last page, empty pages and data that changes while paging.
- **Real-time features need special tests.** For WebSockets and SSE, test reconnect after network loss, message order, and duplicate messages.
- **Set timeouts on every HTTP call in tests**, and log the request ID header so developers can find the server logs quickly.

Key takeaways

- A **client** asks and a **server** answers; an **API** is the agreed contract between them.
- HTTP **methods** express intent; GET must be safe, and PUT and DELETE are idempotent.
- **Status codes:** 2xx success, 3xx redirect, 4xx client mistake, 5xx server mistake. 401 means not logged in; 403 means not allowed.
- HTTP is **stateless**; sessions use **cookies**, while **JWTs** are signed (not encrypted) tokens that carry claims.
- **REST** uses plural nouns, IDs in paths, query parameters for filters, and versioned URLs.
- **Idempotency keys** make unsafe operations like payments safe to retry.
- **Cursor pagination** is stable for large, changing lists; **WebSockets** are two-way, **SSE** is server-to-client.
- An **API gateway** is a front door that handles routing, auth, rate limits and logging.

Quiz

1. Which status code means "you are logged in but not allowed to do this"?
A) 400 B) 401 C) 403 D) 404
2. True or false: A JWT payload is encrypted, so it is safe to put a password in it.
3. Which HTTP method should never change data on the server?
A) GET B) POST C) PATCH D) DELETE
4. Which is the best REST URL to get all orders of user 42?
A) /getOrders?user=42 B) /users/42/orders C) /orders/getAllForUser/42 D) /user-orders-42
5. What does "idempotent" mean?
6. A client gets a 503 response with `Retry-After: 30`. What should the client do?
7. Which real-time method gives a two-way, always-open channel?
A) Short polling B) Long polling C) Server-Sent Events D) WebSockets

8. True or false: With offset pagination, items can be skipped or repeated if new items are inserted while the user is paging.
9. What would you do? A "create payment" API sometimes charges a user twice when the mobile network is weak.
10. What would you do? You change the order ID in `GET /orders/1001` to `1002` while logged in as a normal user, and you see another customer's address.

Answer key

1. **C** - 403 Forbidden means the server knows who you are but refuses. 401 means you are not authenticated.
2. **False** - The payload is only Base64URL-encoded, so anyone can read it. The signature stops changes, not reading.
3. **A** - GET is a safe method; it must only read data.
4. **B** - REST uses plural nouns and nesting for ownership, with no verbs in the URL.
5. **Same result if repeated** - Doing the operation once or many times leaves the server in the same final state, so it is safe to retry.
6. **Wait and retry** - Wait at least 30 seconds before retrying, ideally with backoff, instead of retrying immediately and adding more load.
7. **D** - WebSockets allow both sides to send messages at any time over one open connection. SSE is one way.
8. **True** - Offsets shift when data changes. Cursor pagination avoids this problem.
9. **Add idempotency** - Raise a bug and suggest that the client sends a unique idempotency key per payment and the server stores processed keys with a unique constraint. Then write a test that sends the same request twice and confirms one charge.
10. **Report a critical IDOR bug** - This is a broken authorisation (IDOR) bug and a privacy leak. Report it as high severity, and add automated tests that check every resource endpoint returns 403 or 404 for other users' IDs.

Flashcards

- **Q:** What is an API? — **A:** The agreed contract (URLs, inputs, outputs, errors) that lets a client use a server without knowing its insides.
- **Q:** What does a safe HTTP method mean? — **A:** It does not change server state; GET, HEAD and OPTIONS are safe.
- **Q:** What does idempotent mean? — **A:** Repeating the operation gives the same final state as doing it once.
- **Q:** 401 vs 403? — **A:** 401 means not authenticated (log in); 403 means authenticated but not allowed.
- **Q:** 4xx vs 5xx? — **A:** 4xx is a client mistake; 5xx is a server mistake.
- **Q:** Why does HTTP need cookies or tokens? — **A:** HTTP is stateless, so each request must carry proof of who the user is.

- **Q:** What are the three parts of a JWT? — **A:** Header, payload (claims) and signature, separated by dots.
- **Q:** Is a JWT payload encrypted? — **A:** No, it is only encoded; anyone can read it, but the signature prevents tampering.
- **Q:** What is an idempotency key? — **A:** A unique client-generated ID that lets the server detect and ignore repeated requests.
- **Q:** Offset vs cursor pagination? — **A:** Offset is simple and allows page jumps; cursor is stable and fast for large, changing lists.
- **Q:** SSE vs WebSocket? — **A:** SSE is one-way server-to-client over HTTP; WebSocket is a two-way open channel.
- **Q:** What does an API gateway do? — **A:** It is the single front door that routes requests and handles auth, rate limits, TLS and logging.

Databases from Zero

In this chapter:

- Understand tables, rows, columns, primary keys and foreign keys
- Read and write basic SQL: SELECT, WHERE, JOIN and GROUP BY
- Know why indexes make queries fast, using a B-tree picture
- Explain transactions, ACID, isolation levels and their anomalies with a bank-transfer example
- Compare relational, key-value, document, wide-column and graph stores, and the basics of replication

Time: ~45 minutes | **Level:** Zero

Why databases matter

A **database** is a program that stores data safely and lets you find it again quickly. When your app restarts, memory is wiped. The database is what remembers your orders, your UPI history, and your test results.

Almost every system design answer has a database in it, and many bugs that testers find are really database bugs: duplicate records, lost updates, slow pages, and data that "disappears". This chapter gives you the base. The later lessons on SQL vs NoSQL, sharding and CAP build on it.

Tables, rows and columns

A **relational database** stores data in **tables**. A table is like an Excel sheet with strict rules.

- A **column** is one kind of information, with a fixed **type** (number, text, date).
- A **row** is one record, for example one customer.
- The **schema** is the list of tables, columns and types. It is the design of the data.

Here is a `customers` table:

id	name	city
1	Asha	Pune
2	Ravi	Delhi
3	Meena	Pune

And an `orders` table:

id	customer_id	amount	status
101	1	450	PAID
102	1	120	PAID
103	2	900	CANCELLED

Primary keys and foreign keys

A **primary key** is a column that uniquely identifies each row. In `customers`, `id` is the primary key. No two customers can have the same `id`, and it can never be empty. Think of it as an Aadhaar number for a row.

A **foreign key** is a column that points to the primary key of another table. In `orders`, `customer_id` is a foreign key to `customers.id`. It says "this order belongs to that customer".

The database can **enforce** foreign keys. If you try to insert an order with `customer_id = 99` and there is no customer 99, the database refuses. This is called **referential integrity**. It stops "orphan" rows that point to nothing.

customers				orders		
id	name			id	customer_id	amount
1	Asha			101	1	450
2	Ravi			103	2	900

Other useful **constraints** (rules the database checks for you):

- **UNIQUE** - no duplicates, for example on `email`.
- **NOT NULL** - the value is required.
- **CHECK** - a custom rule, for example `amount >= 0`.

Constraints are your last line of defence. Even if application code has a bug, the database can still block bad data.

SQL basics

SQL (Structured Query Language) is the language for talking to relational databases. You describe **what** you want, and the database decides **how** to get it.

SELECT and WHERE

```
SELECT name, city
FROM customers
WHERE city = 'Pune';
```

Result: Asha (Pune), Meena (Pune).

- `SELECT` chooses the columns.
- `FROM` chooses the table.
- `WHERE` filters the rows.
- `ORDER BY amount DESC` sorts, and `LIMIT 10` takes only the first 10.

JOIN

A **JOIN** combines rows from two tables using a matching column.

```
SELECT c.name, o.id, o.amount
FROM customers c
JOIN orders o ON o.customer_id = c.id;
```

Result:

name	id	amount
Asha	101	450
Asha	102	120
Ravi	103	900

Meena does not appear because she has no orders. This is an **INNER JOIN**: only rows that match on both sides. A **LEFT JOIN** keeps every row from the left table, and fills missing right-side values with `NULL`. A `LEFT JOIN` would show Meena with `NULL` for the order columns. That is how you find "customers who never ordered".

GROUP BY

GROUP BY puts rows into groups and calculates a summary for each group, using functions like `COUNT`, `SUM`, `AVG`, `MIN` and `MAX`.

```
SELECT customer_id, COUNT(*) AS num_orders, SUM(amount) AS total
FROM orders
WHERE status = 'PAID'
GROUP BY customer_id
HAVING SUM(amount) > 200;
```

Result: customer 1, 2 orders, total 570.

`WHERE` filters rows **before** grouping. `HAVING` filters groups **after** grouping.

You can try SQL right now with Python's built-in SQLite:

```
import sqlite3

db = sqlite3.connect(":memory:") # a temporary database in memory
db.execute("CREATE TABLE orders (id INTEGER PRIMARY KEY, customer_id INT, amount INT)")
db.executemany("INSERT INTO orders VALUES (?, ?, ?)",
               [(101, 1, 450), (102, 1, 120), (103, 2, 900)])
rows = db.execute(
    "SELECT customer_id, SUM(amount) FROM orders GROUP BY customer_id"
).fetchall()
print(rows)
# Expected output: [(1, 570), (2, 900)]
```

Note the `?` placeholders. They pass values safely. Building SQL by joining strings with user input leads to **SQL injection**, a classic security bug that testers should always try.

Indexes: the index at the back of a book

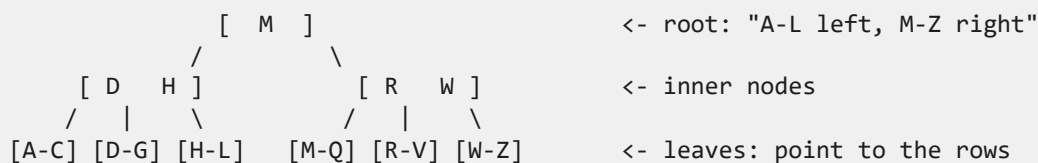
Imagine a 1,000-page book on cricket. To find every page that mentions "Dhoni", you can read all 1,000 pages. Or you can turn to the **index** at the back, which lists "Dhoni: 45, 210, 388". The second way is much faster.

Without an index, a query like `WHERE email = 'asha@x.com'` must check every row. This is a **full table scan**. With a million rows, that is slow.

An **index** is an extra data structure that the database keeps, sorted by one or more columns, so it can jump to the right rows quickly.

B-tree intuition

Most relational databases use a **B-tree** (or its close cousin, the B+ tree) for indexes. Think of it as a sorted, multi-level signboard system, like the floor directory in a big mall.



To find "Meena", you start at the root, go right (M or later), then to the leftmost leaf (M-Q), and find her. Each node holds many keys (often hundreds), so the tree is very wide and very short. Even with hundreds of millions of rows, a lookup usually needs only a few levels. In Big-O terms, a lookup is **O(log n)** instead of **O(n)** for a full scan.

Because leaves are kept in sorted order, B-trees are also good for **range queries** like `WHERE created_at BETWEEN '2026-01-01' AND '2026-01-31'` and for `ORDER BY`.

Indexes are not free

- Every `INSERT`, `UPDATE` and `DELETE` must also update every index on that table. More indexes mean slower writes.
- Indexes use extra disk and memory.
- An index on a column with very few distinct values (like `is_active` true/false) often does not help much.

Rule of thumb: index the columns you often search, join or sort by. Use the database's `EXPLAIN` command to see whether a query uses an index or does a full scan.

Transactions and ACID

A **transaction** is a group of database operations that must succeed or fail **as one unit**.

The classic example is a bank transfer. Asha sends Rs 500 to Ravi:

```
BEGIN;  
UPDATE accounts SET balance = balance - 500 WHERE id = 'asha';  
UPDATE accounts SET balance = balance + 500 WHERE id = 'ravi';  
COMMIT;
```

What if the server crashes after the first `UPDATE` but before the second? Without a transaction, Rs 500 disappears from the world. With a transaction, the database guarantees that either **both** updates happen or **neither** does. If something goes wrong, you call `ROLLBACK` and everything is undone.

Relational databases promise four properties, called **ACID**:

- **A - Atomicity**: all or nothing. The transfer never half-happens.
- **C - Consistency**: the data moves from one valid state to another. Rules (constraints like `balance >= 0`) are never broken after a commit.
- **I - Isolation**: transactions running at the same time do not see each other's half-done work. It looks as if they ran one after another (how strictly this holds depends on the isolation level, below).
- **D - Durability**: once the database says "committed", the data survives a crash or power cut. It is written to disk (usually to a log first).

Analogy: a UPI payment. You never see "money left my account but never reached the shop" as a final state. Either it completes, or it is reversed.

```
import sqlite3

db = sqlite3.connect(":memory:")
db.execute("CREATE TABLE accounts (id TEXT PRIMARY KEY, balance INT CHECK (balance >= 0))")
db.executemany("INSERT INTO accounts VALUES (?, ?)", [("asha", 300), ("ravi", 100)])
db.commit()
try:
    with db: # opens a transaction; commits on success, rolls back on error
        db.execute("UPDATE accounts SET balance = balance + 500 WHERE id = 'ravi'")
        db.execute("UPDATE accounts SET balance = balance - 500 WHERE id = 'asha'")
except sqlite3.IntegrityError:
    print("transfer failed, rolled back")
print(db.execute("SELECT * FROM accounts").fetchall())
# Expected output:
# transfer failed, rolled back
# [('asha', 300), ('ravi', 100)] <- Ravi did NOT keep the extra 500
```

Isolation levels and anomalies

Full isolation is expensive, because it means transactions must often wait for each other. So databases offer **isolation levels**: weaker levels are faster but allow some strange effects, called **anomalies**.

The main anomalies:

- **Dirty read**: you read data that another transaction has changed but **not yet committed**. If that transaction rolls back, you used data that never really existed.
- **Non-repeatable read**: you read the same row twice in one transaction and get **different values**, because someone else committed a change in between.
- **Phantom read**: you run the same query twice (for example "all orders over Rs 1000") and **new rows appear**, because someone inserted them in between.
- **Lost update**: two transactions read the same value, both change it, and the second write **overwrites** the first. Example: two people book the last seat on a train at the same moment; both read "1 seat left", both write "0 seats left", and two tickets are sold for one seat.

The standard SQL isolation levels:

Level	Dirty read	Non-repeatable read	Phantom read
Read Uncommitted	Possible	Possible	Possible
Read Committed	Prevented	Possible	Possible
Repeatable Read	Prevented	Prevented	Possible (in the standard)
Serializable	Prevented	Prevented	Prevented

Real databases differ in the details. For example, PostgreSQL's default is Read Committed, and MySQL's InnoDB default is Repeatable Read. Always check the documentation of the database you test.

How do you prevent the lost update for the train seat?

- Do it in **one atomic statement**: `UPDATE trains SET seats = seats - 1 WHERE id = 7 AND seats > 0;` and check that one row was updated.
- Or **lock the row** while reading: `SELECT ... FOR UPDATE`.
- Or use **optimistic locking**: keep a `version` column; update only `WHERE version = 5`, and if zero rows change, someone else won, so retry.
- Or use the **Serializable** level and retry when the database reports a conflict.

Normalisation basics

Normalisation means designing tables so that each fact is stored **in only one place**.

Bad design: store the customer's city in every order row. If Asha moves from Pune to Mumbai, you must update 200 order rows. If you miss one, the data now disagrees with itself. This is called an **update anomaly**.

Good design: store the city once in `customers`, and let `orders` point to the customer with a foreign key.

The common normal forms, in simple words:

- **1NF (First Normal Form)**: each cell has one value. No lists like `"phone1, phone2"` in one cell.
- **2NF**: every non-key column depends on the **whole** primary key, not just part of it.
- **3NF**: non-key columns depend **only** on the key, not on other non-key columns. (City depends on the customer, not on the order.)

Denormalisation is the opposite: copying data on purpose to make reads faster and avoid expensive joins. Large systems often denormalise for speed, and accept that they must keep the copies in sync. It is a trade-off, not a mistake.

Beyond tables: NoSQL families

Relational databases are excellent, but not perfect for every job. "**NoSQL**" is a loose name for databases that do not use the classic table model. There are four main families.

Key-value stores

Data is a giant dictionary: a **key** maps to a **value**. You can get, put and delete by key, and that is mostly all.

- Example: `session:abc123 -> {"user_id": 42}`.
- Very fast and simple. Used for caches, sessions, and counters.
- Examples: Redis, Memcached, Amazon DynamoDB (which also supports more).

Document stores

Each record is a **document**, usually JSON-like, and documents in the same collection can have different fields.

```
{ "_id": "order_7", "customer": {"name": "Asha", "city": "Pune"},  
  "items": [{"sku": "DOSA-01", "qty": 2}], "status": "PAID" }
```

- Good when data is naturally nested and read together (a product catalogue, a user profile).
- Flexible schema, but joins across documents are weaker.
- Examples: MongoDB, Firestore.

Wide-column stores

Data is grouped by a **row key**, and each row can have a huge, flexible set of columns. Data is stored and sorted by the row key, which makes it very good for massive write volumes and time-series data.

- Good for logs, metrics, sensor data, and messages by user and time.
- Examples: Apache Cassandra, Apache HBase, and Google Cloud Bigtable. Google's original Bigtable design is described in the public 2006 paper "Bigtable: A Distributed Storage System for Structured Data".

Graph databases

Data is stored as **nodes** (things) and **edges** (relationships). Questions like "friends of friends who live in Pune" are natural and fast.

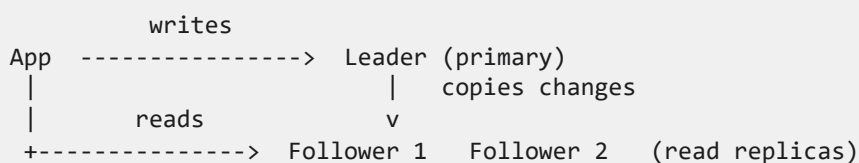
- Good for social networks, recommendations, fraud detection (which accounts are linked).
- Examples: Neo4j, Amazon Neptune.

Replication basics

Replication means keeping **copies** of the same data on several machines. Why?

- **Availability**: if one machine dies, another can take over.
- **Read scaling**: many copies can serve many reads.
- **Lower latency**: a copy can sit near users in another region.

The most common setup is **leader-follower** (also called primary-replica):



- All **writes** go to the leader.
- The leader sends changes to the followers.
- **Reads** can go to the leader or to followers.

Synchronous replication: the leader waits until followers confirm before saying "committed". Safer, but slower.

Asynchronous replication: the leader confirms immediately and sends changes later. Faster, but followers can be **behind**. This delay is called **replication lag**.

Replication lag causes a famous bug: you update your profile picture, the page reloads, and you still see the old picture, because the read went to a follower that has not caught up. A common fix is **read-your-own-writes**: send a user's reads to the leader for a short time after they write.

What happens when copies disagree, and how distributed databases choose between consistency and availability, is covered in the later lessons on replication and the CAP theorem. Splitting data across machines (sharding) also has its own lesson.

When to choose what

Need	Good first choice	Why
Money, orders, bookings, anything needing transactions	Relational (PostgreSQL, MySQL, Spanner)	ACID, constraints, joins
Sessions, cache, counters, rate limits	Key-value (Redis)	Very fast lookups by key
Flexible, nested records read as a whole	Document (MongoDB, Firestore)	Schema flexibility, one read per object
Huge write volume, time-series, logs	Wide-column (Cassandra, Bigtable)	Scales writes across many machines
Relationship-heavy queries	Graph (Neo4j)	Fast traversal of connections

In interviews, a safe default is: **start with a relational database unless you have a clear reason not to**, then explain the reason when you switch. Many real systems use several databases together, each for the job it does best.

Tester's corner

- **Test constraints directly.** Try to insert duplicates, nulls and negative amounts through the API. If they get in, the database is missing a constraint, and application code alone is protecting the data.
- **Test concurrency, not just correctness.** Fire two requests to book the last seat or redeem the same coupon at the same time. Lost updates only show up under parallel load.
- **Look for SQL injection.** Put `' OR '1'='1` and similar values into search and filter fields. Errors or extra data mean the query is built unsafely.
- **Watch for slow queries as data grows.** A page that is fast with 100 test rows can be slow with 10 million. Seed realistic data volumes and use `EXPLAIN` with developers to check index use.
- **Replication lag causes "flaky" reads.** A test that writes and then immediately reads may sometimes fail if reads go to a replica. Know the read path before calling a test flaky.
- **Keep test data isolated.** Use transactions that roll back after each test, or unique IDs per test run, so tests do not depend on each other's data.

Key takeaways

- A relational database stores data in **tables** of rows and columns; a **primary key** identifies a row, and a **foreign key** links to another table.
- **SQL** uses SELECT/WHERE to filter, **JOIN** to combine tables, and **GROUP BY** with HAVING to summarise.
- **Indexes** (usually B-trees) turn full scans into fast $O(\log n)$ lookups, but slow down writes.
- A **transaction** is all-or-nothing; **ACID** means atomicity, consistency, isolation and durability.
- Weaker **isolation levels** allow anomalies like dirty reads, non-repeatable reads, phantoms and lost updates.
- **Normalisation** stores each fact once; **denormalisation** copies data on purpose for faster reads.
- NoSQL families are **key-value**, **document**, **wide-column** and **graph**, each good for different access patterns.
- **Replication** copies data for availability and read scaling; asynchronous replication causes **replication lag**.

Quiz

1. What is a foreign key?
A) A key used for encryption B) A column that points to the primary key of another table C) The first column of any table D) A key stored in a different database
2. True or false: **WHERE** filters groups after **GROUP BY**, and **HAVING** filters rows before it.
3. Which JOIN would you use to list all customers, including those with no orders?
A) INNER JOIN B) LEFT JOIN with customers on the left C) No join is possible D) GROUP BY
4. Why can adding many indexes make a table slower?
5. In a bank transfer, which ACID property guarantees that money is never debited without being credited?
A) Atomicity B) Consistency C) Isolation D) Durability
6. Two users book the last train seat at the same instant and both get a ticket. What is this anomaly called?
7. Which store fits a user session cache best?
A) Graph database B) Key-value store C) Wide-column store D) A spreadsheet
8. True or false: With asynchronous replication, a read from a follower can return old data.
9. What would you do? An API test creates a user and then immediately fetches it, and it fails with 404 about 3% of the time.
10. What would you do? The orders page is fast in the test environment but takes 8 seconds in production.

Answer key

1. **B** - A foreign key references another table's primary key and keeps the link valid (referential integrity).
2. **False** - It is the other way round: **WHERE** filters rows before grouping; **HAVING** filters groups after.

3. **B** - A LEFT JOIN keeps every customer, with NULL order columns for customers who have no orders.
4. **Every write updates every index** - Each insert, update and delete must also change all indexes, which adds work and storage.
5. **A** - Atomicity means both updates happen or neither does.
6. **Lost update** - Both read "1 seat", both write "0", and one write overwrites the other. Fix it with an atomic conditional update, row locks or optimistic locking.
7. **B** - Sessions are looked up by a single key, very often, so a fast key-value store like Redis fits well.
8. **True** - The follower may not have received the latest changes yet; this is replication lag.
9. **Suspect replication lag** - Check whether reads go to a replica. Confirm by logging which database served the read, and discuss read-your-own-writes with the developers instead of just adding a retry or sleep in the test.
10. **Suspect data volume and indexes** - Production has much more data. Seed a realistic volume in a test environment, use EXPLAIN to check for full table scans, and suggest an index on the filtered or sorted columns. Add a performance test with large data.

Flashcards

- **Q:** What is a primary key? — **A:** A column (or set of columns) that uniquely identifies each row and cannot be null.
- **Q:** What is a foreign key? — **A:** A column that points to another table's primary key, enforcing a valid link.
- **Q:** INNER JOIN vs LEFT JOIN? — **A:** INNER keeps only matching rows; LEFT keeps all left rows and fills missing right values with NULL.
- **Q:** WHERE vs HAVING? — **A:** WHERE filters rows before grouping; HAVING filters groups after GROUP BY.
- **Q:** Why are B-tree indexes fast? — **A:** They are sorted, wide and short trees, so a lookup needs only a few steps ($O(\log n)$).
- **Q:** What is the cost of an index? — **A:** Slower writes and extra storage, because every index must be updated.
- **Q:** What does ACID stand for? — **A:** Atomicity, Consistency, Isolation, Durability.
- **Q:** What is a dirty read? — **A:** Reading another transaction's changes before they are committed.
- **Q:** What is a lost update? — **A:** Two transactions read the same value and the second write silently overwrites the first.
- **Q:** What is normalisation? — **A:** Designing tables so each fact is stored only once, avoiding update anomalies.
- **Q:** Name the four NoSQL families. — **A:** Key-value, document, wide-column and graph.
- **Q:** What is replication lag? — **A:** The delay before a follower copy receives changes from the leader, causing stale reads.

Operating System and Concurrency Basics

In this chapter:

- Understand the four main resources of a computer: CPU, memory, disk and network
- Know the difference between processes and threads, and between concurrency and parallelism
- See how race conditions happen, and how locks fix them (and how deadlocks appear)
- Learn `async/await`, event loops, the Python GIL, timeouts and retries
- Connect race conditions and timing directly to flaky tests

Time: ~40 minutes | **Level:** Zero

Why a tester should care

You have seen this many times: a test passes 9 times out of 10, and fails once with no code change. People call it a **flaky test**. Very often, the real cause is **timing**: two things happen in a different order than the test expected.

To understand timing bugs, you need to know how a computer runs many things at once. That is the job of the **operating system** (OS): Linux, Windows, macOS, Android. This chapter explains the core ideas in simple words. They also appear in system design interviews whenever you talk about servers handling many requests.

The four resources: CPU, memory, disk, network

Think of a computer as a restaurant kitchen.

- **CPU** (Central Processing Unit) is the **chef**. It does the actual work: calculations and running instructions. A modern CPU has several **cores**; each core is like one chef who can work on one task at a time.
- **Memory** (RAM) is the **kitchen counter**. It is fast to reach, but limited in size, and it is cleared when the power goes off.
- **Disk** (SSD or hard disk) is the **storeroom**. It is large and keeps things after power off, but it is much slower to reach than the counter.
- **Network** is the **delivery van** to other restaurants. It is the slowest, and sometimes the van gets stuck in traffic.

The speed difference is huge. Reading from memory takes around 100 nanoseconds, while a network round trip to another continent takes around 150 milliseconds, which is more than a million times longer. (Chapter 5 has a full table of approximate numbers.) This is why programs spend most of their time **waiting**, not calculating.

A task that mostly waits for disk or network is called **I/O-bound** (I/O means input/output). A task that mostly uses the CPU, like resizing images or compressing files, is called **CPU-bound**. This difference decides which concurrency tool you should use.

Processes and threads

A **program** is a file on disk, like `chrome.exe` or `python`. When you run it, the OS creates a **process**.

A **process** is a running program with its **own memory space**. One process cannot normally read another process's memory. If one process crashes, others keep running. Each Chrome tab often runs in its own process, so one bad website does not crash the whole browser.

A **thread** is a smaller unit of work **inside** a process. A process can have many threads, and they all **share the same memory**.

Analogy: a process is a **flat** (apartment). Each flat has its own kitchen and locked door. Threads are **family members inside one flat**. They share the same fridge. This is convenient (easy to share food), but if two people take the last bottle of milk at the same time, there is a problem.

	Process	Thread
Memory	Own, separate	Shared with other threads in the process
Cost to create	Higher	Lower
Crash impact	Only that process	Can crash the whole process
Talking to each other	Harder (pipes, sockets, files)	Easy (shared variables), but risky

Context switching

A machine may have 8 cores but hundreds of processes and thousands of threads. How do they all run? The OS **scheduler** gives each thread a small slice of CPU time (a few milliseconds), then pauses it and runs another. It switches so quickly that everything seems to run at the same time.

Switching from one thread to another is called a **context switch**. The OS must save the current thread's state (where it was, its register values) and load the next one's. Like a chef who stops making dosa to stir the sambar: he must remember exactly where he stopped.

Context switches are not free. Each one costs some microseconds and makes CPU caches less effective. With too many threads, the CPU can spend a big share of its time switching instead of working.

The key point for testers: **you do not control when the OS pauses a thread**. A thread can be paused in the middle of any operation. This is the root of race conditions.

Concurrency vs parallelism

These two words are often mixed up.

- **Concurrency** means **dealing with** many tasks at once. Tasks make progress in overlapping time periods, but not necessarily at the same instant.
- **Parallelism** means **doing** many tasks at the exact same instant, on different cores.

Analogy: one chef cooking three dishes, switching between them while each one simmers, is **concurrency**. Three chefs each cooking one dish at the same time is **parallelism**.

```
Concurrency (1 core):  A A B B A C C B A C    (tasks take turns)
Parallelism (3 cores): core1: A A A A A
                      core2: B B B B B    (tasks run truly together)
                      core3: C C C C C
```

You can have concurrency without parallelism (one core switching tasks), and most real servers use both.

Race conditions

A **race condition** happens when the result depends on the **timing or order** of events that you do not control. The threads "race", and the winner decides the result.

Classic example: two threads both add 1 to a shared counter. `count += 1` looks like one step, but it is really three:

1. Read `count` from memory.
2. Add 1.
3. Write the result back.

If the OS switches threads at the wrong moment:

```
count = 5
Thread A: read count  (5)
Thread B: read count  (5)      <- B reads before A writes
Thread A: write 5+1    (count = 6)
Thread B: write 5+1    (count = 6) <- A's update is lost; expected 7
```

This is the same "lost update" you saw with the train seat in the databases chapter. It is the same idea, at a different level.

```

import threading, time

count = 0
def add_many():
    global count
    for _ in range(1_000):
        value = count          # 1. read
        time.sleep(0)          # Let another thread run between read and write
        count = value + 1      # 2. add and 3. write: NOT atomic

threads = [threading.Thread(target=add_many) for _ in range(4)]
for t in threads: t.start()
for t in threads: t.join()
print(count)
# Expected: 4000. Actual: much lower, and different on every run.
# Without the sleep the gap is tiny, so the bug hides most of the time.

```

Notice the comment. The bug **does not always show up**. That is exactly what makes race conditions dangerous, and exactly what makes flaky tests.

The part of code that touches shared data and must not be interrupted by another thread is called the **critical section**.

Locks and mutexes

A **lock**, also called a **mutex** (mutual exclusion), makes sure only **one thread at a time** can enter a critical section.

Analogy: a single-key bathroom at a petrol pump. You take the key, use the bathroom, and return the key. Anyone else must wait until the key is back.

```

import threading

count = 0
lock = threading.Lock()

def add_many():
    global count
    for _ in range(100_000):
        with lock:             # only one thread at a time runs this line
            count += 1

threads = [threading.Thread(target=add_many) for _ in range(4)]
for t in threads: t.start()
for t in threads: t.join()
print(count) # Expected output: 400000, every run

```

Locks fix the race, but they have costs:

- Threads **wait** for the lock, so the program can become slower.
- If you forget to release a lock (for example after an exception), other threads wait forever. Using `with lock:` releases it automatically.
- Locking too much code removes the benefit of concurrency.

Other tools you may hear about: **atomic operations** (hardware-level single-step updates), **semaphores** (allow up to N threads at once, like a parking lot with N spaces), and **thread-safe queues** (one thread puts work in, others take it out safely).

Deadlock

A **deadlock** happens when two or more threads each wait for something another one holds, so **none** of them can ever continue.

Analogy: two cars meet on a narrow one-lane bridge from opposite sides. Each waits for the other to reverse. Nobody moves, forever.

```
Thread 1: holds Lock A, waiting for Lock B
Thread 2: holds Lock B, waiting for Lock A
---> both wait forever
```

Deadlock needs four conditions at the same time (known as the Coffman conditions):

1. **Mutual exclusion:** a resource can be held by only one thread.
2. **Hold and wait:** a thread holds one resource while waiting for another.
3. **No preemption:** you cannot force a thread to give up a resource.
4. **Circular wait:** there is a cycle of threads each waiting on the next.

Break any one condition, and deadlock cannot happen. The most practical fixes:

- **Always take locks in the same order** (for example, always A before B). This breaks circular wait.
- **Use timeouts when acquiring locks** (`lock.acquire(timeout=2)`), and back off if you fail.
- **Hold fewer locks, for shorter times.**

Databases detect deadlocks between transactions themselves, and cancel one of them with an error. Your application should be ready to retry that transaction.

Async/await and event loops

Threads are one way to handle many waiting tasks. Another way is **asynchronous programming**, using an **event loop**.

Analogy: a single waiter in a busy restaurant. He takes an order from table 1, gives it to the kitchen, and does **not** stand there waiting for the food. He goes to table 2, then table 3. When the kitchen rings the bell for table 1's food, he picks it up and serves it. One waiter serves many tables because most of the time is spent waiting for the kitchen.

The **event loop** is that waiter. It runs on one thread, keeps a list of tasks, and runs whichever task is ready. When a task must wait for I/O (a network call, a timer), it **gives control back** to the loop with `await`, and the loop runs something else.

```
import asyncio, time

async def fetch(name, seconds):
    await asyncio.sleep(seconds)    # pretend network wait; loop runs others meanwhile
    return f"{name} done"

async def main():
    start = time.perf_counter()
    results = await asyncio.gather(fetch("orders", 1), fetch("users", 1), fetch("cart", 1))
    print(results, round(time.perf_counter() - start))

asyncio.run(main())
# Expected output: ['orders done', 'users done', 'cart done'] 1
# Three 1-second waits finish in about 1 second total, not 3.
```

Important rules:

- Async is great for **I/O-bound** work (many network calls). It does **not** make CPU-bound work faster.
- One blocking call inside async code (like `time.sleep(5)` or a slow non-async library call) **freezes the whole loop**, and every other task waits.
- JavaScript and Node.js work this way. Playwright's API is built on async in Node.js and offers both async and sync APIs in Python. Browser JavaScript also runs on an event loop, which is why page updates happen "later" than your test's click.

The Python GIL

In the main Python implementation (**CPython**), there is a **Global Interpreter Lock** (GIL). It allows only **one thread to run Python bytecode at a time** inside a process.

What this means in practice:

- For **I/O-bound** work, threads still help a lot. While one thread waits for the network, the GIL is released and another thread runs.
- For **CPU-bound** work, threads in CPython usually give **little or no speed-up**, because only one runs Python code at a time. Use **multiprocessing** (separate processes, each with its own GIL) instead.
- The GIL does **not** make your code safe from race conditions. The `count += 1` example can still go wrong, because a thread can be switched between the read and the write. Always use locks for shared data.

Note: Python 3.13 added an optional, experimental "free-threaded" build that can run without the GIL. The normal default build still has the GIL.

Work type	Good tool in Python
Many network calls (API tests, scraping)	<code>asyncio</code> or threads
Heavy calculation (image processing, big data crunch)	<code>multiprocessing</code>
Running many test files in parallel	Separate processes (for example <code>pytest-xdist</code>)

Timeouts

A **timeout** is the maximum time you are willing to wait for something. Without one, a call to a hung server can wait **forever**, and so does everything waiting behind it.

Analogy: you call a customer care number. If nobody answers in 2 minutes, you hang up and try later. You do not hold the phone for three days.

Good timeout practices:

- **Every network call needs a timeout:** HTTP calls, database queries, message queue reads.
- There are often two kinds: a **connect timeout** (how long to wait to open a connection) and a **read timeout** (how long to wait for the response once connected).
- In a chain of services, inner timeouts should be **shorter** than outer ones. If the gateway waits 5 seconds, the service behind it should give up before that, so the error can be reported clearly.

Retries, backoff and jitter

Some failures are **temporary (transient)**: a brief network glitch, a 503 during a deploy, a database deadlock. Retrying often fixes them. But careless retries can make things worse.

Rules for safe retries:

1. **Only retry transient errors:** timeouts, 503, 429, connection resets. Do not retry 400 or 404; they will fail again.
2. **Only retry idempotent operations**, or use an idempotency key (see Chapter 2). Otherwise you might charge a customer twice.
3. **Use exponential backoff:** wait 1s, then 2s, then 4s, and so on. This gives the struggling server time to recover.
4. **Add jitter** (a small random amount to each wait), so thousands of clients do not all retry at the same instant. This "retry storm" problem is sometimes called the **thundering herd**.
5. **Limit the number of retries**, then fail with a clear error.

```
import random, time

def call_with_retry(func, max_tries=4, base=0.5):
    for attempt in range(max_tries):
        try:
            return func()
        except TimeoutError:
            if attempt == max_tries - 1:
                raise
            wait = base * (2 ** attempt)
            time.sleep(wait + random.uniform(0, wait))

# If func fails twice with TimeoutError and then succeeds,
# call_with_retry returns its result after roughly 0.5-1 s + 1-2 s of waiting.
```

Race conditions, timing and flaky tests

Now we can connect everything to your daily work. Most flaky tests come from one of these timing problems.

1. The test races the application. Your test clicks "Save" and immediately checks for the "Saved!" message. The app sends an API call and updates the DOM later, through the browser's event loop. Sometimes the check runs first. This is a race condition between your test and the app.

- Bad fix: `time.sleep(3)`. Too short on a slow CI machine (still flaky), too long everywhere else (slow suite).
- Good fix: **wait for a condition**. Playwright's auto-waiting and web-first assertions (`expect(locator).to_have_text("Saved!")`) retry until the condition is true or a timeout is reached. In Selenium, use explicit waits (`WebDriverWait` with expected conditions).

2. Tests race each other. Tests running in parallel share the same user account, database row or file. Test A changes the user's address while test B checks it. The result depends on which runs first.

- Fix: give each test its **own data** (unique users, unique IDs per run), and avoid shared global state.

3. Order dependence. Test B only passes if test A ran first and created some data. When the runner changes order or runs them in parallel, B fails.

- Fix: each test sets up and cleans up what it needs. Tools that run tests in random order help reveal this.

4. Asynchronous back-end work. The API returns 202 Accepted, and a background worker finishes the job later. The test checks the result too early. Or a write goes to the database leader, and the read comes from a lagging replica.

- Fix: **poll with a timeout** until the expected state appears, and know the system's consistency guarantees.

5. Real time and clocks. Tests that use the current time can fail near midnight, at month end, or in a different time zone on the CI server.

- Fix: inject or freeze the clock in tests, and use UTC.

6. Missing timeouts and bad retries. A test hangs forever on a dead service, or a test framework blindly retries failed tests and hides a real race condition in the product.

- Fix: set timeouts everywhere. Treat "passes on retry" as a **signal to investigate**, not as a pass.

A powerful habit: when a test is flaky, ask "**What two things are racing here?**" Usually you find the test racing the app, or two tests racing each other. And remember that a flaky test can be exposing a **real race condition in the product**, the kind that users will also hit.

A simple way to find flakiness is to run one test many times, in parallel and under load. For example, `pytest --count=100` with the `pytest-repeat` plugin, or Playwright's `--repeat-each=100`. A test

that fails 1 time in 100 is still broken.

Tester's corner

- **Never use fixed sleeps to "fix" flakiness.** Wait for a specific condition with a timeout instead (Playwright web-first assertions, Selenium explicit waits).
- **Isolate test data.** Unique users and IDs per test remove most test-versus-test races when running in parallel.
- **Run new tests many times before merging** (repeat runs, random order, parallel workers) to catch timing bugs early.
- **Hunt for product races on purpose.** Fire concurrent requests at "use once" actions: coupons, last-seat booking, OTP verification, wallet withdrawals.
- **Check timeouts and retries in the product.** Ask: what happens if this dependency hangs? Does the retry use backoff and only on idempotent calls?
- **Treat "passes on retry" as data.** Track how often tests need retries; a rising number is an early warning.
- **Know your tools' concurrency model.** pytest-xdist uses separate processes; asyncio runs on one thread; browser JavaScript runs on an event loop. Each causes different kinds of timing issues.

Key takeaways

- The **CPU** computes; **memory** is fast but small; **disk** is large but slower; **network** is slowest. Most programs spend their time waiting on I/O.
- A **process** has its own memory; **threads** inside a process share memory, which is convenient but risky.
- **Concurrency** is handling many tasks in overlapping time; **parallelism** is running them at the same instant on different cores.
- A **race condition** is when the result depends on uncontrolled timing; **locks** protect critical sections.
- A **deadlock** is a cycle of threads waiting on each other; taking locks in a fixed order prevents it.
- **Async/await** with an event loop handles many I/O tasks on one thread; a blocking call freezes the loop.
- The **Python GIL** limits CPU-bound threading but does not prevent race conditions.
- Use **timeouts** everywhere and **retries with exponential backoff and jitter** only for transient errors on idempotent operations. Most **flaky tests** are timing problems.

Quiz

1. What is the main difference between a process and a thread?
A) Threads have their own memory; processes share memory B) Processes have their own memory; threads in a process share memory C) There is no difference D) Processes only run on one core
2. True or false: Concurrency and parallelism mean exactly the same thing.
3. Why can `count += 1` give the wrong result with multiple threads?

4. Thread 1 holds lock A and waits for lock B. Thread 2 holds lock B and waits for lock A. What is this called?
A) Race condition B) Deadlock C) Context switch D) Starvation by priority
5. Which is the simplest practical way to prevent the deadlock in question 4?
6. You need to make 500 API calls in a Python test setup script. Which tool is a good fit?
A) multiprocessing with 500 processes B) asyncio or a thread pool C) A single loop with `time.sleep(1)` between calls D) None, it cannot be done
7. True or false: Because of the GIL, Python threads can never have race conditions.
8. Which errors should a client usually retry?
A) 400 Bad Request B) 404 Not Found C) 503 Service Unavailable D) 403 Forbidden
9. What would you do? A UI test sometimes fails because the "Order placed" toast is not found. A teammate adds `time.sleep(5)` before the check.
10. What would you do? Two parallel tests both log in as `testuser1` and update the profile. One of them fails randomly.

Answer key

1. **B** - Each process has its own memory space; threads in the same process share memory.
2. **False** - Concurrency is handling many tasks in overlapping time (possibly on one core); parallelism is running tasks at the same instant on multiple cores.
3. **It is not atomic** - It is a read, an add and a write. Another thread can read the old value before the first thread writes, so one update is lost.
4. **B** - Each thread waits for a lock the other holds, so neither can continue. That is a deadlock.
5. **Fixed lock order** - Make every thread acquire locks in the same order (always A then B). This breaks the circular wait. Lock timeouts also help.
6. **B** - API calls are I/O-bound, so asyncio or a thread pool lets many calls wait at the same time. 500 processes would waste memory.
7. **False** - The GIL allows one thread to run bytecode at a time, but threads can still be switched between the read and the write, so shared data still needs locks.
8. **C** - 503 is usually temporary. 400, 403 and 404 will fail again on retry.
9. **Replace the sleep with a condition wait** - Use a web-first assertion or explicit wait for the toast with a sensible timeout. Fixed sleeps are slow and still flaky on slow machines. Also check whether the app itself has a bug in showing the toast.
10. **Isolate test data** - The tests race each other on shared data. Create a unique user per test (or per worker), so parallel tests never touch the same records.

Flashcards

- **Q:** What is a process? — **A:** A running program with its own separate memory space.
- **Q:** What is a thread? — **A:** A unit of execution inside a process that shares memory with the other threads of that process.
- **Q:** What is a context switch? — **A:** The OS saving one thread's state and loading another's so they can take turns on a CPU core.

- **Q:** Concurrency vs parallelism? — **A:** Concurrency is handling many tasks in overlapping time; parallelism is running them at the same instant on different cores.
- **Q:** What is a race condition? — **A:** A bug where the result depends on the uncontrolled timing or order of events.
- **Q:** What does a mutex do? — **A:** It lets only one thread at a time enter a critical section.
- **Q:** What is a deadlock? — **A:** Threads each waiting for a resource another holds, so none can ever proceed.
- **Q:** How do you prevent most deadlocks? — **A:** Acquire locks in the same fixed order everywhere, and use lock timeouts.
- **Q:** What is an event loop? — **A:** A single-threaded scheduler that runs whichever task is ready while others wait on I/O.
- **Q:** What does the Python GIL do? — **A:** It allows only one thread to run Python bytecode at a time in CPython, limiting CPU-bound threading.
- **Q:** What is exponential backoff with jitter? — **A:** Waiting longer after each retry (1s, 2s, 4s) plus a random extra, so clients do not retry all at once.
- **Q:** What is the most common root cause of flaky tests? — **A:** Timing: the test racing the app, or tests racing each other over shared data.

Numbers and Back-of-the-Envelope Estimation

In this chapter:

- Remember the approximate latency numbers that every engineer should know
- Use powers of two and common data sizes without a calculator
- Calculate QPS, storage and bandwidth step by step
- Work through three full estimates: a URL shortener, a photo app and a chat app
- Understand availability "nines" and simple rules of thumb

Time: ~40 minutes | **Level:** Beginner

Why estimation matters

In a system design interview, you will often hear: "Assume 100 million users. How much storage do we need? How many servers?" The interviewer is **not** testing your arithmetic. They want to see if you can:

- Turn vague words ("lots of users") into numbers.
- Make reasonable assumptions and **say them out loud**.
- Find the **bottleneck**: is this system limited by storage, by bandwidth, by database writes, or by open connections?

This is called **back-of-the-envelope estimation**: quick, rough maths that you could do on the back of an envelope. Being within a factor of 2 or even 10 is usually fine. Being off by a factor of 1,000 is not, because that changes the design.

Analogy: before planning a wedding, you estimate "about 500 guests, so about 50 tables and food for 550". You do not count every guest's name first. You just need the right size of hall.

Latency numbers every engineer should know

Latency is the time one operation takes. The table below shows the classic list made popular by Jeff Dean (Google) and Peter Norvig around 2010. Hardware has changed since then (memory and SSDs are faster now), so treat these as **approximate orders of magnitude**, not exact facts. What matters is how they compare.

Operation	Approximate time
L1 CPU cache reference	0.5 ns
Branch mispredict	5 ns
L2 CPU cache reference	7 ns

Operation	Approximate time
Main memory (RAM) reference	100 ns
Send 1 KB over a 1 Gbps network	10 μ s
Read 4 KB randomly from an SSD	~150 μ s
Read 1 MB sequentially from memory	~250 μ s
Round trip within the same data center	500 μ s
Read 1 MB sequentially from an SSD	~1 ms
Hard disk seek	10 ms
Read 1 MB sequentially from a hard disk	~20 ms
Send a packet California to Netherlands and back	150 ms

Units: 1 ns (nanosecond) = one billionth of a second. 1 μ s (microsecond) = 1,000 ns. 1 ms (millisecond) = 1,000 μ s.

To feel the difference, imagine 1 ns is **1 second**. Then:

- An L1 cache read is half a second.
- A RAM read is nearly 2 minutes.
- An SSD random read is about 2 days.
- A hard disk seek is about 4 months.
- A round trip from California to the Netherlands is about 5 years.

The lessons from this table:

1. **Memory is much faster than disk.** This is why we use caches.
2. **Sequential reads are much faster than random reads**, especially on hard disks.
3. **Network calls inside a data center are cheap compared to calls across continents.** This is why we put servers and CDNs close to users.
4. **You cannot beat the speed of light.** A round trip from India to the USA will always take well over 100 ms. No code optimisation fixes that; only moving data closer does.

Powers of two and data sizes

Computers count in powers of two. For estimation, you can treat 2^{10} as roughly 1,000.

Power	Exact value	Approximate	Name
2^{10}	1,024	1 thousand	1 KB (kilobyte)
2^{20}	1,048,576	1 million	1 MB (megabyte)
2^{30}	about 1.07 billion	1 billion	1 GB (gigabyte)
2^{40}	about 1.1 trillion	1 trillion	1 TB (terabyte)

Power	Exact value	Approximate	Name
2 ⁵⁰	about 1.13 quadrillion	10 ¹⁵	1 PB (petabyte)

Strictly, 1,024 bytes is a "KiB" (kibibyte) and 1,000 bytes is a "KB", but in interviews everyone rounds, and the 2-7% difference does not change the design.

Common sizes to remember (approximate):

Thing	Size
One ASCII character	1 byte
One Hindi (Devanagari) character in UTF-8	3 bytes
An integer (int32)	4 bytes
A long integer or timestamp (int64)	8 bytes
A UUID (binary)	16 bytes (36 bytes as text)
A short text message with metadata	a few hundred bytes
A compressed phone photo	about 1-5 MB
A small thumbnail image	tens of KB

Also remember **1 byte = 8 bits**. Network speeds are in **bits** per second (Mbps, Gbps), but file sizes are in **bytes**. A 100 Mbps connection moves about 12.5 MB per second.

Time numbers

- 1 day = 86,400 seconds. Round it to **about 100,000 (10⁵) seconds**. This makes division easy.
- 1 month is about 2.5 million seconds.
- 1 year is about 31.5 million seconds (round to 3×10^7).
- 1 year is about 365 days; for storage, "times 400" is a fine quick upper estimate, or "times 365" if you want closer numbers.

Calculating QPS

QPS means **queries per second** (also called RPS, requests per second). It tells you how busy the servers are.

The basic formula:

```
average QPS = (number of requests per day) / 86,400
peak QPS    = average QPS x peak factor    (often 2 to 5)
```

Traffic is not flat. People use apps more in the evening, and spikes happen (a cricket final, a flash sale). So we multiply by a **peak factor**. Say which factor you chose and why.

Example: 10 million daily active users (**DAU**), each doing 20 requests per day.

- Requests per day = $10,000,000 \times 20 = 200,000,000$.
- Average QPS = $200,000,000 / 100,000$ (rounded) = about 2,000. (Exact: $200,000,000 / 86,400 =$ about 2,315.)
- Peak QPS with a factor of 3 = about 6,000-7,000.

Also separate **reads** from **writes**. Most systems are **read-heavy**: many people view a post, few people write one. The **read-to-write ratio** decides whether to focus on caching (read-heavy) or on fast database writes (write-heavy).

Storage and bandwidth estimation

Storage formula:

```
storage per day = new items per day x size of one item
storage total   = storage per day x days kept x replication factor
```

The **replication factor** is how many copies we keep (often 3, for safety, see the databases chapter).

Bandwidth formula:

```
bandwidth = QPS x size of one request or response
```

Do this separately for **ingress** (data coming in, like uploads) and **egress** (data going out, like downloads). Egress is usually much bigger, and it is what CDNs help with.

Here is a small Python helper you can use to check your numbers while practising:

```
SECONDS_PER_DAY = 86_400

def estimate(per_day, item_bytes, read_ratio=1, peak=3, years=5, copies=3):
    write_qps = per_day / SECONDS_PER_DAY
    read_qps = write_qps * read_ratio
    storage_tb = per_day * item_bytes * 365 * years * copies / 1e12
    return round(write_qps), round(read_qps), round(read_qps * peak), round(storage_tb)

# URL shortener: 100M new URLs/day, 500 bytes each, 100 reads per write
print(estimate(100_000_000, 500, read_ratio=100, copies=1))
# Expected output: (1157, 115741, 347222, 91)
# write QPS, read QPS, peak read QPS, storage in TB (no replication)
```

Now let us do three full examples by hand. Notice the same pattern every time: **assumptions, traffic, storage, bandwidth, then what it means for the design.**

Worked example 1: a URL shortener

A URL shortener turns a long link into a short one, like `sho.rt/aB3x9Kp`, and redirects people who visit it.

Step 1 - Assumptions (say these out loud):

- 100 million new short URLs created per day.
- Read-to-write ratio 100:1 (each link is clicked about 100 times).
- Each record (short code, long URL, creation time, user ID) is about 500 bytes.
- Keep data for 5 years.

Step 2 - Traffic:

- Write QPS = $100,000,000 / 86,400$ = about **1,160 writes per second**.
- Read QPS = $1,160 \times 100$ = about **116,000 reads per second**.
- Peak (factor 2) = about 2,300 writes and 230,000 reads per second.

Step 3 - Storage:

- Per day = $100,000,000 \times 500$ bytes = 50,000,000,000 bytes = **50 GB per day**.
- Per year = $50 \text{ GB} \times 365$ = about **18 TB**.
- 5 years = about **91 TB**, before replication. With 3 copies, about 275 TB.

Step 4 - How long must the short code be?

- Total URLs in 5 years = $100 \text{ million} \times 365 \times 5$ = about **183 billion**.
- Using 62 characters (a-z, A-Z, 0-9), called **base62**:
 - 6 characters: 62^6 = about 57 billion. **Not enough**.
 - 7 characters: 62^7 = about 3.5 trillion. **Enough**, with lots of room.
- So use **7 characters**.

Step 5 - Bandwidth:

- Read egress = $116,000 \times 500$ bytes = about **58 MB per second** (around 460 Mbps). Small for a modern data center.

Step 6 - Memory for a cache:

- Use the **80/20 rule**: about 20% of links get most of the clicks. Caching 20% of one day's new links = $0.2 \times 50 \text{ GB}$ = **10 GB**. That fits in the memory of one large cache server (or a small cluster).

What it means for the design: The system is very **read-heavy** (116,000 reads per second), so a cache in front of the database is the key component. Storage of about 100 TB means one machine is not enough over 5 years, so the data must be split across machines (sharding, covered in a later lesson). Bandwidth is not the bottleneck.

Worked example 2: a photo-sharing app

Step 1 - Assumptions:

- 10 million daily active users.
- 10% of users upload 2 photos a day, so 2 million new photos per day.
- Average stored photo: 2 MB. (We also store small versions, which we will add as extra.)
- Each user views 50 photos per day, served as a resized version of about 200 KB.
- Keep photos forever; 3 copies for safety.

Step 2 - Traffic:

- Upload QPS = $2,000,000 / 86,400$ = about **23 uploads per second**. Peak (factor 3) about 70.
- Views per day = $10,000,000 \times 50$ = 500 million.
- View QPS = $500,000,000 / 86,400$ = about **5,800 per second**. Peak about 17,000.
- Read-to-write ratio = 500 million / 2 million = **250:1**.

Step 3 - Storage:

- Photos per day = $2,000,000 \times 2$ MB = **4 TB per day**.
- Add about 20% for resized versions and thumbnails = about 4.8 TB per day.
- Per year = 4.8 TB $\times$ 365 = about **1.75 PB**.
- With 3 copies = about **5.3 PB per year**.
- Metadata (owner, caption, time, location) at about 1 KB per photo = $2,000,000 \times 1$ KB = **2 GB per day**, under 1 TB per year. Very small compared to the images.

Step 4 - Bandwidth:

- Ingress (uploads) = 23×2 MB = about **46 MB per second**.
- Egress (views) = $5,800 \times 200$ KB = about **1.16 GB per second**, which is about **9.3 Gbps** on average, and nearly 3 times that at peak.

What it means for the design: The biggest numbers are **image storage** (petabytes) and **egress bandwidth** (tens of Gbps at peak). So photos go into a cheap, durable **object store** (blob storage), not into a relational database. Metadata is small and can live in a normal database. Egress is large, so a **CDN** should serve images from cities close to users. Upload QPS is tiny; it is not where the difficulty is.

Worked example 3: a chat app

Step 1 - Assumptions:

- 50 million daily active users.
- Each user sends 40 messages per day.
- Each message with metadata (IDs, timestamp, status) is about 200 bytes.
- At peak, 20% of daily users are online at the same time.
- Keep messages for 1 year on the server; 3 copies.

Step 2 - Traffic:

- Messages per day = $50,000,000 \times 40$ = **2 billion**.
- Message QPS = $2,000,000,000 / 86,400$ = about **23,000 per second**. Peak (factor 3) about **70,000 per second**.

- For one-to-one chat, each message is also delivered once, so delivery QPS is similar. Group chats multiply delivery: a message to a 100-person group is 100 deliveries.

Step 3 - Storage:

- Per day = $2,000,000,000 \times 200$ bytes = **400 GB per day**.
- Per year = $400 \text{ GB} \times 365$ = about **146 TB**.
- With 3 copies = about **440 TB**.

Step 4 - Bandwidth:

- Incoming = $23,000 \times 200$ bytes = about **4.6 MB per second**. Even at peak, this is small.

Step 5 - Connections:

- Online at peak = $20\% \times 50$ million = **10 million open connections** (for example WebSockets, see Chapter 2).
- If one server can hold about 50,000 connections (an assumption; the real number depends on hardware and tuning), you need $10,000,000 / 50,000$ = about **200 connection servers**, plus spare capacity.

What it means for the design: Bandwidth is small. The hard parts are **millions of open connections** and a **high, steady write rate** (tens of thousands of messages per second), which suits a database built for heavy writes and time-ordered data (like the wide-column stores from the databases chapter). Group chats increase delivery work, so the fan-out must be handled carefully.

Notice how three apps gave three different bottlenecks: **reads and caching** for the URL shortener, **storage and egress** for photos, and **connections and writes** for chat. That is the real point of estimation: it tells you where to spend your design effort.

Availability and the "nines"

Availability is the percentage of time a system works correctly. It is often written as a number of "nines".

Availability	Downtime per year	Downtime per month (30 days)	Downtime per day
99% ("two nines")	about 3.65 days	about 7.2 hours	about 14.4 minutes
99.9% ("three nines")	about 8.76 hours	about 43.2 minutes	about 1.44 minutes
99.95%	about 4.38 hours	about 21.6 minutes	about 43 seconds
99.99% ("four nines")	about 52.6 minutes	about 4.3 minutes	about 8.6 seconds
99.999% ("five nines")	about 5.26 minutes	about 26 seconds	under 1 second

Each extra nine cuts allowed downtime by 10 times, and usually costs much more money and effort. Google's public SRE book makes this point: 100% is the wrong target, because users cannot tell the difference beyond a certain level (their own phone and network fail more often), and chasing it slows

down releases. The SRE book calls the allowed unreliability an **error budget**. (SLIs, SLOs and SLAs are covered in a later lesson.)

Two quick calculations:

- **Components in a chain multiply.** If a request needs service A (99.9%) **and** service B (99.9%), the total is $0.999 \times 0.999 =$ about **99.8%**. More dependencies in a chain mean lower availability.
- **Redundant copies help.** If you have two independent copies, each 99% available, and either one is enough, the chance both are down is $0.01 \times 0.01 = 0.0001$. So availability is **99.99%**. (This assumes failures are independent; in real life, a shared power supply or a bad deploy can take both down together.)

Rules of thumb

- **Round aggressively.** 86,400 becomes 100,000. 365 becomes 400 when you want a safe upper estimate. Use powers of ten.
- **State assumptions first**, and write them down. The interviewer can correct them, which is good.
- **Separate reads and writes**, and calculate peak, not just average.
- **Multiply by the replication factor** for storage (usually 3).
- **The 80/20 rule:** about 20% of data gets about 80% of traffic, which helps you size caches.
- **Sanity check** each result. Does 5 PB per year for a photo app with 10 million users feel reasonable? Compare with what one disk holds (tens of TB in a big hard disk today).
- **Memory is much faster than SSD, and SSD is much faster than hard disk.** In the latency table, a random RAM read is over 1,000 times faster than a random SSD read. Network across continents is the slowest of all.
- **End with a design decision.** Every estimate should finish with "so this means we need a cache / a CDN / sharding / many connection servers".

Tester's corner

- **Use estimation to plan performance and load tests.** If production peak is 70,000 messages per second, a load test at 500 per second proves very little. Target the estimated peak, then go beyond it.
- **Estimate test infrastructure too.** 20,000 tests $\times$ 30 seconds each = 600,000 seconds, which is about 7 days on one machine. To finish in 10 minutes (600 seconds), you need about 1,000 parallel workers. This is the kind of reasoning used in "design a test platform" questions.
- **Latency numbers explain slow tests.** A test that calls a service in another region 200 times pays about 150 ms each time, which is 30 seconds of pure waiting. Run tests close to the services, or batch calls.
- **Check limits and boundaries** from the estimates: the maximum short-code length, the number of open connections a server can hold, the largest file upload. These limits become test cases.
- **Availability maths applies to test pipelines.** If your CI depends on 5 services that are each 99% available, the pipeline works only about 95% of the time (0.99^5). That is a lot of "flaky" red builds that are really infrastructure.

- **Translate SLOs into test assertions.** "99% of requests under 300 ms" becomes a measurable performance check, rather than "it should be fast".

Key takeaways

- Estimation is about **finding the bottleneck** with reasonable assumptions, not about exact numbers.
- Memory is far faster than disk, and cross-continent network round trips are the slowest; the classic latency table is **approximate**.
- Use powers of two (2^{10} is about 1 thousand, 2^{20} about 1 million, 2^{30} about 1 billion), and remember 1 byte = 8 bits.
- **QPS = requests per day / 86,400** (about 10^5), then multiply by a peak factor.
- **Storage = items per day × size × days × replicas; bandwidth = QPS × size**, for ingress and egress separately.
- Different apps have different bottlenecks: reads (URL shortener), storage and egress (photos), connections and writes (chat).
- Each extra nine of availability is 10 times less downtime; components in a chain multiply their availability.

Quiz

1. About how many seconds are in a day, rounded for quick estimation?
A) 1,000 B) 10,000 C) 100,000 D) 1,000,000
2. True or false: Reading from main memory is roughly as fast as a hard disk seek.
3. An app gets 864 million requests per day. What is the average QPS?
4. Which is the best way to describe 2^{30} bytes for estimation?
A) About 1 MB B) About 1 GB C) About 1 TB D) About 1 PB
5. A network link is 800 Mbps. About how many megabytes per second can it move?
6. How much downtime per year does 99.9% availability allow?
A) About 5 minutes B) About 52 minutes C) About 8.8 hours D) About 3.7 days
7. A request needs three services in a chain, each 99% available. What is the approximate total availability?
8. True or false: In the photo-sharing example, upload QPS was the main bottleneck.
9. What would you do? Your team plans a load test of a chat service at 1,000 messages per second, but your estimate of peak traffic is 70,000 per second.
10. What would you do? Your CI suite has 12,000 tests averaging 20 seconds each, and the team wants results in 15 minutes.

Answer key

1. **C** - 86,400 seconds, rounded to about 100,000 (10^5) for easy division.
2. **False** - A memory reference is about 100 ns, and a disk seek is about 10 ms, which is roughly 100,000 times slower.
3. **10,000 QPS** - $864,000,000 / 86,400 = 10,000$ requests per second on average. Peak will be higher.

4. **B** - 2^{30} is about 1.07 billion, so 2^{30} bytes is about 1 GB.
5. **About 100 MB per second** - Divide bits by 8: $800 / 8 = 100$ MB per second (in practice a bit less because of protocol overhead).
6. **C** - 0.1% of 8,760 hours in a year is about 8.76 hours.
7. **About 97%** - $0.99 \times 0.99 \times 0.99 =$ about 0.970. Chained dependencies reduce availability.
8. **False** - Uploads were only about 23 per second. The bottlenecks were petabyte-scale storage and large egress bandwidth, which is why we used object storage and a CDN.
9. **Raise the target** - A test at 1/70th of peak cannot show how the system behaves under real load. Plan stages up to the estimated peak and above it (for example 1.5 times peak), and watch latency, errors and connection counts.
10. **Calculate the parallelism** - Total work is $12,000 \times 20 = 240,000$ seconds. To finish in 900 seconds, you need about $240,000 / 900 =$ about 267 parallel workers, plus some extra for setup time and uneven test lengths. Also look at removing or speeding up the slowest tests.

Flashcards

- **Q:** How many seconds are in a day, rounded? — **A:** About 100,000 (exactly 86,400).
- **Q:** How do you calculate average QPS? — **A:** Divide requests per day by 86,400, then multiply by a peak factor for peak QPS.
- **Q:** About how long is a main memory reference? — **A:** About 100 nanoseconds (approximate).
- **Q:** About how long is a round trip from California to the Netherlands? — **A:** About 150 milliseconds (approximate).
- **Q:** What are 2^{10} , 2^{20} , 2^{30} and 2^{40} roughly? — **A:** About a thousand, a million, a billion and a trillion (KB, MB, GB, TB).
- **Q:** How do you convert Mbps to MB per second? — **A:** Divide by 8, because 1 byte is 8 bits.
- **Q:** What is the storage formula? — **A:** Items per day $\times$ item size $\times$ days kept $\times$ replication factor.
- **Q:** What is the bandwidth formula? — **A:** QPS $\times$ size of each request or response, for ingress and egress separately.
- **Q:** Why did the URL shortener need 7 base62 characters? — **A:** 62^6 is about 57 billion, less than the 183 billion links in 5 years; 62^7 is about 3.5 trillion.
- **Q:** How much downtime per year does 99.99% allow? — **A:** About 52.6 minutes.
- **Q:** What happens to availability when services are chained? — **A:** Their availabilities multiply, so the total is lower than any single one.
- **Q:** What is the 80/20 rule in caching? — **A:** About 20% of the data gets about 80% of the traffic, so caching that 20% gives most of the benefit.

Building block 1: How to approach a design interview

What the interviewer wants

A design interview is not a quiz with one right answer. The interviewer wants to see how you think. They check if you can take a vague problem, ask good questions, make a simple design, and then improve it.

They also check if you can explain trade-offs clearly. Talking is part of the test.

The 45-minute framework

1. Requirements (5 min): ask about features, users, scale, and what matters most (speed, consistency, cost).
2. Estimates (5 min): users, requests per second, storage per day and per year, bandwidth.
3. API design (5 min): list 3-5 main endpoints with inputs and outputs.
4. Data model (5 min): main tables or objects, keys, and the database type.
5. High-level design (10 min): draw boxes and arrows, then walk through one request end to end.
6. Deep dive (10 min): pick the 1-2 hardest parts and go deep.
7. Wrap up (5 min): bottlenecks, failures, monitoring, and what you would do next.

Real example

Question: "Design a URL shortener." Do not start drawing. First ask: How many new URLs per day? Do links expire? Custom aliases? Analytics?

Then say: "100M new URLs per day is about 1,200 writes per second. Reads are 10x more, so about 12,000 reads per second. This is read-heavy, so caching matters."

Now every later decision has a reason.

```
Client -> Load Balancer -> API Servers -> Cache -> Database
```

When to use it

- Use this structure for every design question, even small ones.
- If the interviewer pushes you to a deep dive early, follow them. The framework is a guide, not a cage.

Trade-offs in time

- Too long on requirements means no time for deep dive.
- Skipping estimates means you cannot justify caching, sharding, or database choice.
- Say your assumptions out loud, and write them down.

Common follow-up questions

- "What happens if this server dies?" Answer: explain replication, health checks, and failover.
- "How does this scale to 10x?" Answer: find the first bottleneck (usually the database), then shard, cache, or add replicas.
- "Why this database?" Answer: link it to access patterns and consistency needs.
- "What would you build first?" Answer: the simplest version that meets core requirements, then iterate.

Tips for clear communication

- Use short sentences: "I will add a cache here because reads are 10x writes."
- Check in often: "Does this direction make sense, or should I go deeper somewhere?"
- If you do not know something, say how you would find out.

How a test engineer should think about it

Treat the design like a feature under test: for every box, ask "how can this fail, and how would I detect it?" and mention it in the wrap up.

Videos

- [System design interview: step-by-step guide](#) - ByteByteGo (English)
- [5 tips for system design interviews](#) - Gaurav Sen (English)

Further reading

- [System Design Primer - how to approach](#)

Building block 2: Scalability basics

What is scalability

Scalability means the system can handle more load by adding resources, without a redesign. Load can be more users, more requests per second, or more data.

Vertical vs horizontal scaling

- Vertical scaling (scale up): buy a bigger machine with more CPU, RAM, and disk. Simple, but there is a hard limit and it is a single point of failure.
- Horizontal scaling (scale out): add more machines behind a load balancer. No hard limit and better fault tolerance, but more complex.
- Large systems at Google scale are almost always horizontal.

Stateless services

A stateless server keeps no user data in memory between requests. Any server can handle any request. Session data goes to a shared store like Redis or a database, or into a signed token (JWT). Stateless servers make horizontal scaling easy: just add more servers.

```
Client -> LB -> [API 1, API 2, API 3] -> Redis (sessions) + DB
```

Latency vs throughput

- Latency: time for one request (for example 120 ms). Measure p50, p95, p99, not only average.
- Throughput: how many requests per second the system handles (for example 5,000 QPS).
- They are related but different. Batching can increase throughput but also increase latency.

Latency numbers to remember (approximate)

- L1 cache: 1 ns. Main memory: 100 ns.
- Read 1 MB from memory: 250 us. Read 1 MB from SSD: 1 ms.
- Round trip in same data center: 0.5 ms.
- Disk seek (HDD): 10 ms.
- Round trip US to Europe: 150 ms.
- Lesson: memory is fast, network across regions is slow, disk seeks are slow.

Real example

An e-commerce site has 1 server. During a sale, CPU hits 100%. Short term: scale up. Long term: make the app stateless, put 10 servers behind a load balancer, and add read replicas for the database.

Trade-offs

- Horizontal scaling adds network calls, partial failures, and data consistency problems.
- Vertical scaling is cheaper to operate at small scale.

Common follow-up questions

- "Why p99 and not average?" Answer: average hides slow requests. 1% of users with 5 s latency is still a bad experience.
- "How do you handle sessions with many servers?" Answer: external session store or stateless tokens; avoid sticky sessions if possible.
- "What is the first bottleneck usually?" Answer: the database, because app servers are easy to copy.

How a test engineer should think about it

Load test to find the breaking point, track p95/p99 latency and error rate as QPS grows, and verify that adding servers really increases throughput linearly.

Videos

- [Horizontal vs vertical scaling](#) - Gaurav Sen (English)

Further reading

- [System Design Primer - scalability](#)

Building block 3: Load balancing

What a load balancer does

A load balancer (LB) sits in front of many servers and spreads requests across them. It improves capacity and availability. If one server dies, the LB stops sending traffic to it.

L4 vs L7

- L4 (transport layer): routes by IP and port. Does not read the HTTP request. Very fast and cheap.
- L7 (application layer): reads HTTP path, headers, cookies. Can route `/api` to one pool and `/images` to another. Can do TLS termination, auth checks, and rewrites.
- Many systems use both: L4 at the edge, L7 inside.

Algorithms

- Round robin: next server in order. Simple, good when servers are equal.
- Weighted round robin: bigger servers get more traffic.
- Least connections: send to the server with fewest active connections. Good for long requests.
- IP hash: same client goes to same server (sticky).
- Consistent hashing: used when the same key should go to the same server, like cache nodes.

Consistent hashing in simple words

Put servers on a ring (0 to 2^{32}). Hash the key, walk clockwise, and pick the first server. When a server is added or removed, only about $1/N$ of keys move, not all of them. Use virtual nodes (each server appears many times on the ring) to spread load evenly.

Health checks

- Active: LB calls `/health` every few seconds. After 3 failures, remove the server.
- Passive: LB watches real traffic errors and timeouts.
- A good health check tests real dependencies lightly, but should not fail just because one optional dependency is slow.

Real example

```
Users -> DNS -> L4 LB -> L7 LB -> [web-1, web-2, web-3]
```

A deploy goes wrong on web-2. Its /health starts returning 500. The LB removes it within 10 seconds, and users see no errors.

Trade-offs

- The LB itself can be a single point of failure. Use a pair (active-passive) or a managed LB.
- Sticky sessions make scaling and deploys harder.
- L7 gives more features but costs more CPU and latency.

Common follow-up questions

- "What if the LB fails?" Answer: run redundant LBs with a floating IP or DNS failover.
- "How do you drain a server for deploy?" Answer: mark it unhealthy, wait for active requests to finish, then stop it.
- "Why consistent hashing instead of hash mod N?" Answer: mod N remaps almost all keys when N changes.

How a test engineer should think about it

Kill a backend during a load test and verify zero or few failed requests, check traffic is evenly spread, and test that health checks remove and re-add servers correctly.

Videos

- [What is a load balancer?](#) - IBM Technology (English)
- [Load balancer in system design](#) - Gate Smashers (Hindi)

Further reading

- [System Design Primer - load balancer](#)

Building block 4: Caching

What is a cache

A cache stores copies of data in fast storage (usually memory) so we do not hit the slow source every time. It reduces latency and reduces load on the database.

Where to cache

- Client / browser cache (HTTP Cache-Control headers).
- CDN: static files and videos close to users.
- Application cache: Redis or Memcached in front of the database.
- Database cache: buffer pool inside the database.

Write and read patterns

- Cache-aside (lazy loading): app reads cache; on miss, reads DB and writes to cache. Most common.
- Write-through: app writes to cache and DB together. Cache is always fresh, but writes are slower.
- Write-back (write-behind): app writes to cache only; cache writes to DB later. Very fast writes, but data can be lost if the cache crashes.

```
Read: App -> Cache (hit? return) -> miss -> DB -> put in Cache -> return
```

Eviction policies

- LRU (least recently used): most common.
- LFU (least frequently used): good when some items are always popular.
- TTL: each key expires after a fixed time.

Stale data

When the DB changes, the cache may still have the old value. Options: set a TTL, delete the cache key on write, or use change events to invalidate.

Delete on write is usually safer than update on write, because two updates can race.

Thundering herd (cache stampede)

A hot key expires, and 10,000 requests miss at the same moment and all hit the DB.

Fixes: a lock so only one request rebuilds the value, random jitter on TTL, or refresh the key before it expires.

Real example

A product page is read 50,000 times per second but changes once per hour. Cache-aside with a 5 minute TTL removes over 99% of DB reads.

Trade-offs

- Faster reads vs risk of stale data.
- Memory is expensive; cache only hot data.
- Another component that can fail. The system must still work (slower) if the cache is down.

Common follow-up questions

- "What hit ratio is good?" Answer: depends, but 90%+ is common for read-heavy data.
- "What if Redis goes down?" Answer: fall back to DB with rate limiting, and use replicas for Redis.
- "How do you cache user-specific data?" Answer: key by user ID, shorter TTL, never put private data in a shared CDN.

How a test engineer should think about it

Test the cache-miss path, the stale-data window after an update, cache-down behaviour, and measure hit ratio and DB load in performance tests.

Videos

- [Cache systems every developer should know](#) - ByteByteGo (English)
- [Caching in system design interviews](#) - Hello Interview (English)

Further reading

- [System Design Primer - cache](#)

Building block 5: Databases: SQL vs NoSQL

SQL databases

SQL (relational) databases store data in tables with a fixed schema. Examples: PostgreSQL, MySQL, Spanner.

They support joins and transactions with ACID guarantees.

- Atomicity: all steps succeed or none.
- Consistency: data always follows the rules (constraints).
- Isolation: parallel transactions do not see half-done work.
- Durability: once committed, data survives a crash.

NoSQL databases

NoSQL databases trade some guarantees for scale and flexibility. Many follow BASE: Basically Available, Soft state, Eventually consistent.

- Key-value (Redis, DynamoDB): simple get/put by key. Sessions, carts, caches.
- Document (MongoDB, Firestore): JSON-like documents. Product catalogs, user profiles.
- Wide-column (Cassandra, Bigtable): huge write volume, time series, chat messages.
- Graph (Neo4j): relationships like friends of friends.

Indexes

An index is like a book index. It makes reads by a column fast (B-tree, $O(\log n)$) instead of a full scan. Cost: extra storage and slower writes, because every write also updates the index.

Index the columns you filter and sort by often.

Replication

- Leader-follower: one leader takes writes, followers copy data and serve reads. Followers may lag (read-your-own-write problems).
- Multi-leader: writes in many regions, but conflicts need resolution.
- Leaderless (quorum): write to W nodes, read from R nodes.
- Synchronous replication is safer but slower; async is faster but can lose recent writes on failover.

Real example

A bank uses SQL for account balances because money transfers need ACID. The same bank stores app click events in Bigtable because it gets millions of writes per second and does not need joins.

When to use which

- Use SQL when data is relational, you need transactions, or you are not sure. It is a safe default.
- Use NoSQL when scale is very large, the schema changes often, or access is simple by key.

Common follow-up questions

- "Can SQL scale?" Answer: yes, with read replicas and sharding; Spanner scales globally with strong consistency.
- "Why is my query slow?" Answer: check the query plan, missing index, or a full table scan.
- "What is replication lag?" Answer: the delay before a follower has the latest write.

How a test engineer should think about it

Test data integrity under concurrent writes, failover with no data loss, migration scripts on real-sized data, and reads from replicas right after writes.

Videos

- [SQL vs NoSQL in 4 minutes](#) - Exponent (English)

Further reading

- [System Design Primer - database](#)

Building block 6: Sharding and partitioning

What is sharding

Sharding splits one large dataset across many database machines. Each machine (shard) holds a part of the data.

We shard when one machine cannot hold the data or handle the write traffic.

Partitioning is the general word; sharding usually means partitions on different machines.

Choosing a shard key

- Range-based: users A-M on shard 1, N-Z on shard 2. Good for range queries, but can be uneven.
- Hash-based: $\text{shard} = \text{hash}(\text{user_id}) \% N$. Even spread, but range queries hit all shards.
- Directory-based: a lookup service maps key to shard. Flexible, but the directory is another component.

A good key spreads load evenly and keeps data that is read together on the same shard.

Consistent hashing

With hash mod N , adding a shard moves almost all keys. With consistent hashing, only about $1/N$ of keys move.

Virtual nodes give smoother distribution.

Hot keys

A hot key is one key with huge traffic, for example a celebrity account or a viral video.

Fixes: cache the hot key, add a random suffix to split it across shards (key#1..key#10), or give it a dedicated shard.

Rebalancing

- When adding shards, data must move. Do it in the background while serving traffic.
- Use many small logical partitions (for example 1,024) mapped to fewer machines. Moving a partition is easier than splitting data.
- Throttle the copy so it does not hurt live traffic.

```
App -> Shard Router -> hash(user_id) -> [Shard 0, Shard 1, Shard 2, Shard 3]
```

Real example

A chat app shards messages by `conversation_id`. All messages of one chat live on one shard, so loading a chat is a single-shard query.

Trade-offs

- Cross-shard joins and transactions are hard and slow.
- Wrong shard key is very costly to change later.
- Operations become harder: backups, schema changes, monitoring per shard.

Common follow-up questions

- "How do you query across shards?" Answer: scatter-gather to all shards and merge, or keep a separate index.
- "How to get a global unique ID?" Answer: Snowflake-style IDs (time + machine + sequence).
- "When should you shard?" Answer: as late as possible; first try indexes, caching, and read replicas.

How a test engineer should think about it

Test even data distribution, cross-shard queries, rebalancing with live traffic and no data loss, and the behaviour when one shard is down.

Videos

- [Database sharding and partitioning](#) - Arpit Bhayani (English)
- [What is database sharding?](#) - Anton Putra (English)

Further reading

- [System Design Primer - sharding](#)

Building block 7: Message queues and async processing

Why message queues

A message queue lets one service send work to another without waiting. The producer puts a message; a consumer processes it later.

Examples: Kafka, RabbitMQ, Google Pub/Sub, Amazon SQS.

Benefits: decoupling, smoothing traffic spikes, and retrying failed work.

```
API -> Queue -> [Worker 1, Worker 2, Worker 3] -> DB
```

Delivery guarantees

- At-most-once: message may be lost, never duplicated.
- At-least-once: message is never lost, but may be delivered twice. Most common.
- Exactly-once: very hard in practice; usually achieved as at-least-once plus idempotent processing.

Idempotency

Idempotent means doing the same thing twice gives the same result as once.

Example: store processed message IDs; if the ID is already there, skip it. Or use "set balance = 100" instead of "add 10".

With at-least-once delivery, consumers must be idempotent.

Queue vs pub/sub

- Queue (point to point): each message goes to one consumer. Good for job processing.
- Pub/sub: each message goes to every subscriber group. Good for events like "order placed" that email, billing, and analytics all need.

Dead letter queue (DLQ)

If a message fails many times (for example 5 retries), move it to a DLQ. This stops one bad message from blocking the queue.

Engineers inspect the DLQ, fix the bug, and replay messages.

Back pressure

When consumers are slower than producers, the queue grows. Back pressure means slowing producers down: reject requests, return 429, or scale consumers.

Monitor queue depth and the age of the oldest message.

Real example

A user uploads a video. The API saves the file and puts a "transcode" message in a queue. It returns "processing" immediately. Workers transcode in the background.

Trade-offs

- Better resilience and throughput, but eventual consistency and harder debugging.
- Message ordering is only guaranteed within a partition (Kafka) or not at all.

Common follow-up questions

- "How do you keep order?" Answer: partition by key (for example user_id) so one key goes to one partition.
- "What if the consumer crashes mid-work?" Answer: message is not acked, so it is redelivered after a timeout.
- "Kafka vs RabbitMQ?" Answer: Kafka is a durable log with replay and high throughput; RabbitMQ is a classic broker with flexible routing.

How a test engineer should think about it

Send duplicate and out-of-order messages, kill consumers mid-processing, fill the queue to test back pressure, and verify poison messages reach the DLQ.

Videos

- [What is a message queue?](#) - Gaurav Sen (English)
- [Message queues in system design interviews](#) - Hello Interview (English)

Further reading

- [System Design Primer - asynchronism](#)

Building block 8: CAP theorem and consistency

CAP theorem in simple words

In a distributed system, a network partition (P) means some nodes cannot talk to others. When a partition happens, you must choose:

- Consistency (C): every read gets the latest write, or an error.
 - Availability (A): every request gets a response, but it may be old data.
- Partitions will happen, so the real choice is CP or AP during a partition.

CP vs AP examples

- CP: bank balances, inventory for the last item, leader election (Spanner, ZooKeeper, etcd).
- AP: social media likes, shopping cart, DNS (Cassandra, DynamoDB default).

Consistency models

- Strong consistency: after a write, all readers see it. Simple to reason about, but slower.
- Eventual consistency: readers may see old data for a short time, but all copies agree later.
- Read-your-own-writes: a user always sees their own changes, even if others do not yet.
- Monotonic reads: a user never sees data go back in time.

Quorum

With N copies of data, write to W nodes and read from R nodes.

If $R + W > N$, every read overlaps with the latest write, so you get strong reads.

Example: $N = 3$, $W = 2$, $R = 2$. One node can be down and the system still works with strong reads.

$W = 1$, $R = 1$ is fast but only eventually consistent.

```
Client -> Coordinator -> [Replica A (ok), Replica B (ok), Replica C (down)] W=2 met
```

PACELC

An extension: if Partition, choose A or C; Else (normal time), choose Latency or Consistency.

Even without failures, strong consistency costs latency.

Real example

You post a comment on YouTube. Your friend in another country may see it 2 seconds later. That is fine (AP). But a payment must not be charged twice, so the payment service is CP.

Trade-offs

- Strong consistency: correct but higher latency and lower availability.
- Eventual: fast and available, but the app must handle conflicts and stale reads.

Common follow-up questions

- "How do you resolve conflicts?" Answer: last-write-wins with timestamps, version vectors, or app-level merge.
- "Is CAP about normal operation?" Answer: no, only during partitions; PACELC covers normal time.
- "How does Spanner give strong consistency globally?" Answer: TrueTime with synchronized clocks and Paxos.

How a test engineer should think about it

Simulate network partitions (for example with fault injection), write on one side and read on the other, and verify the system behaves as its contract says: error, stale, or fresh.

Videos

- [CAP theorem simplified](#) - ByteByteGo (English)
- [CAP theorem in interviews](#) - Hello Interview (English)

Further reading

- [System Design Primer - CAP](#)

Building block 9: APIs and communication

API styles

- REST: resources with HTTP verbs (GET /users/42). Simple, cacheable, human readable. Most public APIs.
- gRPC: binary Protocol Buffers over HTTP/2. Fast, typed contracts, streaming. Best for service-to-service calls inside a company (Google uses it widely).
- GraphQL: client asks for exactly the fields it needs in one query. Good for mobile apps with many screens; harder to cache and rate limit.

Pagination

- Offset: ?offset=100&limit=20. Simple, but slow on big tables and items shift when new data arrives.
- Cursor (keyset): ?cursor=abc&limit=20, where cursor encodes the last seen ID. Fast and stable. Preferred for feeds.

Idempotency keys

A client sends Idempotency-Key: <uuid> with a POST. The server stores the key and the result.

If the client retries after a timeout, the server returns the saved result instead of creating a second order or payment.

- GET, PUT, DELETE should be idempotent by design. POST is not, so it needs keys.

Versioning

- URL version: /v1/users. Clear and common.
- Header version: Accept: application/vnd.app.v2+json.
- Rules: never remove or rename fields in the same version; only add optional fields. Deprecate with a timeline.

Rate limiting with token bucket

Each user has a bucket with capacity C tokens. Tokens refill at rate R per second. Each request takes one token. No token means 429 Too Many Requests.

It allows short bursts (up to C) but limits the average rate to R.

Other algorithms: leaky bucket, fixed window counter, sliding window log, sliding window counter.

Client -> API Gateway (auth, rate limit) -> Service -> DB

Real example

A payments API: POST /v1/payments with an idempotency key, GET /v1/payments?cursor=...&limit=50, limit of 100 requests per second per API key.

Trade-offs

- REST is simple but can over-fetch; GraphQL is flexible but complex on the server.
- gRPC is fast but not browser friendly without a proxy.

Common follow-up questions

- "Which status codes?" Answer: 200 ok, 201 created, 400 bad input, 401 not logged in, 403 not allowed, 404 not found, 409 conflict, 429 rate limited, 500/503 server errors.
- "How do you avoid breaking clients?" Answer: contract tests, additive changes only, versioning.
- "Where does rate limiting live?" Answer: usually in the API gateway with counters in Redis.

How a test engineer should think about it

Write contract tests for every version, test retries with the same idempotency key, test pagination boundaries (empty, last page, data inserted mid-scroll), and test 429 behaviour and headers.

Videos

- [tRPC, gRPC, GraphQL or REST](#) - Software Developer Diaries (English)
- [gRPC vs REST vs GraphQL](#) - Anton Putra (English)

Further reading

- [System Design Primer - communication](#)

Building block 10: Reliability and observability

SLI, SLO, SLA

- SLI (indicator): a measured number, like "percent of requests that succeed in under 300 ms".
- SLO (objective): the internal target, like "99.9% of requests succeed each month".
- SLA (agreement): a contract with customers, with money back if broken. Usually looser than the SLO.

Error budget

If the SLO is 99.9%, the error budget is 0.1%. In 30 days that is about 43 minutes of downtime.

If budget remains, the team can ship fast. If budget is used up, freeze risky launches and fix reliability.

This turns "reliability vs speed" into a data-based decision. It is a core Google SRE idea.

Three pillars of observability

- Logs: detailed events with context (request ID, user ID). Good for debugging one request.
- Metrics: numbers over time (QPS, error rate, p99 latency, CPU). Good for dashboards and alerts.
- Traces: the path of one request across many services, with timing per hop. Good for finding which service is slow.

Golden signals: latency, traffic, errors, saturation.

Retries with exponential backoff

Retry failed calls, but wait longer each time: 100 ms, 200 ms, 400 ms, 800 ms, plus random jitter.

Jitter stops all clients from retrying at the same moment.

Only retry idempotent operations or use idempotency keys. Set a max retry count and an overall deadline.

Circuit breaker

If a dependency fails a lot, stop calling it for a while. This protects both sides.

- Closed: normal calls.
- Open: fail fast without calling, for example for 30 seconds.
- Half-open: let a few test calls through; if they work, close again.

```
Service A -> [Circuit Breaker] -> Service B (failing) => A returns fallback
```

Real example

A recommendation service is slow. Without a breaker, the home page waits and times out. With a breaker, the home page shows "popular items" as fallback and stays fast.

Trade-offs

- Higher SLO means much more cost and slower releases. 99.99% allows only 4 minutes per month.
- Too many alerts cause alert fatigue. Alert on symptoms (SLO burn rate), not every cause.

Common follow-up questions

- "What is a retry storm?" Answer: many layers retrying multiply load on a failing service; use budgets, backoff, and retry at one layer.
- "How do you choose an SLO?" Answer: based on user needs and current performance, not 100%.
- "What is graceful degradation?" Answer: turn off non-critical features to keep core features working.

How a test engineer should think about it

Use fault injection and chaos tests to verify retries, timeouts, breakers, and fallbacks really work, and check that alerts fire and dashboards show the problem.

Videos

- [SLO vs SLI vs SLA vs error budget](#) - Tech Tutorials with Piyush (English)

Further reading

- [Google SRE book \(free\)](#).

Building block 11: Designing test infrastructure

Why test infrastructure is a design topic

At Google, test engineers often build systems that run millions of tests per day. Interviewers may ask you to design one.

The goals: fast feedback, reliable results, low cost, and useful data about flakiness.

Main components

- Trigger: a commit, a pull request, or a schedule creates a test run.
- Orchestrator / scheduler: splits tests into shards and assigns them to workers.
- Workers: containers or VMs that run tests in a clean, hermetic environment.
- Artifact store: logs, screenshots, videos, coverage files (object storage like GCS).
- Results database: pass/fail, duration, error messages per test per run.
- Dashboard and notifier: shows results and alerts the author.

```
Commit -> Trigger -> Orchestrator -> Queue -> [Workers] -> Results DB + Artifact Store -> Dashboard
```

Sharding tests

Split tests across workers so they run in parallel. Balance by historical duration, not by count.

Example: 10,000 tests, total 500 minutes. With 50 shards of about 10 minutes each, the run finishes in about 10 minutes.

Retries and flakiness

- A flaky test passes and fails on the same code. Retrying hides the problem but unblocks developers.
- Policy: retry a failed test up to 2 times. If it passes on retry, mark it "flaky", not "pass".
- Flakiness analytics: flaky rate per test over the last N runs. Auto-quarantine tests above a threshold and file a bug for the owner.

Result storage

Store one row per test execution: run_id, test_id, commit, status, duration, attempt, worker, error hash. This lets you find slowest tests, most flaky tests, and which commit broke a test.

Device farms

For mobile and browser tests, keep a pool of real devices and emulators. A device manager leases devices, checks health, resets them, and returns them to the pool.

Smart test selection

Run only tests affected by the changed files (using a build dependency graph like Bazel). This cuts cost a lot.

Trade-offs

- More shards means faster runs but more startup overhead and cost.
- Retries reduce noise but can hide real bugs (race conditions).
- Test selection is fast but can miss failures; run the full suite periodically.

Common follow-up questions

- "How do you find which commit broke the build?" Answer: bisect between last green and first red commits.
- "How do you make tests hermetic?" Answer: fresh containers, fake or local dependencies, fixed data, no shared state.
- "How do you handle a worker that dies?" Answer: heartbeat timeout, then reschedule the shard on another worker.

How a test engineer should think about it

This is your home ground: talk about signal quality (flaky rate, false failures), feedback time (p50/p95 run time), and cost per run as the key metrics.

Further reading

- [SWE at Google - Ch 23 CI](#)

Case study 1: Design a URL shortener (like bit.ly)

Interview prompt: Design a URL shortener (like bit.ly). Talk through it in 35 minutes, then write your summary.

Try it yourself for 35 minutes before you read the model answer.

What a strong answer covers (checklist)

- ☐ Clarified functional requirements
- ☐ Non-functional: scale (users, QPS), latency, availability, consistency
- ☐ Back-of-the-envelope numbers (storage, bandwidth)
- ☐ API design (endpoints / methods)
- ☐ Data model and database choice (SQL vs NoSQL, why)
- ☐ High-level diagram: clients, load balancer, services, DB, cache, queue
- ☐ Deep dive on the hardest part
- ☐ Bottlenecks, single points of failure, scaling plan
- ☐ How you would test and monitor it

Model answer

Requirements

- Functional: given a long URL, return a short URL. Visiting the short URL redirects to the long URL.
- Functional: optional custom alias and expiry time. Basic click analytics.
- Non-functional: very low redirect latency (under 50 ms p99), high availability (99.99%), short codes must be unique and not easy to guess.
- Read-heavy: about 10 reads for every write.

Back-of-the-envelope estimates

- Writes: 100M new URLs per day. $100M / 86,400 \text{ s} = \text{about } 1,200 \text{ writes per second}$. Peak $2x = 2,400$.
- Reads: $10x = 1B$ per day = about 12,000 reads per second. Peak about 25,000.
- Storage: 500 bytes per record. $100M \times 500 \text{ B} = 50 \text{ GB per day}$. $50 \text{ GB} \times 365 \times 5 \text{ years} = \text{about } 90 \text{ TB}$.
- Code length: base62 with 7 chars = $62^7 = \text{about } 3.5 \text{ trillion codes}$. $100M \times 365 \times 5 = 182B$, so 7 chars is enough.
- Cache: 20% of URLs get 80% of traffic. Daily reads cover maybe 20M hot URLs $\times 500 \text{ B} = 10 \text{ GB}$. Fits in memory.

API design

- `POST /v1/urls` body {long_url, custom_alias?, expires_at?} returns {short_url}.
- `GET /{code}` returns 301 or 302 redirect with Location header.
- `GET /v1/urls/{code}/stats` returns click counts.
- `DELETE /v1/urls/{code}` (owner only).

Data model

- Table urls: code (primary key), long_url, user_id, created_at, expires_at.
- Table clicks (analytics, separate store): code, timestamp, country, referrer.
- A key-value store (Bigtable, DynamoDB, Cassandra) fits well: simple lookup by code, huge scale.

High-level design

- Load balancer in front of stateless API servers.
- ID generator service for unique codes.
- Redis cache for hot code -> long_url mappings.
- Key-value database as source of truth.
- Kafka queue for click events, consumed by an analytics pipeline.

```
Create:  Client -> LB -> API -> ID Gen -> DB
Redirect: Client -> LB -> API -> Cache (miss -> DB) -> 302 -> Kafka (click)
```

Redirect flow step by step:

1. User opens `https://sho.rt/aB3dE9x`.
2. LB sends request to any API server.
3. API checks Redis for `aB3dE9x`. On hit, it has the long URL.
4. On miss, API reads the DB, writes the result to Redis with a TTL.
5. API returns 302 with Location. It sends a click event to Kafka asynchronously.

Deep dive

Generating unique codes. Option 1: hash the long URL (MD5) and take 7 chars. Collisions are possible, so check and retry. Option 2: a global counter converted to base62. No collisions, but sequential codes are guessable and the counter is a bottleneck.

My choice: a counter service that gives each API server a range of 1M IDs at a time (stored in ZooKeeper or a DB row). Servers generate IDs locally from their range, so there is no per-request coordination. To avoid guessable codes, shuffle the ID with a reversible bit permutation before base62 encoding.

301 vs 302. 301 is cached by browsers, so fewer requests reach us, but we lose analytics. 302 keeps analytics accurate. I would use 302 if analytics matter.

Bottlenecks and scaling

- Reads: cache handles most. Add Redis replicas and a CDN edge for very hot links.
- DB: shard by code (hash). Key-value stores do this automatically.
- ID ranges: if a server dies, its unused range is lost. That is fine; we have trillions of codes.

- Analytics writes go through Kafka so they never slow down redirects.

How I would test and monitor it

- Unit tests: base62 encode/decode round trip, custom alias validation, expiry logic.
- API tests: create then redirect, unknown code returns 404, expired code returns 410, duplicate custom alias returns 409, invalid URL returns 400.
- Security tests: block javascript: URLs and known malware domains, rate limit creation per user, check open-redirect abuse.
- Concurrency test: 1,000 parallel creates must give 1,000 unique codes.
- Load test: 25,000 redirects per second, check p99 under 50 ms and cache hit ratio over 90%.
- Failure tests: Redis down (redirects still work, slower), one DB node down.
- Monitoring: redirect p99 latency, 5xx rate, cache hit ratio, ID range usage, Kafka consumer lag. Synthetic probe that creates and follows a link every minute.

Trade-offs I would mention

- Counter vs hash for codes: counter avoids collisions, hash avoids coordination.
- 301 vs 302: performance vs analytics accuracy.
- NoSQL for scale and simple access vs SQL for easy queries on user data.
- Eventual consistency for analytics is fine; the mapping itself should be strongly consistent after create.

Case study 2: Design a rate limiter

Interview prompt: Design a rate limiter. Talk through it in 35 minutes, then write your summary.

Try it yourself for 35 minutes before you read the model answer.

What a strong answer covers (checklist)

- ☐ Clarified functional requirements
- ☐ Non-functional: scale (users, QPS), latency, availability, consistency
- ☐ Back-of-the-envelope numbers (storage, bandwidth)
- ☐ API design (endpoints / methods)
- ☐ Data model and database choice (SQL vs NoSQL, why)
- ☐ High-level diagram: clients, load balancer, services, DB, cache, queue
- ☐ Deep dive on the hardest part
- ☐ Bottlenecks, single points of failure, scaling plan
- ☐ How you would test and monitor it

Model answer

Requirements

- Functional: limit how many requests a client (user ID, API key, or IP) can make in a time window, for example 100 per minute.
- Functional: return HTTP 429 with a Retry-After header when the limit is hit. Support different rules per endpoint and plan.
- Non-functional: add very little latency (under 2 ms), work across many servers (distributed), highly available.
- If the limiter fails, the API should still work (fail open) for most endpoints.

Back-of-the-envelope estimates

- Traffic: 1M requests per second across the whole API.
- Active clients: 10M users. Each counter needs about 50 bytes (key + count + timestamp).
- Memory: $10M \times 50 \text{ B} = 500 \text{ MB}$ per rule. With 5 rules, 2.5 GB. Fits easily in a Redis cluster.
- Redis ops: each request does about 1-2 operations, so 1-2M ops per second. One Redis node does about 100K ops per second, so we need about 20 shards.

API design

- Internal: `allow(client_id, rule_id) -> {allowed: bool, remaining: int, reset_at: ts}`.
- Response headers: `X-RateLimit-Limit`, `X-RateLimit-Remaining`, `Retry-After`.
- Admin: `PUT /v1/rules/{rule_id}` with `{limit, window, scope}`.

Data model

- Rules (config store, cached in memory): `rule_id`, endpoint pattern, limit, window_seconds, key_type.
- Counters (Redis): key = `rl:{rule_id}:{client_id}`, value = tokens and last refill time (token bucket).

High-level design

- The limiter runs as middleware inside the API gateway.
- Rules are loaded from a config service and cached locally, refreshed every 30 seconds.
- Counters live in a sharded Redis cluster, sharded by client key.

```
Client -> LB -> API Gateway [Rate Limiter] -> Redis (counters)
                        | allowed -> Backend Service
                        | denied  -> 429
```

Request flow:

1. Request arrives at the gateway. It finds the client ID from the API key.
2. It matches the request to a rule from the local rule cache.
3. It calls Redis with an atomic Lua script that refills and takes a token.
4. If a token was taken, forward to backend. Otherwise return 429 with `Retry-After`.

Deep dive

Algorithm choice. Fixed window is simple but allows 2x bursts at window edges (100 at 0:59 and 100 at 1:00). Sliding window log is exact but stores every timestamp (memory heavy). Sliding window counter mixes current and previous window counts; it is cheap and accurate enough. Token bucket allows controlled bursts and is easy to explain. I would choose token bucket: capacity C and refill rate R.

Race conditions. Two gateways may read count = 99 at the same time and both allow. Fix: do read-modify-write in one atomic Redis Lua script, so Redis runs it as one step.

Latency. A Redis call per request costs about 0.5-1 ms. For very high traffic, keep a small local token bucket per server and sync with Redis in batches. This is slightly less accurate but much faster.

Bottlenecks and scaling

- Redis: shard by client key; add replicas for failover.
- Hot client (one key with huge traffic): use local pre-check to reject quickly without Redis.
- Multi-region: keep limits per region, or accept slight over-limit with async sync between regions.
- If Redis is down: fail open (allow) for normal APIs, fail closed for sensitive ones like login.

How I would test and monitor it

- Unit tests: token refill math, boundaries (exactly at limit, limit + 1), clock edge cases.
- Functional API tests: send 100 requests, the 101st gets 429; wait for refill, request passes again; headers are correct.
- Concurrency test: 50 threads send 200 requests at the same time for one key; exactly 100 must pass. This catches race conditions.
- Distributed test: send requests through different gateway instances; the limit is shared.
- Failure test: stop Redis, verify fail-open behaviour and an alert.
- Performance test: measure added latency at 1M QPS (target under 2 ms p99).
- Monitoring: allowed vs denied counts per rule, Redis latency and errors, top limited clients, fail-open events.

Trade-offs I would mention

- Accuracy vs latency: central Redis is accurate; local counters are fast but approximate.
- Fail open vs fail closed: availability vs protection.
- Token bucket vs sliding window: bursts allowed vs smoother limits.
- Gateway middleware vs separate service: less latency vs independent scaling.

Case study 3: Design a key-value store

Interview prompt: Design a key-value store. Talk through it in 35 minutes, then write your summary.

Try it yourself for 35 minutes before you read the model answer.

What a strong answer covers (checklist)

- ☐ Clarified functional requirements
- ☐ Non-functional: scale (users, QPS), latency, availability, consistency
- ☐ Back-of-the-envelope numbers (storage, bandwidth)
- ☐ API design (endpoints / methods)
- ☐ Data model and database choice (SQL vs NoSQL, why)
- ☐ High-level diagram: clients, load balancer, services, DB, cache, queue
- ☐ Deep dive on the hardest part
- ☐ Bottlenecks, single points of failure, scaling plan
- ☐ How you would test and monitor it

Model answer

Requirements

- Functional: put(key, value), get(key), delete(key). Values up to 1 MB.
- Non-functional: highly available, scales to many TB, low latency (under 10 ms p99), tunable consistency, survives node failures.
- Assume an AP-style system (like Dynamo or Cassandra) with an option for strong reads.

Back-of-the-envelope estimates

- Data: 10B keys x 1 KB average = 10 TB. With replication factor 3 = 30 TB.
- Node size: 2 TB usable SSD per node. 30 TB / 2 TB = 15 nodes, plan 25 for headroom.
- Traffic: 500K reads per second, 100K writes per second.
- Per node: 600K / 25 = 24K ops per second per node. Fine for SSD with memory cache.

API design

- `PUT /v1/kv/{key}` body value, optional header consistency=quorum.
- `GET /v1/kv/{key}` returns value and version.
- `DELETE /v1/kv/{key}` (writes a tombstone).

Data model

- Each record: key, value, version (vector clock or timestamp), tombstone flag.
- On disk: LSM tree. Writes go to a commit log (for durability) and a memtable. Memtable is flushed to sorted SSTable files. Background compaction merges files.
- Bloom filters per SSTable avoid disk reads for missing keys.

High-level design

- Client library or coordinator node routes requests.
- Consistent hashing ring with virtual nodes decides which nodes own a key.
- Each key is replicated to $N = 3$ nodes (next 3 on the ring, in different racks or zones).
- Gossip protocol shares membership and failure info.

```
Client -> Coordinator -> hash(key) on ring -> [Replica 1, Replica 2, Replica 3]
```

Write flow:

1. Client sends put to any node; it becomes coordinator.
2. Coordinator finds the 3 replicas for the key.
3. It sends the write to all 3 and waits for $W = 2$ acks.
4. Each replica appends to its commit log, updates memtable, and acks.
5. Coordinator returns success. The third replica catches up.

Read flow: coordinator asks replicas, waits for $R = 2$ answers, returns the newest version, and repairs stale replicas (read repair).

Deep dive

Consistency with quorum. $N = 3$, $W = 2$, $R = 2$ gives $R + W > N$, so reads see the latest write. For speed, allow $W = 1$, $R = 1$ (eventual).

Handling failures. If a replica is down, the coordinator writes to another node with a hint (hinted handoff). When the node is back, the hint is delivered. For long outages, anti-entropy uses Merkle trees to compare and sync data efficiently.

Conflicts. With concurrent writes, use last-write-wins (simple, may lose data) or vector clocks (detect conflicts and let the client merge). I would start with last-write-wins and explain the risk.

Bottlenecks and scaling

- Add nodes: consistent hashing moves only about $1/N$ of the data.
- Hot keys: add a cache layer, or replicate hot keys more.
- Compaction can cause latency spikes; throttle it.
- Large values: store big blobs in object storage and keep only a pointer.

How I would test and monitor it

- Unit tests: memtable flush, SSTable merge, Bloom filter false positive rate, tombstone handling.

- Correctness tests: write then read with different R/W settings; delete then read returns not found even after compaction.
- Linearizability checks with a tool like Jepsen: random operations plus network partitions, then verify history is valid for the chosen consistency.
- Failure tests: kill a node during writes, verify no acknowledged write is lost; restart and verify hinted handoff and repair.
- Rebalancing test: add a node under load; check data moves and latency stays within SLO.
- Performance: p99 read and write latency at target QPS, with and without compaction.
- Monitoring: per-node latency, disk usage, compaction backlog, hinted handoff queue, read repair rate, gossip health.

Trade-offs I would mention

- AP vs CP: available during partitions but may return stale data.
- LSM tree vs B-tree: fast writes vs faster reads.
- Last-write-wins vs vector clocks: simplicity vs no lost updates.
- Replication factor 3: durability vs storage cost.

Case study 4: Design a news feed

Interview prompt: Design a news feed. Talk through it in 35 minutes, then write your summary.

Try it yourself for 35 minutes before you read the model answer.

What a strong answer covers (checklist)

- ☐ Clarified functional requirements
- ☐ Non-functional: scale (users, QPS), latency, availability, consistency
- ☐ Back-of-the-envelope numbers (storage, bandwidth)
- ☐ API design (endpoints / methods)
- ☐ Data model and database choice (SQL vs NoSQL, why)
- ☐ High-level diagram: clients, load balancer, services, DB, cache, queue
- ☐ Deep dive on the hardest part
- ☐ Bottlenecks, single points of failure, scaling plan
- ☐ How you would test and monitor it

Model answer

Requirements

- Functional: users post text and images, follow others, and see a home feed of posts from people they follow, newest or ranked first.
- Functional: like and comment (basic).
- Non-functional: feed loads fast (under 200 ms p99), highly available, eventual consistency is fine (a post can appear a few seconds late).
- Read-heavy: feed reads are much more frequent than posts.

Back-of-the-envelope estimates

- 500M daily active users. Each opens the feed 10 times per day = 5B feed reads per day = about 58K reads per second. Peak about 120K.
- Posts: 50M per day = about 600 posts per second.
- Average 200 followers per user. Fan-out writes: $600 \times 200 = 120K$ feed inserts per second.
- Feed cache: store 500 post IDs per user $\times 8$ bytes = 4 KB. $500M \times 4$ KB = 2 TB in Redis, sharded.
- Post storage: 1 KB text $\times 50M = 50$ GB per day; media goes to object storage and CDN.

API design

- `POST /v1/posts` body {text, media_ids} returns post_id.
- `GET /v1/feed?cursor=...&limit=20` returns posts and next cursor.
- `POST /v1/users/{id}/follow`, `DELETE /v1/users/{id}/follow`.
- `POST /v1/posts/{id}/like`.

Data model

- posts: post_id (time-sortable Snowflake ID), author_id, text, media_urls, created_at. Stored in a wide-column DB sharded by post_id.
- follows: follower_id, followee_id. Two indexes: who I follow, and who follows me.
- feed cache (Redis): key feed:{user_id}, sorted list of post IDs.
- users: profile data in SQL.

High-level design

- Post service writes posts and publishes a "new post" event to Kafka.
- Fan-out workers read events, get followers, and push post IDs into each follower feed cache.
- Feed service reads the feed cache, fetches post details (with a post cache), and ranks.
- Media service uploads images to object storage served by CDN.

Post: Client -> API -> Post Service -> Posts DB -> Kafka -> Fan-out Workers -> Feed Cache
 Read: Client -> API -> Feed Service -> Feed Cache -> Post Cache/DB -> Ranker -> Client

Read flow step by step:

1. Client calls `GET /feed`.
2. Feed service reads 500 post IDs from feed:{user_id}.
3. It merges in recent posts from followed celebrities (pull model, see deep dive).
4. It batch-fetches post details from the post cache.
5. It ranks, returns 20 posts and a cursor.

Deep dive

Fan-out on write vs on read. Push (on write) makes reads fast but a celebrity with 50M followers creates 50M writes per post. Pull (on read) is cheap on write but slow on read, because we query many authors. Hybrid: push for normal users, pull for accounts with over 100K followers. At read time, merge both lists.

Ranking. Start with reverse chronological. Then add a ranking model with features like affinity, post age, and engagement. Ranking runs on a few hundred candidates only, so it stays fast.

Inactive users. Do not fan out to users inactive for 30 days; build their feed on demand when they return.

Bottlenecks and scaling

- Feed cache is big: shard Redis by user_id.
- Fan-out workers: scale by Kafka partitions; fan-out lag is the key metric.
- Hot posts (viral): cache post details and like counts; batch like count updates.
- Media: CDN handles bandwidth.

How I would test and monitor it

- Functional tests: follow, post, then the post appears in the follower feed; unfollow removes future posts; blocked users never appear.
- Pagination tests: no duplicates or gaps when new posts arrive while scrolling (cursor based).
- Celebrity test: account with 1M followers posts; verify followers see it via the pull path.
- Privacy tests: private posts never leak into non-follower feeds. This is the most important correctness check.
- Load test: 120K feed reads per second at p99 under 200 ms; fan-out lag under 5 seconds.
- Failure tests: Redis shard down (rebuild feed from DB, slower), Kafka lag spike.
- Monitoring: feed latency, fan-out lag, cache hit ratio, empty feed rate, ranking errors. Ranking changes go through A/B tests with guardrail metrics.

Trade-offs I would mention

- Push vs pull vs hybrid: write cost vs read latency.
- Eventual consistency: a post can show late, which is acceptable.
- Precomputed feeds use a lot of memory; we save cost by skipping inactive users.
- Chronological is simple and predictable; ranked feed increases engagement but is harder to test.

Case study 5: Design a chat system

Interview prompt: Design a chat system. Talk through it in 35 minutes, then write your summary.

Try it yourself for 35 minutes before you read the model answer.

What a strong answer covers (checklist)

- ☐ Clarified functional requirements
- ☐ Non-functional: scale (users, QPS), latency, availability, consistency
- ☐ Back-of-the-envelope numbers (storage, bandwidth)
- ☐ API design (endpoints / methods)
- ☐ Data model and database choice (SQL vs NoSQL, why)
- ☐ High-level diagram: clients, load balancer, services, DB, cache, queue
- ☐ Deep dive on the hardest part
- ☐ Bottlenecks, single points of failure, scaling plan
- ☐ How you would test and monitor it

Model answer

Requirements

- Functional: one-to-one and group chat (up to 500 members), send and receive text in real time, message history, online status, delivered and read receipts.
- Functional: push notifications when the user is offline.
- Non-functional: low delivery latency (under 200 ms), messages never lost, order kept within a conversation, high availability.

Back-of-the-envelope estimates

- 100M daily active users, 50 messages each per day = 5B messages per day = about 58K messages per second. Peak 150K.
- Message size 200 bytes. $5B \times 200 B = 1 TB$ per day. 1 year = about 365 TB (before replication).
- Concurrent connections: 30M online at peak. One gateway server holds about 100K WebSocket connections, so about 300 servers.

API design

- WebSocket `connect(token)` for real-time messages.

- WS event `send {conversation_id, client_msg_id, text}` returns `ack {message_id, timestamp}`.
- WS event `message` pushed to receivers.
- `GET /v1/conversations/{id}/messages?before=msg_id&limit=50` for history.
- `POST /v1/conversations` to create a group.

Data model

- messages (wide-column, like Cassandra or Bigtable): partition key `conversation_id`, sort key `message_id` (time ordered), `sender_id`, `text`, `created_at`.
- conversations: `conversation_id`, `type`, `members`.
- user_conversations: `user_id` -> list of `conversation_id` with `last_read_message_id`.
- presence (Redis): `user_id` -> gateway server, `last_seen`.

High-level design

- Chat gateways keep WebSocket connections.
- Session registry (Redis) maps user to gateway.
- Chat service validates, assigns message IDs, stores messages, and routes.
- Message store (wide-column DB).
- Push notification service for offline users.

```
Sender -> Gateway A -> Chat Service -> Message DB
                                     -> Registry lookup -> Gateway B -> Receiver
                                     -> (offline) -> Push Service -> APNs/FCM
```

Send flow step by step:

1. Alice sends a message on her WebSocket to Gateway A with a `client_msg_id`.
2. Chat service checks membership, assigns a `message_id`, saves it to the DB.
3. It acks Alice (single tick = sent).
4. It looks up Bob in the registry. If online, it forwards to Gateway B, which pushes it to Bob.
5. Bob acks; the server marks delivered and notifies Alice (double tick).
6. If Bob is offline, send a push notification. Bob syncs history when he reconnects.

Deep dive

Ordering and no duplicates. Use a per-conversation sequence number or time-ordered ID assigned by the server. The client sends `client_msg_id`; the server dedups on it, so retries do not create duplicates. Receivers sort by sequence and detect gaps.

Offline sync. Each device stores the last message ID it saw per conversation. On reconnect, it asks for messages after that ID. This works for multiple devices too.

Group chat fan-out. For 500 members, the chat service looks up all online members and sends to their gateways. Store the message once, not per member.

Bottlenecks and scaling

- Gateways: scale horizontally; reconnect storms after a deploy need jitter on client reconnect.
- Registry: Redis cluster sharded by user_id.
- Message DB: partition by conversation_id; very active groups can be hot, so split by time bucket.
- Presence updates are noisy: batch them and only send to users who are looking.

How I would test and monitor it

- Functional tests: send and receive 1:1 and group, receipts, history paging, leaving a group stops delivery.
- Reliability tests: drop the network mid-send, client retries, verify exactly one message appears (dedup works).
- Ordering tests: send 1,000 fast messages from two users, verify both clients show the same order.
- Multi-device: read on phone, verify read state on laptop.
- Load test: 30M simulated connections (scaled down per gateway), 150K msgs/s, p99 delivery under 200 ms.
- Chaos: kill a gateway, clients reconnect to another and lose no messages.
- Monitoring: end-to-end delivery latency (measured by synthetic bots chatting), connection count per gateway, message drop rate, push success rate.

Trade-offs I would mention

- WebSocket vs long polling: lower latency vs simpler infrastructure.
- Server-side storage vs end-to-end encryption: easier search and sync vs privacy.
- Strict global order is expensive; order per conversation is enough.
- Wide-column DB for write volume vs SQL for complex queries.

Case study 6: Design a notification system

Interview prompt: Design a notification system. Talk through it in 35 minutes, then write your summary.

Try it yourself for 35 minutes before you read the model answer.

What a strong answer covers (checklist)

- [] Clarified functional requirements
- [] Non-functional: scale (users, QPS), latency, availability, consistency
- [] Back-of-the-envelope numbers (storage, bandwidth)
- [] API design (endpoints / methods)
- [] Data model and database choice (SQL vs NoSQL, why)
- [] High-level diagram: clients, load balancer, services, DB, cache, queue
- [] Deep dive on the hardest part
- [] Bottlenecks, single points of failure, scaling plan
- [] How you would test and monitor it

Model answer

Requirements

- Functional: other services send notifications to users by push (mobile), email, and SMS.
- Functional: user preferences (opt out per channel and type), templates, scheduling, and rate limits per user.
- Non-functional: reliable (no lost important notifications), no duplicates, scalable to big bursts (marketing campaigns), soft real time (seconds).

Back-of-the-envelope estimates

- Daily: 1B push, 100M email, 10M SMS. Total about 1.1B per day = about 13K per second average.
- Campaign burst: 50M notifications in 10 minutes = $50M / 600\text{ s}$ = about 83K per second.
- Storage of notification logs: 500 bytes x 1.1B = 550 GB per day; keep 30 days = 16.5 TB.

API design

- `POST /v1/notifications` body {user_ids or segment, type, template_id, params, channels?, send_at?, idempotency_key} returns notification_id.
- `GET /v1/notifications/{id}/status`.

- `PUT /v1/users/{id}/preferences` body {channel, type, enabled}.

Data model

- templates: template_id, channel, locale, body with placeholders.
- preferences: user_id, type, channel, enabled.
- devices: user_id, device_token, platform, last_active.
- notification_log: notification_id, user_id, channel, status (queued, sent, delivered, failed), attempts, timestamps.

High-level design

- Notification API validates requests and checks the idempotency key.
- Scheduler holds delayed notifications.
- Processor expands segments to users, checks preferences and rate limits, renders templates.
- One queue per channel (push, email, SMS) so a slow channel does not block others.
- Channel workers call providers: FCM/APNs, an email provider, an SMS gateway.
- Status tracker records results and provider callbacks.

```
Service -> Notification API -> Kafka -> Processor (prefs, limits, template)
-> [Push Queue -> Push Workers -> FCM/APNs]
-> [Email Queue -> Email Workers -> Email Provider]
-> [SMS Queue -> SMS Workers -> SMS Gateway]
-> Status Log
```

Flow step by step:

1. Order service calls the API: "order shipped" for user 42.
2. API stores the request with its idempotency key and puts it on Kafka.
3. Processor loads preferences: user 42 wants push and email, not SMS.
4. It renders templates in the user language and puts messages on push and email queues.
5. Workers send to providers, record status, and retry on temporary errors.

Deep dive

Reliability and no duplicates. Queues give at-least-once delivery. Each message has a dedup key (notification_id + user_id + channel). Workers check a dedup store (Redis with TTL) before sending. Retries use exponential backoff. After 5 failures, the message goes to a DLQ.

Priority. Separate high-priority queues for OTP and security alerts from low-priority marketing. A campaign of 50M messages must never delay a login OTP.

Provider failures. Use a circuit breaker per provider and fail over to a backup SMS or email provider.

Bottlenecks and scaling

- Provider rate limits (for example APNs or SMS throughput) are often the real bottleneck; workers must respect them.
- Segment expansion for 50M users: do it in batches, stream user IDs.

- Preference lookups: cache in Redis.
- Invalid device tokens: remove them when providers report them.

How I would test and monitor it

- Functional tests: each channel delivers, opted-out users receive nothing, templates render all placeholders and locales, scheduled send fires at the right time and time zone.
- Idempotency test: send the same request twice with the same key, user gets one notification.
- Failure tests: mock provider returns 500 and timeouts; verify retries, backoff, and DLQ. Provider down triggers failover.
- Priority test: run a 1M message campaign and measure OTP latency; it must stay under its SLO.
- Load test: 83K per second burst with mock providers.
- Contract tests with provider APIs, using sandbox environments.
- Monitoring: queue depth and age per channel, send success rate per provider, end-to-end latency, DLQ size, unsubscribe and complaint rates.

Trade-offs I would mention

- At-least-once plus dedup vs complex exactly-once.
- Separate queues per channel and priority: more components but better isolation.
- Fan-out at send time vs precomputed segments.
- Delivery receipts depend on providers and are not always reliable.

Case study 7: Design a web crawler

Interview prompt: Design a web crawler. Talk through it in 35 minutes, then write your summary.

Try it yourself for 35 minutes before you read the model answer.

What a strong answer covers (checklist)

- ☐ Clarified functional requirements
- ☐ Non-functional: scale (users, QPS), latency, availability, consistency
- ☐ Back-of-the-envelope numbers (storage, bandwidth)
- ☐ API design (endpoints / methods)
- ☐ Data model and database choice (SQL vs NoSQL, why)
- ☐ High-level diagram: clients, load balancer, services, DB, cache, queue
- ☐ Deep dive on the hardest part
- ☐ Bottlenecks, single points of failure, scaling plan
- ☐ How you would test and monitor it

Model answer

Requirements

- Functional: start from seed URLs, download pages, extract links, and keep crawling. Store page content for indexing.
- Functional: respect robots.txt, avoid duplicate pages, re-crawl pages based on how often they change.
- Non-functional: scale to billions of pages, be polite (do not overload any site), robust to bad HTML, traps, and slow servers.

Back-of-the-envelope estimates

- Target: 1B pages per month. $1B / (30 \times 86,400) =$ about 400 pages per second. Plan for 1,000 at peak.
- Page size: 500 KB average. $1B \times 500 \text{ KB} = 500 \text{ TB}$ per month raw. Compressed (5x) = 100 TB.
- Bandwidth: $400 \times 500 \text{ KB} = 200 \text{ MB/s} = 1.6 \text{ Gbps}$.
- URL frontier: $10B \text{ known URLs} \times 100 \text{ bytes} = 1 \text{ TB}$; needs disk-backed queues.

API design

- Internal only. `addSeeds(urls)`, `getStatus()`, and a config for crawl rate per domain.

- Output: pages written to storage and an event "page_fetched {url, content_hash, storage_path}" for the indexer.

Data model

- url_store: url_hash (key), url, last_crawled, next_crawl, status, content_hash.
- page_store (object storage or Bigtable): url_hash, raw HTML, headers, fetch time.
- robots cache: domain -> rules, fetched_at.
- seen content hashes: for duplicate detection (simhash for near duplicates).

High-level design

- URL frontier: prioritized queues of URLs to crawl, with per-host politeness.
- Fetchers: many workers that download pages; DNS resolver with cache.
- Parser: extracts links and text.
- URL filter and dedup: normalize URLs, check robots.txt, check if already seen (Bloom filter + url_store).
- Content dedup: skip pages with a known content hash.
- Storage for pages.

```
Seeds -> Frontier -> Fetchers -> Parser -> Link Extractor -> URL Filter/Dedup -> Frontier
                                     |
                                     -> Content Dedup -> Page Store -> Indexer
```

Flow step by step:

1. Fetcher takes the next URL from the frontier for a host that is allowed now.
2. It checks robots.txt (cached), resolves DNS (cached), and downloads with a timeout.
3. Parser extracts links and content. Content hash is checked against seen hashes.
4. New content is stored. Links are normalized, filtered, deduped, and added to the frontier.
5. url_store is updated with next_crawl time.

Deep dive

Politeness and the frontier. The frontier has two layers. Front queues by priority (PageRank, change frequency). Back queues: one per host, each mapped to one fetcher thread, with a minimum delay between requests (for example 1 second, or Crawl-delay from robots.txt). This guarantees we never hit one site in parallel.

Dedup at scale. 10B URLs do not fit in a hash set in memory. Use a Bloom filter (about 10 bits per URL = 12.5 GB) for fast "probably seen" checks, backed by the url_store. For content, use simhash to detect near-duplicate pages (mirrors, tracking parameters).

Crawler traps. Infinite calendars or session IDs in URLs. Limit URL length, depth per site, and pages per domain; strip known tracking parameters.

Bottlenecks and scaling

- Partition the frontier and fetchers by host hash, so one host is handled by one machine.

- DNS is slow: use a local caching resolver.
- Storage writes: batch and compress.
- Re-crawl scheduling: pages that change often (news) get short intervals; static pages get long ones.

How I would test and monitor it

- Build a local fake web: a test server with known pages, link loops, redirects, broken HTML, slow responses, huge files, and robots.txt rules.
- Correctness tests: every reachable page is crawled once, disallowed paths are never fetched, redirects followed up to a limit, relative links resolved correctly.
- Politeness test: the fake server logs request times; assert no two requests to one host come faster than the delay.
- Trap tests: infinite calendar pages stop at the depth limit.
- URL normalization unit tests: case, trailing slash, default port, fragments, query order.
- Load test: 1,000 pages per second against the fake web.
- Monitoring: pages per second, error rate by type (DNS, timeout, 4xx, 5xx), frontier size, duplicate rate, freshness (age of crawled pages), complaints from site owners.

Trade-offs I would mention

- Freshness vs coverage: re-crawling known pages vs discovering new ones.
- Bloom filter: memory efficient but small false positive rate (we may skip a few new URLs).
- BFS vs priority-based crawl: simple vs better quality pages first.
- Politeness limits our speed per site but protects the web and our reputation.

Case study 8: Design Google Drive / file storage

Interview prompt: Design Google Drive / file storage. Talk through it in 35 minutes, then write your summary.

Try it yourself for 35 minutes before you read the model answer.

What a strong answer covers (checklist)

- ☐ Clarified functional requirements
- ☐ Non-functional: scale (users, QPS), latency, availability, consistency
- ☐ Back-of-the-envelope numbers (storage, bandwidth)
- ☐ API design (endpoints / methods)
- ☐ Data model and database choice (SQL vs NoSQL, why)
- ☐ High-level diagram: clients, load balancer, services, DB, cache, queue
- ☐ Deep dive on the hardest part
- ☐ Bottlenecks, single points of failure, scaling plan
- ☐ How you would test and monitor it

Model answer

Requirements

- Functional: upload, download, delete files and folders; sync across devices; share with other users (view or edit); file versions.
- Non-functional: never lose data (very high durability), reliable large uploads (resume after network drop), fast sync, strong consistency for metadata.

Back-of-the-envelope estimates

- 500M users, 100M daily active. Each has 10 GB average stored. Total $500M \times 10 \text{ GB} = 5 \text{ EB}$ (exabytes).
- Uploads: $100M \text{ DAU} \times 2 \text{ files per day} = 200M \text{ uploads per day} = \text{about } 2,300 \text{ per second}$.
- Average file 1 MB: $2,300 \times 1 \text{ MB} = 2.3 \text{ GB/s ingress}$.
- Metadata: $100 \text{ files per user} \times 500M = 50B \text{ file records} \times 500 \text{ bytes} = 25 \text{ TB of metadata}$.

API design

- `POST /v1/uploads` body {name, parent_id, size, chunk_hashes} returns upload_id and which chunks are missing.

- `PUT /v1/uploads/{upload_id}/chunks/{index}` uploads one chunk (4 MB).
- `POST /v1/uploads/{upload_id}/commit` creates the file version.
- `GET /v1/files/{id}/download` returns signed URLs for chunks.
- `GET /v1/changes?cursor=...` returns changes since the last sync.
- `POST /v1/files/{id}/permissions` body {user, role}.

Data model

- files: file_id, owner_id, parent_id, name, is_folder, current_version, updated_at.
- versions: file_id, version, chunk_list (ordered chunk hashes), size, created_at.
- chunks: chunk_hash (SHA-256, key), storage_location, ref_count.
- permissions: file_id, user_id, role.
- change_log: user_id, sequence, file_id, change_type (for sync).
- Metadata in a strongly consistent SQL DB (Spanner-like), sharded by owner_id. Chunks in object storage.

High-level design

- Client app splits files into 4 MB chunks and hashes them.
- Metadata service handles files, versions, permissions, and change log.
- Upload / block service stores chunks in object storage (via signed URLs).
- Notification service tells other devices to sync (long poll or push).
- CDN or edge cache for popular downloads.

```
Client -> API -> Metadata Service -> Metadata DB
Client -> Block Service / signed URL -> Object Storage (chunks)
Metadata Service -> Change Log -> Notification Service -> Other Devices
```

Upload flow step by step:

1. Client chunks the file and computes hashes.
2. Client sends the chunk hashes; server replies with only the chunks it does not already have (dedup).
3. Client uploads missing chunks in parallel. Failed chunks are retried alone.
4. Client calls commit. Metadata service creates a new version in one transaction and writes to the change log.
5. Other devices are notified, call /changes, and download only changed chunks.

Deep dive

Chunking and dedup. Chunks make uploads resumable and let us sync only changed parts. Content hashes mean identical chunks are stored once. Using content-defined chunking (rolling hash) means an insert at the start of a file does not change every chunk.

Sync conflicts. Two devices edit the same file offline. On commit, the client sends the base version. If the server version is newer, we do not overwrite; we save the second edit as a "conflicted copy" and tell the user.

Durability. Object storage keeps data with replication or erasure coding across zones (for example 11 nines durability). Metadata DB has synchronous replication. Deleted files go to trash for 30 days.

Bottlenecks and scaling

- Metadata DB is the hot spot: shard by owner, cache folder listings.
- Shared folders with many users cross shards; keep permissions in a separate service.
- Upload bandwidth goes directly to object storage via signed URLs, not through API servers.
- Garbage collection removes chunks with ref_count 0.

How I would test and monitor it

- Functional tests: upload, download, rename, move, delete, restore from trash, version history, sharing roles (viewer cannot edit, removed user loses access).
- Integrity tests: download hash equals upload hash for files of 0 bytes, 1 byte, exactly 4 MB, 4 MB + 1, and 10 GB.
- Resumable upload test: cut network at 50%, resume, verify only missing chunks are sent.
- Sync tests: two devices, edit conflict produces a conflicted copy, no silent data loss.
- Permission security tests: guess file IDs, expired signed URLs, access after unshare.
- Failure tests: storage node down, metadata DB failover during commit (must be atomic).
- Monitoring: upload success rate, sync latency, data integrity scrubbing results, storage growth, permission errors.

Trade-offs I would mention

- Chunk size: small chunks give better dedup and resume but more metadata.
- Strong consistency for metadata vs eventual for notifications.
- Dedup across users saves storage but needs care for privacy and encryption.
- Conflicted copies are safe but less convenient than automatic merge.

Case study 9: Design a CI system that runs tests for every commit

Interview prompt: Design a CI system that runs tests for every commit. Talk through it in 35 minutes, then write your summary.

Try it yourself for 35 minutes before you read the model answer.

What a strong answer covers (checklist)

- ☐ Clarified functional requirements
- ☐ Non-functional: scale (users, QPS), latency, availability, consistency
- ☐ Back-of-the-envelope numbers (storage, bandwidth)
- ☐ API design (endpoints / methods)
- ☐ Data model and database choice (SQL vs NoSQL, why)
- ☐ High-level diagram: clients, load balancer, services, DB, cache, queue
- ☐ Deep dive on the hardest part
- ☐ Bottlenecks, single points of failure, scaling plan
- ☐ How you would test and monitor it

Model answer

Requirements

- Functional: for every commit or pull request, build the code and run tests, report pass/fail to the code review tool, and block merge if tests fail.
- Functional: logs and artifacts for each run, retry of a job, history of results per test, flaky test detection.
- Non-functional: fast feedback (p50 under 15 minutes), scalable to thousands of commits per day, reliable results (low flakiness), hermetic and reproducible, cost efficient.

Back-of-the-envelope estimates

- 5,000 engineers, 4 commits each per day = 20,000 runs per day. Peak hour 3x average: 20,000 / 8 working hours = 2,500 per hour, peak 7,500 per hour.
- Each run: 2,000 tests, total 200 CPU minutes if serial. With test selection, average 50 CPU minutes.
- CPU: 20,000 x 50 = 1M CPU minutes per day. At peak 7,500 runs per hour x 50 min = 375K CPU min per hour = about 6,250 CPUs busy.
- Logs and artifacts: 50 MB per run x 20,000 = 1 TB per day; keep 30 days = 30 TB.

- Results: 20,000 runs x 500 tests run = 10M result rows per day.

API design

- Webhook from the code host: `POST /v1/events {repo, commit_sha, pr_id, author}`.
- `GET /v1/runs/{run_id}` returns status, jobs, links.
- `POST /v1/runs/{run_id}/retry` (failed jobs only).
- `GET /v1/tests/{test_id}/history?limit=100` returns recent results and flaky rate.
- Status callback to the code host: commit status success/failure.

Data model

- runs: run_id, repo, commit_sha, pr_id, status, created_at, finished_at.
- jobs (shards): job_id, run_id, worker_id, status, attempt, started_at, duration.
- test_results: run_id, test_id, status (pass, fail, flaky, skipped), duration_ms, attempt, error_signature.
- tests: test_id, owner, avg_duration, flaky_rate, quarantined.
- Artifacts in object storage, path stored in jobs.

High-level design

- Event receiver: gets webhooks, dedups, creates runs.
- Planner: finds affected targets from the dependency graph, splits tests into shards by historical duration.
- Job queue (priority): PR runs before nightly runs.
- Worker pool: autoscaled containers with a fresh, hermetic environment; remote build cache.
- Results service: collects results, applies flaky logic, stores in DB.
- Reporter: updates the PR status, notifies the author.
- Dashboard: runs, test history, flaky leaderboard.

```
Code Host -> Webhook -> Event Receiver -> Planner -> Job Queue -> [Workers] -> Results Service
-> DB
Results Service -> Reporter -> Code Host (status) + Author notification
-> Artifact Store
```

Flow step by step:

1. A developer pushes a commit to a PR. The code host sends a webhook.
2. Event receiver creates a run (dedup on commit SHA) and cancels older runs for the same PR.
3. Planner computes affected tests (for example 600 of 2,000) and creates 20 shards of about 3 minutes each.
4. Workers pull jobs, check out the commit, use the build cache, run tests, upload logs.
5. Results service merges results, retries failed tests once, marks flaky ones.
6. Reporter sets the PR status to green or red with a link to failures.

Deep dive

Fast feedback. Three levers: test selection (run only affected tests), duration-based sharding (balance shards so they finish together), and caching (build and test result cache keyed by input hash; if inputs did not change, reuse the result). Also start the slowest shards first.

Flakiness handling. A test that fails then passes on retry for the same commit is "flaky". Store every attempt. Compute flaky rate over the last 200 runs. Above 2%, auto-quarantine: the test still runs but cannot block merges, and a bug is filed to the owner. Group failures by error signature to spot infra problems (for example, many tests failing with "connection refused" on one worker means a bad worker, not bad code).

Main branch health. Even with green PRs, main can break (two PRs that conflict). Use a merge queue that tests the PR on top of the latest main, or run post-submit tests and auto-bisect between last green and first red.

Bottlenecks and scaling

- Worker capacity at peak: autoscale on queue depth; use cheaper preemptible VMs for retryable jobs.
- Checkout and build time: shallow clones, remote build cache.
- Results DB write volume: batch inserts; partition by date.
- Noisy neighbours on workers: resource limits per container.

How I would test and monitor it

- Test the CI system itself with a canary repo containing known passing, failing, flaky, and timing-out tests; run it every 10 minutes and assert correct statuses.
- Test webhook dedup (same event twice gives one run) and cancellation of stale runs.
- Test worker death: kill a worker mid-job, the job is rescheduled and the run still finishes correctly.
- Test the test selection logic: changing a file must trigger all tests that depend on it (compare with a full run nightly to find misses).
- Load test: replay one peak day of webhooks.
- Monitoring: queue wait time, run duration p50/p95, infra failure rate (not test failures), flaky rate trend, cache hit ratio, cost per run, and percent of time main is green.

Trade-offs I would mention

- Test selection vs full runs: speed vs risk of missing a failure.
- Auto-retry: unblocks developers but can hide real race conditions; that is why we record flaky separately.
- More shards: faster but more overhead per shard.
- Merge queue: safer main branch but adds wait time for each merge.

Case study 10: Design YouTube (upload + watch)

Interview prompt: Design YouTube (upload + watch). Talk through it in 35 minutes, then write your summary.

Try it yourself for 35 minutes before you read the model answer.

What a strong answer covers (checklist)

- ☐ Clarified functional requirements
- ☐ Non-functional: scale (users, QPS), latency, availability, consistency
- ☐ Back-of-the-envelope numbers (storage, bandwidth)
- ☐ API design (endpoints / methods)
- ☐ Data model and database choice (SQL vs NoSQL, why)
- ☐ High-level diagram: clients, load balancer, services, DB, cache, queue
- ☐ Deep dive on the hardest part
- ☐ Bottlenecks, single points of failure, scaling plan
- ☐ How you would test and monitor it

Model answer

Requirements

- Functional: users upload videos; videos are processed into many resolutions; users watch with smooth adaptive streaming; search, views, likes, comments (basic).
- Non-functional: playback starts fast (under 2 s), little buffering, very high availability for watching, uploads can take minutes to process, global users.
- Very read-heavy: views are far more than uploads.

Back-of-the-envelope estimates

- Uploads: 500 hours of video per minute. $500 \times 60 = 30,000$ hours per hour, = 720,000 hours per day.
- Storage: 1 hour raw = about 3 GB; after transcoding to 5 resolutions about 5 GB total. $720,000 \times 5 \text{ GB} = 3.6 \text{ PB}$ per day.
- Views: 1B hours watched per day. Average bitrate 2.5 Mbps. Concurrent viewers: $1\text{B hours} / 24 =$ about 42M concurrent. $42\text{M} \times 2.5 \text{ Mbps} =$ about 100 Tbps egress. This is why a CDN is required.
- Metadata: 1KB per video, tiny compared to video bytes.

API design

- `POST /v1/videos` body {title, description} returns video_id and a resumable upload URL.
- `PUT {upload_url}` chunked, resumable upload.
- `GET /v1/videos/{id}` returns metadata, status, and manifest URL.
- `GET /v1/videos/{id}/manifest.mpd` (DASH or HLS) lists segments per resolution.
- `POST /v1/videos/{id}/views`, `POST /v1/videos/{id}/like`.

Data model

- videos (SQL or Spanner): video_id, owner_id, title, status (uploading, processing, ready, failed), duration, created_at.
- video_renditions: video_id, resolution, codec, manifest_path.
- Raw and processed files in object storage; segments of 2-6 seconds each.
- view counts: counters in a distributed store, aggregated asynchronously.
- comments: wide-column DB keyed by video_id.

High-level design

- Upload service receives resumable uploads into object storage.
- Processing pipeline: queue plus workers for validation, transcoding, thumbnails, content checks.
- Metadata service and DB.
- CDN serves video segments from edge servers near users.
- Player uses adaptive bitrate streaming (DASH/HLS).

Upload: Creator -> Upload Service -> Raw Storage -> Queue -> Transcode Workers -> Processed Storage -> CDN origin
 Watch: Viewer -> API (metadata, manifest) -> CDN Edge (segments) -> miss -> Origin Storage

Watch flow step by step:

1. Viewer opens a video page. API returns metadata and the manifest URL.
2. Player downloads the manifest listing segments for 240p to 4K.
3. Player measures bandwidth and requests segments from the nearest CDN edge.
4. On edge miss, the edge fetches from origin and caches it.
5. Player switches resolution up or down every few segments as bandwidth changes.
6. View events go async to an analytics pipeline.

Deep dive

Transcoding pipeline. Split the raw video into small chunks (for example 10 seconds) and transcode chunks in parallel on many workers, then stitch. This turns a 1-hour job into minutes. Pipeline as a DAG: validate -> split -> transcode per resolution -> package segments -> thumbnails -> mark ready. Each step is idempotent and retried on failure; the video status is updated at the end.

CDN and cost. Popular videos are pushed to edges in advance. Long tail videos are served from regional caches or origin. Encode popular videos with better codecs (VP9, AV1) to save bandwidth, because bandwidth is the biggest cost.

View counting. Do not update one DB row per view. Aggregate counts in memory per server, flush every few seconds, and merge. Counts can be eventually consistent.

Bottlenecks and scaling

- Egress bandwidth: CDN with high hit ratio (95%+).
- Transcoding compute: autoscale workers on queue depth; prioritize popular creators.
- Hot video (viral): CDN handles it; metadata cached.
- Storage growth: move old, rarely watched renditions to cold storage.

How I would test and monitor it

- Upload tests: resumable upload after network drop, very large files, unsupported formats rejected, corrupt files fail gracefully with a clear status.
- Transcoding tests: a golden set of test videos (different codecs, frame rates, rotations, audio only, very short, very long); verify each rendition plays, duration matches, audio stays in sync. Use quality metrics like VMAF against reference.
- Playback tests: automated players on real devices and browsers under network throttling (3G, packet loss); measure startup time, rebuffer ratio, and resolution switches.
- Failure tests: worker dies mid-transcode, step retries without duplicate output; CDN edge down, player falls back to another edge.
- Load test: CDN and metadata API at peak read load.
- Monitoring: video start time, rebuffering ratio, playback error rate by device, CDN hit ratio, processing time p95, failed processing rate.

Trade-offs I would mention

- More renditions: better playback on all networks vs more storage and compute.
- Pre-warm CDN for popular videos vs fetch on demand for long tail.
- Eventual view counts: cheap and scalable but not exact in real time.
- Better codecs save bandwidth but cost more CPU to encode and are not supported on every device.

Case study 11: Design a device lab for testing Android apps at scale

Interview prompt: Design a device lab for testing Android apps at scale. Talk through it in 35 minutes, then write your summary.

Try it yourself for 35 minutes before you read the model answer.

What a strong answer covers (checklist)

- ☐ Clarified functional requirements
- ☐ Non-functional: scale (users, QPS), latency, availability, consistency
- ☐ Back-of-the-envelope numbers (storage, bandwidth)
- ☐ API design (endpoints / methods)
- ☐ Data model and database choice (SQL vs NoSQL, why)
- ☐ High-level diagram: clients, load balancer, services, DB, cache, queue
- ☐ Deep dive on the hardest part
- ☐ Bottlenecks, single points of failure, scaling plan
- ☐ How you would test and monitor it

Model answer

Requirements

- Functional: teams submit Android test jobs (APK + test APK + device filter such as model, OS version, screen size). The lab runs tests on real devices and emulators and returns results, logs, screenshots, and video.
- Functional: device health monitoring, automatic reset between jobs, sharding a suite across many devices, interactive remote access for debugging.
- Non-functional: high device utilization, short queue times, clean device state for every job (no leakage between teams), reliable results, scalable to thousands of devices.

Back-of-the-envelope estimates

- Fleet: 5,000 real devices across 10 racks per site and 3 sites, plus emulators on demand.
- Jobs: 200,000 test jobs per day, average 10 minutes of device time each = 2M device minutes per day.
- Capacity: 5,000 devices x 1,440 minutes = 7.2M device minutes per day. At a 60% realistic utilization = 4.3M. So 2M fits, with room for peaks.

- Peak: 3x during work hours: $200,000 / 10 \text{ hours} \times 3 = 60,000$ jobs per hour; $60,000 \times 10 \text{ min} / 60 = 10,000$ devices needed. Overflow goes to emulators.
- Artifacts: 30 MB video and logs per job $\times 200,000 = 6 \text{ TB}$ per day; keep 14 days = 84 TB.

API design

- `POST /v1/jobs` body {app_apk, test_apk, device_filter, shards, timeout, priority} returns job_id.
- `GET /v1/jobs/{id}` returns status, per-shard results, artifact links.
- `GET /v1/devices?model=Pixel8&os=15&state=idle` returns available devices.
- `POST /v1/devices/{id}/lease` for interactive debugging, with an expiry.

Data model

- devices: device_id, model, os_version, host_id, state (idle, leased, resetting, unhealthy, offline), battery, last_health_check.
- hosts: host_id, site, rack, connected device IDs, status.
- jobs: job_id, team, filter, priority, status, created_at.
- shards: shard_id, job_id, device_id, status, attempt, start, end.
- results: shard_id, test_name, status, duration, artifact_paths.

High-level design

- Job API receives jobs and stores APKs in object storage.
- Scheduler matches shards to devices using the filter, priority, and team quotas.
- Device manager (one per site) tracks device state and health.
- Host agents run on machines connected by USB to about 20 devices each; they run adb commands, install APKs, run tests, record video.
- Results service stores results and artifacts; dashboard shows them.
- Emulator pool on cloud VMs for overflow and common configs.

```
User/CI -> Job API -> Scheduler -> Device Manager -> Host Agent -> adb -> Device
Host Agent -> Artifact Store + Results Service -> Dashboard / CI status
```

Flow step by step:

1. CI submits a job: run 800 instrumentation tests on Pixel 8 Android 15, 10 shards.
2. Scheduler finds 10 idle matching devices and leases them.
3. Each host agent installs the APKs, runs its shard with a timeout, and streams logs.
4. Agent uploads logcat, screenshots, and video, then sends results.
5. Agent resets the device (uninstall apps, clear data, factory reset if needed) and runs a health check.
6. Healthy device returns to idle; unhealthy goes to quarantine for repair.

Deep dive

Device health and clean state. Real devices fail in many ways: low battery, overheating, stuck dialogs, lost USB, full storage, OS updates. Before every job, run a quick health check (adb responsive, battery over

30%, storage free, screen unlocks, network works). After each job, wipe app data and reboot. Do a full factory reset nightly or after a crash. A device that fails 3 checks in a row goes to quarantine and a ticket is created for a lab technician.

Scheduling. Matching is a filter plus a priority queue. Use team quotas so one team cannot take all Pixel 8 devices. Prefer devices on the same host for one job to reduce overhead. If no real device is free within N minutes and the job allows it, run on an emulator.

Separating test failures from infra failures. If a shard fails because the device disconnected or adb timed out, it is an infra failure: retry on a different device and do not blame the app. Track failure rate per device: a device that fails tests much more often than others with the same model is probably bad.

Bottlenecks and scaling

- Popular models run out first: buy based on demand data from the queue.
- USB and host limits: about 20 devices per host; add hosts and racks.
- Artifact upload bandwidth: compress videos, record video only on failure if configured.
- Scheduler at 60,000 jobs per hour: shard by site; keep device state in memory with a durable backing store.

How I would test and monitor it

- Canary jobs: a known good test suite runs on every device model every hour. If it fails, the device or model has an infra problem, not the app.
- Fault injection: unplug USB, kill adb, drain battery, fill storage on a test device; verify the system detects it, retries the shard elsewhere, and quarantines the device.
- Isolation test: job A writes a file and an account; job B on the same device must not see them after reset.
- Scheduler tests: quotas enforced, priorities respected, filters match exactly, no double lease of one device under concurrency.
- Load test: replay a peak hour of jobs against a simulated device fleet.
- Monitoring: queue wait time per model, device utilization, healthy device percentage, infra failure rate vs test failure rate, reset duration, flaky rate by device.

Trade-offs I would mention

- Real devices vs emulators: real hardware bugs and sensors vs cost, speed, and scale.
- Factory reset every job: cleanest state but slow (minutes); app-data wipe is faster but less clean.
- Auto-retry on infra failures improves signal, but retrying test failures can hide real flakiness.
- Buying many models covers more users but reduces devices per model and increases queue times.

Distributed Systems Deep Dive: Replication, Consensus, Clocks and Testing

In this chapter:

- Understand how machines fail, and how replication, quorums and consensus (Raft, Paxos) keep data safe
- Build consistent hashing with virtual nodes in Python
- Compare distributed transactions (2PC vs sagas) and understand logical clocks, vector clocks and the TrueTime idea
- See why "exactly-once" is mostly a myth, and how idempotency and backpressure save you
- Learn how experts TEST distributed systems: fault injection, Jepsen-style checks, chaos engineering and deterministic simulation

Time: ~75 minutes | **Level:** Expert

In the earlier lessons you learned CAP, quorums, sharding and message queues at a high level. This chapter goes one level deeper. The goal is simple: when an interviewer says "What happens if this node dies?" or "How do these replicas agree?", you can answer step by step, and then explain how you would **test** it. That second part is where a test engineer shines.

A **distributed system** is a group of computers that work together and look like one system to the user. Think of IRCTC: many servers in many data centres, but you see one website and one ticket.

1. Failure models: how things break

Before you design anything, agree on **what can fail**. This list is called the **failure model**.

Failure type	What happens	Everyday analogy
Crash-stop	A node stops and never comes back	A shop closes forever
Crash-recovery	A node stops, then restarts, maybe with old data on disk	A shopkeeper goes home and returns next morning
Omission	Some messages are lost	An SMS that never arrives
Timing / slow node	A node replies, but very late	A Swiggy rider stuck in traffic
Network partition	Two groups of nodes cannot talk to each other	A broken bridge between two towns

Failure type	What happens	Everyday analogy
Byzantine	A node lies or sends wrong data (bug or attacker)	A cashier who gives fake receipts

Key facts to say in an interview:

- **You cannot tell "slow" from "dead"**. If a node does not reply in 2 seconds, it may be dead, or the network may be slow. A **timeout** is only a guess.
- Most company systems assume **crash-recovery plus partitions**, not Byzantine faults. Blockchains (which you have tested) are the main place where Byzantine fault tolerance is used.
- **Partial failure** is the hard part. In one computer, things usually work fully or fail fully. In a distributed system, half the system can work while the other half is broken.

2. Replication: keeping copies

Replication means keeping the same data on several machines. Reasons: survive failures, serve more reads, and put data closer to users.

Leader-follower (single leader)

One node is the **leader**. All writes go to it. It sends a log of changes to **followers**. Reads can go to the leader (fresh) or to followers (maybe stale).

```
Client writes --> [Leader] --log--> [Follower 1]
                        --log--> [Follower 2]
Client reads  --> any node (followers may lag)
```

- **Synchronous** replication: the leader waits for followers before saying "OK". Safe, but slow, and one slow follower blocks writes.
- **Asynchronous** replication: the leader says "OK" at once. Fast, but if the leader dies, recent writes can be lost.
- Analogy: a cricket scorer (leader) writes every run, and the TV channels (followers) copy the score a few seconds later.

Multi-leader

Several nodes accept writes, usually one per region. They sync with each other later. Good for offline apps and multi-region writes. The big problem is **write conflicts**: two people edit the same document in two regions at the same moment. You need a conflict rule, such as last-write-wins (simple, can lose data) or merge logic (like Google Docs style merging, or CRDTs - data types designed to merge automatically).

Leaderless

Any replica accepts reads and writes (the Dynamo / Cassandra style). The client, or a coordinator, sends each write to N replicas and waits for some of them. This needs **quorums**.

3. Quorums: $R + W > N$

- **N** = number of replicas for a key.
- **W** = how many replicas must confirm a write.
- **R** = how many replicas you read from.

If $R + W > N$, every read set overlaps every write set in at least one node, so at least one replica you read has the latest write (it is identified by a version number).

Example with $N = 3$, $W = 2$, $R = 2$. A write reaches nodes A and B. A read asks B and C. B is in both groups, so the read sees the new value.

```
# Check if a quorum config guarantees overlap, and how many failures it tolerates.
def quorum_info(n, r, w):
    overlap = r + w > n
    write_tolerates = n - w    # nodes that can be down and writes still work
    read_tolerates = n - r
    return overlap, write_tolerates, read_tolerates

print(quorum_info(3, 2, 2)) # (True, 1, 1)
print(quorum_info(3, 1, 1)) # (False, 2, 2) fast but may read stale data
print(quorum_info(5, 3, 3)) # (True, 2, 2)
```

Warning: $R + W > N$ is not full "strong consistency". With concurrent writes, sloppy quorums (writing to backup nodes during failures) and clock-based conflict resolution, you can still see odd results. Say this in an interview; it shows depth.

4. Consensus: getting nodes to agree

Consensus means several nodes agree on one value (or one ordered list of values), even if some nodes crash. It is used for leader election, configuration, locks and replicated logs. Consensus works if a **majority** (more than half) is alive. With 5 nodes you can lose 2. With 4 nodes you can still lose only 1, so odd numbers are common.

Raft, step by step

Raft (Ongaro and Ousterhout, 2014) was designed to be easier to understand than Paxos. Each node is a **follower**, **candidate** or **leader**. Time is split into **terms** (term 1, term 2, ...), like numbered match innings.

1. **Start:** all nodes are followers. Each has a random **election timeout** (for example 150-300 ms). Randomness stops everyone from starting elections at the same moment.
2. **Election:** if a follower hears nothing from a leader before its timeout, it becomes a candidate, increases the term, votes for itself, and asks others for votes.
3. **Voting:** each node gives at most **one vote per term**, and only to a candidate whose log is at least as up to date as its own. This rule keeps committed data safe.
4. **Winner:** a candidate with votes from a majority becomes leader. It sends regular **heartbeats** so followers do not start new elections.

5. **Log replication:** a client sends a command to the leader. The leader appends it to its log and sends it to followers (AppendEntries messages).
6. **Commit:** when a majority has stored the entry, the leader marks it **committed**, applies it to its state machine, and replies to the client. Followers apply it later.
7. **Fixing followers:** if a follower's log differs, the leader finds the last matching entry and overwrites the rest. The leader's log always wins.
8. **Old leaders:** if an old leader comes back with a smaller term, it sees the higher term and steps down to follower.

```
Term 3: [Leader A] --AppendEntries(x=5)--> B, C, D, E
        B, C reply OK -> A + B + C = majority (3 of 5) -> x=5 committed
```

Analogy: a class elects a monitor. If the monitor stops talking for too long, someone stands up and asks for votes. You need more than half the class. The monitor writes decisions on the board, and a decision is final once most students have copied it.

Paxos intuition

Paxos (Leslie Lamport, "The Part-Time Parliament", 1998; "Paxos Made Simple", 2001) solves the same problem. The core idea in simple words:

- A **proposer** picks a unique, increasing proposal number and asks a majority of **acceptors** to "promise" not to accept older numbers (phase 1, prepare/promise).
- If acceptors already accepted some value, they report it, and the proposer **must** reuse the value with the highest number. This is how an already-chosen value survives.
- The proposer then asks acceptors to accept the value (phase 2, accept). Once a majority accepts, the value is **chosen**.

Any two majorities overlap, so two different values can never both be chosen. Multi-Paxos runs this for a sequence of log slots with a stable leader, which looks a lot like Raft. Google's Chubby and Spanner use Paxos (see the next chapter).

Leader election in practice

You rarely write Raft yourself. You use a coordination service (etcd, ZooKeeper, or at Google, Chubby) and take a **lease**: a lock with an expiry time. The holder must renew it. Danger: an old leader with a long GC pause may think it is still leader. The fix is a **fencing token**: every new lease gets a bigger number, and the storage rejects writes with an older number.

5. Consistent hashing with virtual nodes

Plain `hash(key) % N` moves almost every key when N changes. **Consistent hashing** puts servers and keys on a ring. A key belongs to the first server clockwise. Adding a server only moves keys from its neighbour. **Virtual nodes** place each server at many points on the ring, so load is more even and a failed server's keys spread across many servers.

```

import bisect, hashlib

def h(s): # stable hash (Python's hash() changes between runs)
    return int(hashlib.md5(s.encode()).hexdigest(), 16)

class Ring:
    def __init__(self, nodes, vnodes=100):
        self.ring = sorted((h(f"{n}#{i}"), n) for n in nodes for i in range(vnodes))
        self.keys = [k for k, _ in self.ring]

    def get(self, key):
        i = bisect.bisect(self.keys, h(key)) % len(self.keys) # wrap around
        return self.ring[i][1]

before = Ring(["A", "B", "C"])
after = Ring(["A", "B", "C", "D"])
users = [f"user{i}" for i in range(10000)]
moved = sum(before.get(u) != after.get(u) for u in users)
print(moved) # about 2500 (~1/4 of keys move), not ~7500 as with % N

```

Test idea: assert load per node is within, say, 10% of the average, and that adding one node moves roughly $1/(N+1)$ of keys.

6. Distributed transactions: 2PC and sagas

A **distributed transaction** changes data on several machines, all or nothing.

Two-phase commit (2PC)

1. **Prepare:** the coordinator asks every participant "Can you commit?" Each one locks its data, writes to disk, and votes yes or no.
2. **Commit:** if all say yes, the coordinator sends "commit"; otherwise "abort".

Problem: if the coordinator crashes after participants voted yes, they are **stuck holding locks** and cannot decide alone. 2PC is a **blocking** protocol. Running the coordinator on a consensus group (as Spanner does with Paxos) reduces this risk.

Sagas

A **saga** is a chain of local transactions. Each step has a **compensating action** that undoes it. Example, booking a trip: reserve train, then reserve hotel, then charge card. If the card charge fails, run "cancel hotel" and "cancel train".

	2PC	Saga
Consistency	Atomic, strong	Eventual; others can see middle states
Availability	Blocks on coordinator failure	Keeps going
Best for	Short transactions inside one database system	Long business flows across microservices

Analogy: 2PC is a wedding where everyone must say "yes" before anything happens. A saga is a UPI refund: money leaves first, and if something fails later, a refund is sent.

7. Clocks and ordering

Machine clocks drift. NTP corrects them, but not perfectly, and a clock can even jump backwards. So "use the timestamp" is dangerous for ordering events.

- **Lamport clock** (Lamport, 1978): each node keeps a counter. Increase it on every event. Send it with every message. On receiving, set `counter = max(local, received) + 1`. If A caused B, then $L(A) < L(B)$. The reverse is not guaranteed.
- **Vector clock**: each node keeps a list of counters, one per node. Comparing two vectors tells you if one event happened before the other, or if they are **concurrent** (a real conflict).

```
def compare(v1, v2): # vector clocks as dicts {node: counter}
    nodes = set(v1) | set(v2)
    le = all(v1.get(n, 0) <= v2.get(n, 0) for n in nodes)
    ge = all(v1.get(n, 0) >= v2.get(n, 0) for n in nodes)
    if le and not ge: return "v1 before v2"
    if ge and not le: return "v2 before v1"
    return "equal" if le else "concurrent (conflict)"

print(compare({"A": 1}, {"A": 2, "B": 1})) # v1 before v2
print(compare({"A": 2, "B": 0}, {"A": 1, "B": 1})) # concurrent (conflict)
```

- **TrueTime idea** (Spanner paper, 2012): instead of one timestamp, the clock API returns an **interval** `[earliest, latest]` with a known error bound, kept small using GPS and atomic clocks. Spanner waits until the uncertainty has passed ("commit wait") before making a commit visible. Then timestamp order matches real-time order. The lesson: **if you know your clock error, you can wait it out.**

8. Idempotency and the exactly-once myth

Networks lose replies. A client retries. Did the first request succeed? The client cannot know. So in practice you get **at-most-once** (no retry, may lose) or **at-least-once** (retry, may duplicate). True **exactly-once delivery** over an unreliable network is not possible.

What systems call "exactly-once" is really **exactly-once processing (effect)** = at-least-once delivery + **idempotent** handling. Idempotent means doing it twice has the same effect as doing it once.

```

processed = {} # in real life: a DB table with a unique key and expiry

def charge(idempotency_key, user, amount):
    if idempotency_key in processed:           # duplicate retry
        return processed[idempotency_key]      # return the same result
    result = f"charged {user} Rs {amount}"     # do the real work once
    processed[idempotency_key] = result
    return result

print(charge("k1", "sahil", 500)) # charged sahil Rs 500
print(charge("k1", "sahil", 500)) # charged sahil Rs 500 (no double charge)

```

This is how UPI and payment gateways avoid double charges when you press "Pay" twice. Important detail: the check and the work must be **atomic** (one transaction), or two parallel retries can both pass the check.

9. Backpressure

Backpressure means a slow consumer tells fast producers to slow down, instead of silently collecting an endless queue. Without it, queues grow, latency explodes, memory runs out, and the system crashes.

Tools: bounded queues (block or reject when full), rate limits, returning HTTP 429 or 503 with "retry later", and client retries with **exponential backoff and jitter** (random delay so all clients do not retry together). Analogy: a railway ticket counter closes the queue gate when the hall is full, rather than letting the crowd crush everyone.

10. How to TEST distributed systems

This is the section that makes you different from other candidates. Normal unit tests run on one machine with perfect timing. Distributed bugs appear only with **specific orderings of failures and messages**.

Fault injection

Deliberately cause failures and check behaviour. Kill a process, drop or delay packets (Linux `tc netem`), fill the disk, make DNS fail, return errors from a dependency. Each test has a **hypothesis**: "If one of three replicas dies, writes still succeed with W=2, and no acknowledged write is lost."

Jepsen-style testing

Jepsen (by Kyle Kingsbury) is a well-known open-source tool that tests real databases. The method:

1. Run many clients doing concurrent operations (read, write, compare-and-set).
2. Meanwhile a "nemesis" injects faults: partitions, clock skew, process pauses, crashes.
3. Record a **history**: every operation's start, end and result.
4. Run a **checker** that asks: "Is there any valid order of these operations that explains the results?" (for example, a linearizability check).

The key idea for you: **do not assert on one response; assert on the whole history against a consistency model**. Jepsen has found data-loss bugs in many popular databases this way.

Chaos engineering

Run controlled experiments in staging or production to build confidence. Steps: define "steady state" with metrics (success rate, latency), form a hypothesis, inject a real failure (kill an instance, cut a zone), limit the **blast radius** (how many users can be affected), and stop the experiment if metrics break. Netflix's Chaos Monkey is the famous example. The SWE at Google book's chapter on larger testing also lists disaster recovery and chaos engineering as a type of large test ([Chapter 14](#)).

Deterministic simulation

Run the whole distributed system **inside one process** with a fake network, fake disk and fake clock, all controlled by one random seed. The simulator picks message order, drops and crashes. If a run fails, re-running the same seed reproduces it exactly. FoundationDB made this approach famous.

```
import random
# Tiny idea: the seed controls message order, so any failure is replayable.
def simulate(seed, messages):
    rng = random.Random(seed)
    order = messages[:]
    rng.shuffle(order)                # simulated network reordering
    dropped = [m for m in order if rng.random() < 0.2] # simulated loss
    return order, dropped

print(simulate(42, ["vote", "append", "heartbeat"])) # same output every run
```

Other useful techniques

- **Property-based testing** (Hypothesis library in Python): generate random operation sequences, check invariants such as "balance never negative".
- **Invariant checks in production**: count rows in two systems and compare.
- **Linearizability checkers** (for example Knossos or Porcupine) for histories.
- **Formal methods** like TLA+ to check a design before coding.

Tester's corner

- A "flaky" test in a distributed system is often a **real race or ordering bug**. Before adding a retry, ask which two messages or nodes were racing.
- Always test the **unhappy paths**: leader dies mid-write, network split during an election, a client retries after a timeout.
- Assert on **histories and invariants** ("no acknowledged write lost", "no double charge"), not only on single responses.
- Use **fixed seeds and fake clocks** so failures are reproducible. A bug you cannot replay is very hard to fix.
- Check **idempotency** directly: send the same request twice, in parallel, and assert one effect.

- For consistent hashing or sharding, test **even load** and **minimal key movement** when nodes join or leave.
- In chaos tests, define the **abort condition** and blast radius before you start.

Key takeaways

- Agree on the failure model first; you cannot tell a slow node from a dead one.
- Replication can be leader-follower, multi-leader or leaderless; each trades consistency, latency and conflict handling.
- $R + W > N$ makes read and write sets overlap, but it is not full strong consistency.
- Raft uses terms, randomized timeouts, majority votes and log replication; Paxos uses prepare/accept with overlapping majorities.
- Consistent hashing with virtual nodes moves only about $1/N$ of keys when a node is added.
- 2PC is atomic but blocking; sagas are available but eventually consistent with compensations.
- Exactly-once delivery is a myth; at-least-once plus idempotency gives exactly-once effect.
- Test distributed systems with fault injection, history checking (Jepsen-style), chaos experiments and deterministic simulation.

Quiz

1. With $N = 5$ replicas, which setting guarantees read/write overlap? A) $R=2, W=2$ B) $R=1, W=4$ C) $R=3, W=3$ D) $R=1, W=1$
2. True or false: if a node does not reply within the timeout, it has definitely crashed.
3. In Raft, why are election timeouts randomized? A) To save CPU B) To reduce split votes when many followers time out together C) To encrypt votes D) To speed up log replication
4. In Raft, when is a log entry committed? A) When the leader writes it B) When any follower stores it C) When a majority of nodes store it D) When the client reads it
5. What is the main weakness of two-phase commit?
6. Two vector clocks are $\{A:2, B:0\}$ and $\{A:1, B:1\}$. What is their relationship? A) First happened before B) Second happened before C) Concurrent D) Equal
7. True or false: a message queue can guarantee exactly-once delivery over an unreliable network without any idempotency on the consumer.
8. A consumer is much slower than producers and memory keeps growing. What would you do?
9. A Jepsen-style test mainly checks: A) Code coverage B) Whether the recorded history of operations is valid under a consistency model while faults are injected C) UI rendering D) Unit test speed
10. A bug in your distributed test appears once in 500 runs and you cannot reproduce it. What technique helps most?

Answer key

1. **C** - $3 + 3 = 6 > 5$. Option B gives $1 + 4 = 5$, which is not strictly greater than 5, so a read and a write can miss each other.

2. **False** - The node may be slow, paused, or the network may be partitioned. A timeout is only a suspicion.
3. **B** - Random timeouts make it likely that one node starts an election first and wins, instead of many candidates splitting votes.
4. **C** - An entry is committed once a majority has stored it; then it survives any minority of failures.
5. **Blocking** - If the coordinator crashes after participants vote yes, they hold locks and cannot decide on their own until it recovers.
6. **C** - A is higher in the first, B is higher in the second, so neither happened before the other: concurrent.
7. **False** - Lost acknowledgements force retries, so delivery is at-least-once. You need idempotent processing for an exactly-once effect.
8. **Add backpressure** - Use a bounded queue, reject or slow producers (429/503), let clients back off with jitter, and scale or speed up consumers.
9. **B** - Jepsen records concurrent operations under faults and checks the full history against a model such as linearizability.
10. **Deterministic simulation** - Run with a fake network and clock driven by a seed, so the failing seed replays the exact same ordering every time.

Flashcards

- **Q:** What does $R + W > N$ guarantee? — **A:** Every read quorum overlaps every write quorum in at least one replica.
- **Q:** How many failures can a 5-node consensus group tolerate? — **A:** 2, because a majority of 3 must stay alive.
- **Q:** What are the three Raft roles? — **A:** Follower, candidate and leader.
- **Q:** What is a Raft term? — **A:** A numbered period with at most one leader; higher terms override lower ones.
- **Q:** What is a fencing token? — **A:** An increasing number given with each lease so storage can reject writes from an old leader.
- **Q:** Why use virtual nodes in consistent hashing? — **A:** To spread load evenly and spread a failed node's keys across many nodes.
- **Q:** What is a saga? — **A:** A chain of local transactions where each step has a compensating action to undo it.
- **Q:** What can a vector clock detect that a Lamport clock cannot? — **A:** Whether two events are concurrent (a true conflict).
- **Q:** What is the TrueTime idea? — **A:** The clock returns an interval with bounded error, and the system waits out the uncertainty before committing.
- **Q:** How do you get an exactly-once effect? — **A:** At-least-once delivery plus idempotent processing with a unique key.
- **Q:** What is deterministic simulation testing? — **A:** Running the whole system with fake network, disk and clock driven by a seed so any failure replays exactly.

Google's Classic Papers Explained: GFS, MapReduce, Bigtable, Chubby, Dapper, Spanner, Borg

In this chapter:

- Understand the problem each famous Google paper solved, in simple words
- Draw the core design of each system from memory
- Explain the key trade-off behind each design
- Use these ideas as building blocks in system design interviews
- Know exactly where each official paper lives, so you can read it yourself

Time: ~70 minutes | **Level:** Expert

Google published a series of research papers that shaped how the whole industry builds large systems. Hadoop copied GFS and MapReduce. HBase and Cassandra borrowed from Bigtable. ZooKeeper was influenced by Chubby. Zipkin and OpenTelemetry follow the Dapper model. Kubernetes came from lessons learned with Borg.

You do not need to memorize every detail. In an interview, you need to (1) know the main idea, (2) reuse the pattern when it fits, and (3) explain the trade-off. This chapter gives you that for seven papers.

Accuracy note: everything here comes from the public papers listed in each section. Google's systems have changed since these papers were written, so always say "according to the 2006 paper" and not "this is how Google works today".

Year	System	One-line summary
2003	GFS	A file system for huge files on cheap machines
2004	MapReduce	Simple batch processing on thousands of machines
2006	Bigtable	A huge sorted key-value table on top of GFS
2006	Chubby	A lock service and tiny file store, built on Paxos
2010	Dapper	Tracing requests across many services
2012	Spanner	A global database with strong consistency, using TrueTime
2015	Borg	The cluster manager that runs everything

1. GFS - The Google File System (2003)

Paper: Ghemawat, Gobioff and Leung, "The Google File System", SOSP 2003.

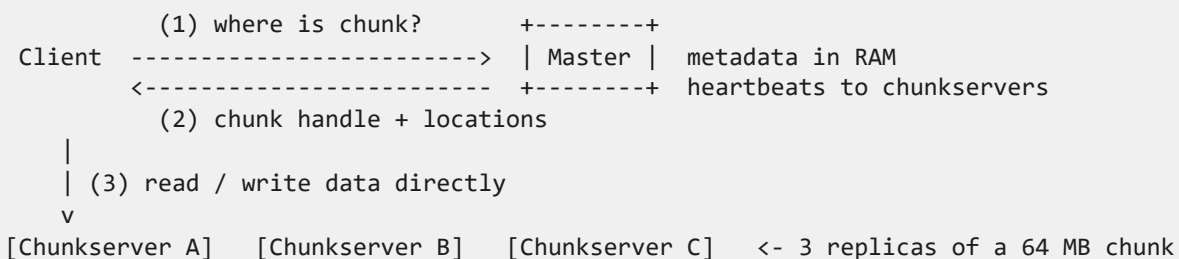
URL: <https://static.googleusercontent.com/media/research.google.com/en/archive/gfs-sosp2003.pdf>

Problem it solved

Google needed to store very large files (many gigabytes) for crawling and indexing, on hundreds or thousands of **cheap commodity machines** where disk and machine failures are normal, every day. The workload was mostly **large streaming reads** and **appending** new data to the end of files, not random small writes.

Core design in simple words

- Files are split into fixed-size **chunks** of 64 MB.
- Each chunk is stored on several **chunkservers** - three copies (replicas) by default.
- A single **master** keeps all **metadata** in memory: the file names, which chunks make up each file, and where each chunk's replicas live.
- A client asks the master "where is chunk 7 of this file?", then reads the data **directly from a chunkserver**. Data never flows through the master, so the master does not become a bandwidth bottleneck.
- The master logs metadata changes in an **operation log** that is replicated to other machines, and it checkpoints its state, so it can recover after a crash.
- For writes, the master gives one replica a **lease** to act as the **primary**, which decides the order of changes for that chunk.
- GFS offers **record append**: many clients can append to the same file at once. Each record is written at least once, so applications must handle duplicates (for example with unique record IDs and checksums).



Analogy: the master is the railway enquiry counter. It tells you which platform (chunkserver) your train (chunk) is on. You then walk to the platform yourself. The enquiry counter does not carry your luggage.

Key trade-offs

- **Single master = simple design**, global knowledge for placement and re-replication. Risk: it is a single point of failure and a scaling limit. GFS reduces this with small metadata, client caching of locations, a replicated log and read-only "shadow" masters.
- **Relaxed consistency**: GFS chose speed and simplicity over strict guarantees. Applications must deal with duplicates and padding.
- **Large chunks** reduce metadata and master traffic, but small files can create hot spots.

What it teaches for interviews

- Separate the **control plane** (metadata) from the **data plane** (bytes). This pattern appears in object storage, CDNs and test artifact stores.
- **Design for failure as normal**: heartbeats, re-replication, checksums.
- When you design a file storage system (like Google Drive), say: "metadata service + chunk storage with replication, clients go direct to storage".

2. MapReduce (2004)

Paper: Dean and Ghemawat, "MapReduce: Simplified Data Processing on Large Clusters", OSDI 2004.

URL: <https://static.googleusercontent.com/media/research.google.com/en//archive/mapreduce-osdi04.pdf>

Problem it solved

Engineers kept writing custom programs to process huge datasets (crawled pages, logs) across many machines. Each program had to handle splitting data, parallelism, machine failures and load balancing. MapReduce hid all this behind two simple functions that a normal engineer could write.

Core design in simple words

- The user writes **map(key, value)**, which outputs intermediate (key, value) pairs, and **reduce(key, list of values)**, which combines all values for one key.
- The library splits input into **M map tasks** and partitions intermediate keys into **R reduce tasks** (by default $\text{hash}(\text{key}) \bmod R$).
- A **master** assigns tasks to idle **workers**. Map workers write output to their **local disk**. Reduce workers fetch their partition from all map workers (the **shuffle**), sort by key and call reduce.
- **Failure handling**: the master pings workers. If a worker dies, its tasks are run again elsewhere. Even finished map tasks are re-run, because their output was on the dead machine's local disk.
- **Stragglers**: near the end of a job, the master starts **backup copies** of the remaining tasks. Whichever copy finishes first wins. The paper reports this makes some jobs much faster.
- **Locality**: the master tries to run map tasks on machines that already hold a GFS replica of the input, to save network bandwidth.
- An optional **combiner** does a partial reduce on the map side (for example, summing word counts locally).

Input splits	Map tasks	Shuffle (by key)	Reduce tasks	Output
[split 1] -->	[map] --(a,1)(b,1)--+		+++> [reduce a]	--> part-0
[split 2] -->	[map] --(a,1)(c,1)--	group by key	+++> [reduce b]	--> part-1
[split 3] -->	[map] --(b,1)(c,1)--+		+++> [reduce c]	--> part-2

```

from collections import defaultdict

def map_fn(doc):
    # emit (word, 1) for each word
    return [(w, 1) for w in doc.split()]

def reduce_fn(word, counts):
    # sum all counts for one word
    return word, sum(counts)

docs = ["upi pay upi", "pay later", "upi"]
groups = defaultdict(list)
# the "shuffle" step
for d in docs:
    for k, v in map_fn(d):
        groups[k].append(v)
print(sorted(reduce_fn(k, v) for k, v in groups.items()))
# [('later', 1), ('pay', 2), ('upi', 3)]

```

Key trade-offs

- Very simple model, automatic fault tolerance. But it is **batch only** with high latency, and writing intermediate data to disk is slow for multi-step pipelines. Later systems (for example Google's FlumeJava and Dataflow, and Apache Spark outside Google) improved on this.
- Re-execution only works cleanly if map and reduce are **deterministic**. The paper notes that deterministic functions produce the same output as a non-faulty sequential run.

What it teaches for interviews

- For "count top search queries per day" or "build an inverted index", describe a **map, shuffle, reduce** pipeline.
- **Re-execution** as fault tolerance needs idempotent, deterministic tasks.
- **Backup tasks** for stragglers is the same idea as sending hedged requests, and it is a great idea for test sharding too (re-run the slowest shard).

3. Bigtable (2006)

Paper: Chang et al., "Bigtable: A Distributed Storage System for Structured Data", OSDI 2006.

URL: <https://static.googleusercontent.com/media/research.google.com/en//archive/bigtable-osdi06.pdf>

Problem it solved

Many Google products (the paper mentions web indexing, Google Earth and Google Finance) needed to store petabytes of structured data with very different needs: some wanted high write throughput, some low-latency reads. A relational database did not scale to this size on commodity machines.

Core design in simple words

- The paper describes Bigtable as a **sparse, distributed, persistent, multi-dimensional sorted map**. The map is indexed by **(row key, column key, timestamp)** and the value is uninterpreted bytes.

- Rows are kept **sorted by row key**. The paper's "Webtable" example uses reversed URLs like `com.cnn.www` so pages from the same domain sit next to each other.
- Columns are grouped into **column families**. Each cell can keep several **timestamped versions**.
- Reads and writes to **one row are atomic**. There are no general multi-row transactions.
- A table is split into row ranges called **tablets**. Each tablet is served by one **tablet server**. A **master** assigns tablets to servers and balances load. Clients talk to tablet servers directly.
- **Write path**: a write goes to a commit log (stored on GFS) and to an in-memory **memtable**. When the memtable is full, it is written out as an immutable **SSTable** file on GFS. Background **compactions** merge SSTables. **Bloom filters** help skip SSTables that cannot contain a key.
- **Chubby** is used to elect the master, track live tablet servers, and store the location of the root tablet.

```
Client --(find tablet: Chubby -> root tablet -> METADATA)--> [Tablet server 3]
      |
      v
write -> commit log (GFS) + memtable (RAM) --flush--> SSTables (GFS) --compaction--> merged
SSTable
[Master] assigns tablets, balances load      [Chubby] master lock, server liveness
```

Analogy: a huge phone directory, sorted by name. The directory is cut into volumes (tablets), each kept by a different librarian (tablet server). New entries go first into a notebook (memtable), and are later copied neatly into a printed volume (SSTable).

Key trade-offs

- **Single-row atomicity only**: simple and scalable, but applications must design row keys carefully.
- **Sorted keys** give fast range scans, but sequential keys (like timestamps at the start of the key) can create a **hot tablet**.
- Building on GFS and Chubby reused existing systems, but created dependencies.

What it teaches for interviews

- The **log-structured merge (LSM) tree** pattern - log + memtable + immutable sorted files + compaction - is behind Bigtable, HBase, Cassandra, RocksDB and LevelDB.
- **Row key design is the main skill**: group what you read together, avoid hot spots (add a hash prefix or reverse a key).
- Google Cloud offers Cloud Bigtable as a public product, so you can name it as a wide-column store for time series, IoT and test result data.

4. Chubby (2006)

Paper: Burrows, "The Chubby lock service for loosely-coupled distributed systems", OSDI 2006.

URL: <https://static.googleusercontent.com/media/research.google.com/en//archive/chubby-osdi06.pdf>

Problem it solved

Many systems (GFS and Bigtable, for example) needed to **elect a leader** or store a small piece of configuration that everyone agrees on. Asking every team to implement Paxos correctly is risky. Chubby gives them a simple **lock service** instead.

Core design in simple words

- A Chubby **cell** is a small group of replicas (the paper describes typically **five**). They use **Paxos** to elect a **master** and to replicate data.
- The interface looks like a **simple file system**: directories and small files. Any file can also be used as a **reader/writer lock**.
- Locks are **coarse-grained**: held for hours or days (for example "I am the Bigtable master"), not for milliseconds.
- Clients hold a **session** kept alive with **KeepAlive** messages. If the session expires, the client loses its locks.
- Clients **cache** file data. The master sends **invalidations** before a file changes, so caches stay consistent.
- **Sequencers**: a lock holder can pass a token describing the lock to servers, which check it - the same idea as a fencing token.
- Many teams also used Chubby as a **name service** to find servers.

```
Clients (GFS master, Bigtable servers, ...)
  | KeepAlive / lock / read small file
  v
+----- Chubby cell (5 replicas) -----+
| [Master] <--Paxos--> [Replica] [Replica] [Replica] [Replica] |
+-----+
```

Analogy: a society's notice board guarded by five committee members who vote. If you want to be "event organiser" (leader), you put your name on the board. Everyone checks the board instead of arguing.

Key trade-offs

- **Availability and reliability over raw speed**: all reads and writes go to the master; writes need a majority.
- A **lock service** is easier for developers than a consensus library, but Chubby becomes a shared dependency that many systems rely on.
- Coarse-grained locks keep load low; fine-grained locking is left to applications.

What it teaches for interviews

- When you need leader election or config, say "use a consensus-backed coordination service like Chubby, ZooKeeper or etcd" instead of inventing one.
- Mention **leases, sessions and fencing tokens** to handle a paused old leader.

5. Dapper (2010)

Paper: Sigelman et al., "Dapper, a Large-Scale Distributed Systems Tracing Infrastructure", Google Technical Report, 2010.

URL: <https://static.googleusercontent.com/media/research.google.com/en//archive/papers/dapper-2010-1.pdf>

Problem it solved

One web search touches many services. When a request is slow, which service is the cause? Logs from each machine do not show the full path. Dapper records the path of a request across services.

Core design in simple words

- A **trace** is a tree of **spans**. A span is one unit of work (for example one RPC) with a span ID, a parent span ID, a shared trace ID, start and end times, and optional **annotations** (notes added by the application).
- Dapper aimed for **application-level transparency**: instrumentation was added to a small set of **common libraries** (RPC, threading, control flow), so most developers did not need to change code.
- **Low overhead** through **sampling**: only a fraction of requests are traced (the paper discusses a uniform rate such as 1 in 1,024, and adaptive sampling).
- Span data is written to local log files, then collected by daemons into a central store (Bigtable in the paper) for analysis.

```
Trace 42
[frontend 0-120 ms]
|-- [search backend 10-90 ms]
|   |-- [index shard 1 15-60 ms]
|   |-- [index shard 2 15-85 ms]    <- slowest child
|-- [ads service 10-40 ms]
```

Analogy: a Swiggy order tracker that shows each step - restaurant accepted, food ready, rider picked up, delivered - with times, so you see exactly where the delay was.

Key trade-offs

- **Sampling** keeps cost low but can miss rare slow requests. That is why tracing is combined with metrics and logs.
- Library-level instrumentation gives wide coverage, but only works if everyone uses the common libraries.

What it teaches for interviews

- In any microservices design, mention **distributed tracing with trace IDs propagated in request headers**, sampled to control cost.
- Public tools such as Zipkin and OpenTelemetry follow this span-and-trace model.

6. Spanner (2012)

Paper: Corbett et al., "Spanner: Google's Globally-Distributed Database", OSDI 2012.

URL: <https://static.googleusercontent.com/media/research.google.com/en//archive/spanner-osdi2012.pdf>

Problem it solved

Bigtable scaled well but lacked cross-row transactions and strong consistency across data centres. Teams (the paper mentions F1, the backend for Google's advertising business) wanted **SQL-like queries, transactions and global replication** together.

Core design in simple words

- Data is split into many shards, and each shard is replicated across data centres using a **Paxos group**. Each group has a long-lived **leader**.
- Transactions that touch several Paxos groups use **two-phase commit**, where each participant is itself a Paxos group. This makes 2PC much less likely to block.
- The paper's key idea is **TrueTime**: an API whose `TT.now()` returns an interval `[earliest, latest]`. The true time is guaranteed to be inside. Google kept the uncertainty small using **GPS receivers and atomic clocks** in each data centre.
- **Commit wait**: after picking a commit timestamp, the leader waits until TrueTime is sure that timestamp is in the past before making the commit visible. Result: if transaction T1 commits before T2 starts, T1 gets a smaller timestamp. The paper calls this **external consistency**.
- Read-only transactions and snapshot reads at a timestamp can run **without locks**, on any sufficiently up-to-date replica.

```
TT.now() = [earliest ----- latest]    (width = clock uncertainty)
Commit: pick timestamp s >= latest
        wait until TT.now().earliest > s    <- "commit wait"
        then release locks and reply
Data:   shard A -> Paxos group (US, EU, Asia)    shard B -> Paxos group
        cross-shard transaction -> 2PC across Paxos groups
```

Analogy: two cricket umpires with slightly different watches. If both know their watch error is at most 5 seconds, they wait 5 seconds before announcing a decision. Then the order of announcements is always correct.

Key trade-offs

- Strong consistency and global transactions cost **write latency** (cross-region Paxos plus commit wait).
- TrueTime needs **special hardware** and careful operations. Without a bounded clock error, the design does not hold.
- In CAP terms, Spanner is a CP system that is engineered to be highly available in practice.

What it teaches for interviews

- For money, inventory and bookings across regions, a Spanner-like database (Cloud Spanner is a public product) is a strong choice: "I pay some write latency for correctness".

- Explain the general lesson: **known clock uncertainty can be waited out**.

7. Borg (2015)

Paper: Verma et al., "Large-scale cluster management at Google with Borg", EuroSys 2015.

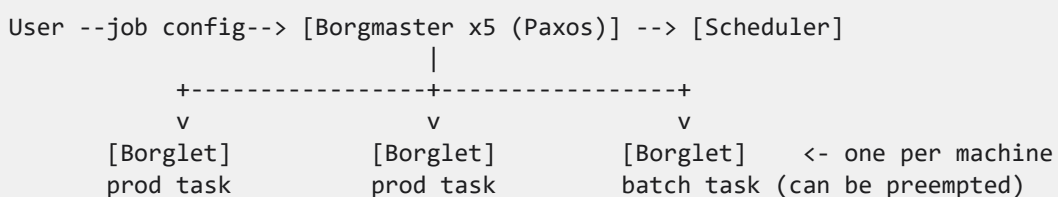
URL: <https://static.googleusercontent.com/media/research.google.com/en//pubs/archive/43438.pdf>

Problem it solved

Google runs a huge number of applications. Giving each one its own machines wastes resources. Borg is a **cluster manager**: users describe what to run, and Borg decides where it runs, restarts it if it fails, and packs many jobs onto shared machines.

Core design in simple words

- Users submit **jobs**. A job has many identical **tasks** (for example 100 copies of a web server). The paper describes **long-running services** ("prod", latency-sensitive) and **batch jobs** ("non-prod") sharing the same machines.
- Machines are grouped into **cells**. Each cell has a **Borgmaster** and a **Borglet** agent on every machine.
- The Borgmaster is **replicated five times** and uses **Paxos** to elect a leader and store state.
- A **scheduler** finds feasible machines for each pending task and scores them. Tasks have **priorities**; a high-priority prod task can **preempt** (evict) lower-priority batch work. **Quota** limits how much each team can ask for.
- Borg's name service (BNS) writes task locations into Chubby so clients can find them.
- The paper lists lessons that Kubernetes later adopted, such as labels instead of rigid job groupings, and one IP address per pod instead of sharing the machine's IP.



Analogy: a big wedding hall manager. Families (teams) request rooms and staff. VIP functions (prod) get priority; small events (batch) fill empty space and can be moved when a VIP arrives.

Key trade-offs

- Mixing prod and batch on the same machines **raises utilization** a lot, but needs good isolation and preemption so batch work does not hurt latency.
- A central, replicated master is simpler than a fully decentralized design, but must be engineered to scale.

What it teaches for interviews

- In any design, mention that services run as **replicated tasks on a cluster manager** (Kubernetes outside Google), with health checks and automatic restarts.
- For a test execution farm, Borg-style ideas apply directly: priorities (presubmit above nightly runs), quotas per team, and preemptible batch work.

How the papers connect

```
Borg runs everything --> GFS stores files --> MapReduce processes them
                        Chubby elects masters for GFS / Bigtable
                        Bigtable sits on GFS + Chubby
                        Spanner adds transactions + TrueTime
                        Dapper traces requests through all of it
```

Tester's corner

- Each paper is mostly about **failure handling**. When you read one, list its failure cases and how it detects them. That list is a test plan.
- GFS record append and MapReduce re-execution both create **duplicates**. Test downstream consumers for idempotency.
- MapReduce **backup tasks** inspire test infrastructure: re-run the slowest shard on another worker and take the first result.
- Bigtable teaches **hot-spot testing**: load test with realistic key distributions, not random keys.
- Dapper shows why tests in microservice systems should **log trace IDs**. A failing end-to-end test with a trace ID is much faster to debug.
- Spanner shows that **clock assumptions** are a source of bugs. Test with skewed clocks.
- Borg-style **preemption** means your test workers can disappear at any time. Your test runner must handle it.

Key takeaways

- GFS: single metadata master, 64 MB chunks, three replicas, clients read data directly from chunkservers.
- MapReduce: map, shuffle, reduce; fault tolerance by re-execution; backup tasks for stragglers.
- Bigtable: sorted map of (row, column, timestamp); tablets; memtable plus SSTables on GFS; single-row atomicity.
- Chubby: Paxos-backed lock service and small file store, typically five replicas, coarse-grained locks.
- Dapper: traces as trees of spans, library-level instrumentation, sampling for low overhead.
- Spanner: Paxos groups plus 2PC, TrueTime intervals and commit wait give external consistency.
- Borg: jobs and tasks in cells, replicated Borgmaster, priorities, quota and preemption; lessons fed into Kubernetes.
- In interviews, say "according to the paper" and reuse the pattern, not internal details.

Quiz

1. In GFS, does file data flow through the master? A) Yes, always B) No, clients read and write data directly with chunkservers C) Only for reads D) Only for appends
2. Why does MapReduce re-run completed map tasks when a worker dies?
3. What is a MapReduce "backup task"? A) A copy of input data B) A duplicate run of a slow remaining task near the end of a job C) A database backup D) A retry of the master
4. Bigtable is indexed by: A) Row key only B) Row key and column key C) Row key, column key and timestamp D) Primary key and foreign key
5. True or false: Bigtable supports atomic reads and writes within a single row.
6. What is Chubby mainly used for? A) Storing large videos B) Coarse-grained locking, leader election and small config files C) Running batch jobs D) Tracing requests
7. In Dapper, what is a span?
8. What does TrueTime's `TT.now()` return? A) An exact timestamp B) An interval [earliest, latest] that contains the true time C) A Lamport counter D) A vector clock
9. True or false: In Borg, prod and batch jobs always run on separate machines.
10. You are designing a test result store with heavy writes and range scans by test ID and time. Which paper's design would you borrow, and what row key would you pick?

Answer key

1. **B** - The master only returns chunk locations; data moves directly between clients and chunkservers, so the master is not a bandwidth bottleneck.
2. **Local disk** - Map output is stored on the worker's local disk, so when that machine dies, the output is lost and must be recomputed.
3. **B** - Near the end, the master schedules duplicate executions of in-progress tasks; whichever finishes first is used, cutting straggler delay.
4. **C** - The paper describes a sorted map indexed by row key, column key and timestamp.
5. **True** - Bigtable gives single-row atomicity but no general multi-row transactions.
6. **B** - Chubby provides coarse-grained locks and a small file system, used by GFS and Bigtable for master election and metadata.
7. **One unit of work** - A span records one operation (like an RPC) with trace ID, span ID, parent ID, timing and annotations; spans form a trace tree.
8. **B** - TrueTime returns an uncertainty interval, and Spanner waits out the uncertainty (commit wait) before making commits visible.
9. **False** - The Borg paper describes mixing prod and non-prod work on shared machines to raise utilization, with priorities and preemption.
10. **Bigtable** - Use a wide-column, LSM-style store with row key like `test_id#reverse_timestamp` (or `test_id#run_time`) so one test's history is contiguous and newest first; add a hash prefix if one test is a hot spot.

Flashcards

- **Q:** What chunk size and replica count did the GFS paper describe? — **A:** 64 MB chunks with three replicas by default.
- **Q:** What is the main GFS design pattern to reuse? — **A:** Separate the metadata master (control plane) from the data servers (data plane).
- **Q:** What are the three phases of MapReduce? — **A:** Map, shuffle (group by key), reduce.
- **Q:** How does MapReduce handle stragglers? — **A:** It runs backup copies of the last in-progress tasks and takes the first to finish.
- **Q:** What is a Bigtable tablet? — **A:** A contiguous range of rows, the unit of distribution and load balancing.
- **Q:** What is the Bigtable write path? — **A:** Commit log plus memtable, flushed to immutable SSTables, merged by compaction.
- **Q:** How many replicas does a typical Chubby cell have, and what protocol? — **A:** Five replicas using Paxos.
- **Q:** What made Dapper low-overhead? — **A:** Sampling only a fraction of requests and instrumenting shared libraries.
- **Q:** What is Spanner's commit wait? — **A:** Waiting until TrueTime guarantees the commit timestamp has passed before making the commit visible.
- **Q:** What does Spanner use for cross-shard transactions? — **A:** Two-phase commit across Paxos groups.
- **Q:** What are the main Borg components? — **A:** A five-way replicated Borgmaster with a scheduler, and a Borglet agent on each machine in a cell.
- **Q:** Which open-source system grew from Borg's lessons? — **A:** Kubernetes.

Reliability, SRE and Observability: SLOs, Alerts, Incidents and Safe Releases

In this chapter:

- Define SLIs, SLOs and SLAs, and calculate an error budget and burn rate by hand and in Python
- Monitor the four golden signals and alert on symptoms, not causes
- Use logs, metrics and traces together to debug
- Run incidents, write blameless postmortems, and release safely with canaries and rollbacks
- Protect systems with load shedding, graceful degradation and capacity planning

Time: ~70 minutes | **Level:** Advanced

In the earlier lesson on reliability you met SLOs, error budgets, retries and circuit breakers. This chapter goes deeper and follows Google's two public SRE books:

- *Site Reliability Engineering: How Google Runs Production Systems* (O'Reilly, 2016), free at <https://sre.google/sre-book/table-of-contents/>
- *The Site Reliability Workbook* (O'Reilly, 2018), free at <https://sre.google/workbook/table-of-contents/>

SRE (Site Reliability Engineering) is, in the book's words, what happens when you ask a software engineer to design an operations team. SREs treat running production as a software problem.

Why should a test engineer care? Because testing does not stop at release. An SLO is like a test that runs on real users all the time. A canary is a test in production. A postmortem is a bug report for the whole system. Interviewers for SWE-Test roles like candidates who think about quality **after** deployment too.

1. SLIs, SLOs and SLAs

Source: SRE book, Chapter 4 "Service Level Objectives" (<https://sre.google/sre-book/service-level-objectives/>).

- **SLI (Service Level Indicator):** a number that measures one aspect of service from the user's point of view. Usually a ratio: good events divided by total events. Example: "proportion of search requests served successfully in under 300 ms".
- **SLO (Service Level Objective):** a target for an SLI over a time window. Example: "99.9% of requests succeed over 30 days".
- **SLA (Service Level Agreement):** a contract with users that says what happens (refunds, credits) if you miss a target. SLAs are business documents. They should be looser than your internal SLO, so you get a warning before you break a promise.

Analogy: for an Indian Railways train, the SLI is "percentage of trains arriving within 15 minutes of schedule". The SLO is "95% of trains on time this month". The SLA is "if the train is more than 3 hours late, you get a refund".

Good SLIs (from the book and Workbook Chapter 2 "Implementing SLOs", <https://sre.google/workbook/implementing-slos/>):

Service type	Typical SLIs
User-facing request/response (API, website)	Availability, latency, quality
Data pipeline	Freshness, correctness, coverage
Storage	Durability, latency, availability

Two important rules:

- **100% is the wrong target.** Users cannot tell 99.999% from 100% because their own phone network fails more often. Every extra "9" costs a lot of money and slows down releases (SRE book, Chapter 3 "Embracing Risk", <https://sre.google/sre-book/embracing-risk/>).
- Measure **percentiles**, not averages, for latency. An average of 100 ms can hide a p99 of 5 seconds.

2. Error budgets with a worked calculation

The **error budget** is the amount of unreliability you are allowed: $1 - \text{SLO}$. If your SLO is 99.9%, your budget is 0.1%.

The book's key idea: the error budget turns a fight ("developers want speed, ops wants stability") into a shared number. While budget remains, teams can release new features quickly. When the budget is spent, releases slow down or stop, and the team focuses on reliability.

Worked example. A payments API has an SLO of 99.9% successful requests over 30 days. It serves 200 million requests per month.

1. Budget fraction = $1 - 0.999 = 0.001$.
2. Allowed failed requests = $200,000,000 \times 0.001 = \mathbf{200,000 \text{ failed requests}}$.
3. As time: 30 days $\times$ 24 h $\times$ 60 min = 43,200 minutes. $43,200 \times 0.001 = \mathbf{43.2 \text{ minutes}}$ of full outage.
4. An incident on day 10 causes 120,000 failed requests. Budget used = $120,000 / 200,000 = \mathbf{60\%}$. Only 40% is left for 20 more days, so the team should slow risky releases.

Burn rate is how fast you use budget compared to "exactly on schedule". Burn rate 1 means you will use 100% of the budget in exactly 30 days. Burn rate 10 means you will use it in 3 days.

```
def error_budget(slo, total_requests, window_days=30):
    budget = 1 - slo
    return {
        "allowed_failures": round(total_requests * budget),
        "allowed_downtime_min": round(window_days * 24 * 60 * budget, 1),
    }

def burn_rate(error_ratio, slo):
    return error_ratio / (1 - slo)    # 1.0 = exactly on budget

print(error_budget(0.999, 200_000_000))
# {'allowed_failures': 200000, 'allowed_downtime_min': 43.2}
print(burn_rate(0.0144, 0.999))    # about 14.4 -> burns 2% of a 30-day budget per hour
print(burn_rate(0.0005, 0.999))    # about 0.5 -> healthy
```

Quick downtime table for a 30-day window: 99% = 7.2 hours; 99.9% = 43.2 minutes; 99.99% = about 4.3 minutes; 99.999% = about 26 seconds.

3. Monitoring: the four golden signals

Source: SRE book, Chapter 6 "Monitoring Distributed Systems" (<https://sre.google/sre-book/monitoring-distributed-systems/>).

The book says that if you can measure only four metrics of a user-facing system, measure these:

1. **Latency** - how long requests take. Track successful and failed requests separately. A fast error is still an error, and a slow error is even worse.
2. **Traffic** - how much demand: requests per second, or transactions per second.
3. **Errors** - the rate of failed requests: explicit (HTTP 500), implicit (HTTP 200 with wrong content), or by policy (slower than your promise).
4. **Saturation** - how "full" the service is: CPU, memory, disk, queue length, thread pool use. Many systems slow down before 100% use, so saturation predicts trouble.

Analogy: a Swiggy kitchen. Latency is how long each order takes. Traffic is orders per minute. Errors are wrong or cancelled orders. Saturation is how many burners are busy.

The book also describes two styles:

- **White-box monitoring**: metrics from inside the system (counters, queue sizes). Good for finding causes and predicting problems.
- **Black-box monitoring**: testing behaviour from outside, like a user would (probers). Good for detecting what users actually feel. A prober is basically an automated end-to-end test running against production.

4. Alerting on symptoms, not causes

The same chapter separates **symptoms** ("what is broken": users see errors) from **causes** ("why": a database is slow). Page a human on symptoms that hurt users. Use cause metrics to debug, not to wake

people up.

Rules for good alerts, based on Chapter 6 and Chapter 10 "Practical Alerting" (<https://sre.google/sre-book/practical-alerting/>):

- Every page must be **urgent, actionable and real**. If the on-call engineer cannot do anything, it should not page.
- Noisy alerts cause **alert fatigue**: people start ignoring them. This is exactly like flaky tests making people ignore red builds. The SWE at Google book's CI chapter makes the same comparison ("CI is alerting", <https://abseil.io/resources/swe-book/html/ch23.html>).
- Use tickets or dashboards for slow, non-urgent issues.

Alerting on SLOs with burn rates. The Workbook, Chapter 5 "Alerting on SLOs"

(<https://sre.google/workbook/alerting-on-slos/>), recommends multi-window, multi-burn-rate alerts. Its example starting values for a 30-day SLO:

Budget consumed	Long window	Burn rate	Action
2%	1 hour	14.4	Page
5%	6 hours	6	Page
10%	3 days	1	Ticket

Check the maths: 30 days = 720 hours. Burning at 14.4 for 1 hour uses $14.4 / 720 = 2\%$ of the budget. The Workbook also adds a **short window** (for example 5 minutes for the 1-hour alert) so the alert stops quickly after the problem is fixed.

5. Logs, metrics and traces

These three are called the **pillars of observability**. **Observability** means you can understand what is happening inside a system from the data it produces, even for problems you did not predict.

Signal	What it is	Best for	Cost
Metrics	Numbers over time (counters, gauges, histograms)	Dashboards, alerts, trends	Cheap, aggregated
Logs	Event records, ideally structured (JSON)	Details of one error	Expensive at volume
Traces	A request's path across services as spans	Finding which service is slow	Usually sampled

A typical debugging flow: an **SLO alert fires** (metrics) -> the latency dashboard shows the checkout service is slow -> a **trace** of a slow request shows the inventory call takes 2 seconds -> **logs** for that trace ID show "connection pool exhausted". Put the same **trace ID** in logs and traces so you can jump between them. Google's Dapper paper (2010) is the classic public description of tracing; see the previous chapter.

Watch out for **high cardinality**: a metric label like `user_id` creates millions of time series and can overload the metrics system. Put per-user detail in logs or traces instead.

6. Incident response

Source: SRE book, Chapter 14 "Managing Incidents" (<https://sre.google/sre-book/managing-incidents/>), and Chapter 13 "Emergency Response" (<https://sre.google/sre-book/emergency-response/>).

An **incident** is an unplanned event that hurts users or risks hurting them. The book describes clear roles so people do not all fight the fire randomly:

- **Incident Commander (IC):** owns the overall incident, assigns roles, makes decisions.
- **Operations lead:** the only team that changes the system during the incident.
- **Communications lead:** sends regular updates to stakeholders.
- **Planning lead:** handles longer-term support, like filing bugs and arranging handoffs.

Other practices from the book: keep a **live incident document**, hand off clearly when people change shifts, and declare an incident **early** rather than late.

Priority during an incident: **mitigate first, find the root cause later**. Roll back the release, drain traffic from a bad region, or turn off a feature flag. Analogy: if a pipe bursts at home, you close the main valve first. You find out why the pipe burst afterwards.

7. Blameless postmortems

Source: SRE book, Chapter 15 "Postmortem Culture: Learning from Failure" (<https://sre.google/sre-book/postmortem-culture/>).

A **postmortem** is a written record of an incident: impact, timeline, root causes, what went well, what went badly, and **action items** with owners. **Blameless** means it focuses on systems and processes, not on punishing people. The book's reasoning: if people fear blame, they hide problems, and the same failure happens again.

A good postmortem asks "How did our system allow one person's mistake to reach production?" not "Who made the mistake?" For example: "An engineer pushed a bad config" becomes the action items "add config validation in presubmit" and "canary config changes".

For a test engineer, many postmortem action items are **missing tests**: a regression test, a config test, a load test or a failure-injection test.

8. Release engineering: canaries, progressive rollouts, rollbacks

Sources: SRE book, Chapter 8 "Release Engineering" (<https://sre.google/sre-book/release-engineering/>), and Workbook Chapter 16 "Canarying Releases" (<https://sre.google/workbook/canarying-releases/>).

Chapter 8 stresses **hermetic, reproducible builds** (the same inputs always give the same binary), self-service release tooling, and enforced policies like code review.

- **Canary:** deploy the new version to a small part of traffic (for example 1%) and compare its metrics with the old version (the control). The name comes from canaries that miners carried to detect gas.
- **Progressive rollout:** if the canary is healthy, increase step by step: 1% -> 10% -> 50% -> 100%, often zone by zone or region by region, with a waiting time between steps.
- **Automatic rollback:** if error rate or latency is worse than the control by more than a threshold, roll back without waiting for a human.
- **Feature flags:** ship code turned off, then turn it on gradually. Turning a flag off is often faster than a rollback.

```
def canary_verdict(control_err, canary_err, canary_reqs, max_ratio=1.5, min_reqs=1000):
    if canary_reqs < min_reqs:
        return "WAIT" # not enough data to judge
    if canary_err > control_err * max_ratio and canary_err > 0.001:
        return "ROLLBACK"
    return "PROMOTE"

print(canary_verdict(0.002, 0.0021, 5000)) # PROMOTE
print(canary_verdict(0.002, 0.009, 5000)) # ROLLBACK
print(canary_verdict(0.002, 0.009, 200)) # WAIT
```

Testing tip: the canary analysis is code, so test it. Feed it known-good and known-bad metric sets and check the verdicts, including the "not enough data" case.

9. Load shedding and graceful degradation

Sources: SRE book, Chapter 21 "Handling Overload" (<https://sre.google/sre-book/handling-overload/>) and Chapter 22 "Addressing Cascading Failures" (<https://sre.google/sre-book/addressing-cascading-failures/>).

A **cascading failure** happens when one overloaded part fails, its load moves to others, and they also fail, like falling dominoes. Retries make it worse: every failed request comes back two or three times.

Defences described in these chapters:

- **Load shedding:** when overloaded, reject some requests early and cheaply (HTTP 503) so the rest succeed. Drop the least important traffic first (for example, background prefetch before user clicks).
- **Graceful degradation:** give a simpler but still useful answer. For example, show cached results, hide recommendations, or search fewer documents. Analogy: during Tatkal rush hour, IRCTC could hide extra features and keep only the booking flow running.
- **Retry budgets and backoff:** limit retries per request and per client, and use exponential backoff with jitter.
- **Deadlines:** pass a deadline with each request. If time is already over, do not start the work.
- **Per-client limits and criticality:** protect the service from one heavy client and serve important requests first.

These paths are rarely used, so they often have bugs. Test them on purpose with load tests that push beyond capacity.

10. Capacity planning

Sources: SRE book, Chapter 1 "Introduction" (<https://sre.google/sre-book/introduction/>) lists demand forecasting and capacity planning as SRE responsibilities; Chapter 18 "Software Engineering in SRE" (<https://sre.google/sre-book/software-engineering-in-sre/>) describes intent-based capacity planning.

Steps:

1. **Forecast demand** from history plus planned launches (marketing events, festivals like Diwali sales).
2. **Measure capacity per instance** with load tests: "one server handles 800 QPS at p99 under 200 ms".
3. **Add headroom and redundancy.** Chapter 18's capacity examples use "N + 2 redundancy". A common way to read this: N units carry the peak load, plus 2 spare, so you survive one unit down for planned maintenance and another failing unexpectedly.
4. **Re-check regularly**, because code changes can change per-instance capacity.

Worked example: peak forecast 40,000 QPS, one instance handles 800 QPS. $40,000 / 800 = 50$ instances. Running each instance at 70% for headroom: $50 / 0.7$ is about 72. Spread across 3 zones so losing one zone still leaves enough: each zone needs $72 / 3 = 24$, so 72 instances in total.

Tester's corner

- An SLO is a **continuous test on real traffic**. Help define SLIs that match what users feel, just like good acceptance criteria.
- Probers (black-box monitoring) are **end-to-end tests in production**. Your Playwright and API test skills transfer directly.
- Flaky tests and noisy alerts are the same disease: low signal quality. Fix or quarantine both.
- Test the **rarely used paths**: load shedding, degradation, rollback scripts and failover. They fail exactly when you need them.
- Treat canary analysis and alert rules as code: **unit test them** with known metric inputs.
- Many postmortem action items are missing tests. Track them until they are done.
- Measure your own test infrastructure with SLOs too (for example "95% of presubmit runs finish in under 15 minutes").

Key takeaways

- SLI is the measurement, SLO the target, SLA the contract; 100% is the wrong target.
- Error budget = $1 - \text{SLO}$; 99.9% over 30 days allows 43.2 minutes or 0.1% failed requests.
- Monitor latency, traffic, errors and saturation; page on user-facing symptoms.
- Multi-window burn-rate alerts (for example 14.4x over 1 hour) catch both fast and slow budget burns.
- Use metrics to detect, traces to locate, logs to explain; link them with trace IDs.
- Incidents need clear roles, mitigation first, and blameless postmortems with owned action items.
- Release with canaries, progressive rollouts, automatic rollback and feature flags.

- Prevent cascading failures with load shedding, graceful degradation, retry budgets and deadlines; plan capacity with headroom and N+2.

Quiz

1. Which is an SLO? A) "Latency of each request" B) "99.9% of requests succeed over 30 days" C) "We refund 10% if uptime is below 99.5%" D) "CPU is 70%"
2. Your SLO is 99.95% over 30 days. How many minutes of full downtime does your budget allow?
3. Which is NOT one of the four golden signals? A) Latency B) Traffic C) Code coverage D) Saturation
4. True or false: you should page the on-call engineer whenever CPU goes above 80%, even if users are not affected.
5. A service has an error ratio of 0.72% with a 99.9% SLO. What is the burn rate?
6. During an incident, what should usually come first? A) Finding the root cause B) Mitigating user impact (for example, rolling back) C) Writing the postmortem D) Blaming the change author
7. What does "blameless" mean in a postmortem?
8. A canary at 1% shows an error rate three times the control after enough requests. What should the release system do?
9. True or false: retrying failed requests aggressively always helps during overload.
10. Peak forecast is 30,000 QPS, one instance handles 600 QPS, and you want to run at 75% utilisation. How many instances do you need (before zone redundancy)?

Answer key

1. **B** - It is a target on an indicator over a window. A is an SLI, C is an SLA, D is a resource metric.
2. **21.6 minutes** - $43,200 \text{ minutes} \times 0.0005 = 21.6 \text{ minutes}$.
3. **C** - The four golden signals are latency, traffic, errors and saturation.
4. **False** - High CPU is a cause, not a user-facing symptom. Page on symptoms; use CPU for dashboards or tickets.
5. **7.2** - Burn rate = $0.0072 / 0.001 = 7.2$, so a 30-day budget would be used in about 4 days.
6. **B** - Stop the user pain first (rollback, drain, flag off); investigate root cause after.
7. **Focus on systems, not people** - The postmortem looks for process and system gaps that allowed the mistake, without punishing individuals, so people report problems honestly.
8. **Roll back automatically** - Stop the rollout and revert the canary, then investigate.
9. **False** - Aggressive retries multiply load and can cause cascading failures. Use retry budgets, backoff with jitter, and load shedding.
10. **67 instances** - $30,000 / 600 = 50$ at full load; $50 / 0.75 = 66.7$, so round up to 67.

Flashcards

- **Q:** What is an SLI? — **A:** A measured ratio of good events to total events that reflects user experience.
- **Q:** Why should an SLA be looser than the SLO? — **A:** So you get a warning and time to react before breaking a contract.

- **Q:** What is the error budget for a 99.9% SLO? — **A:** 0.1% of requests, or 43.2 minutes per 30 days.
- **Q:** What does burn rate 1 mean? — **A:** You will use exactly the whole error budget by the end of the window.
- **Q:** What are the four golden signals? — **A:** Latency, traffic, errors and saturation.
- **Q:** What should you page on? — **A:** User-facing symptoms that are urgent and actionable, not internal causes.
- **Q:** What are the three pillars of observability? — **A:** Metrics, logs and traces.
- **Q:** Who is the only group that changes the system during an incident? — **A:** The operations lead's team.
- **Q:** What is a canary release? — **A:** Sending a small share of traffic to the new version and comparing it with the old version before rolling out further.
- **Q:** What is load shedding? — **A:** Rejecting some requests early and cheaply during overload so the rest can succeed.
- **Q:** What is graceful degradation? — **A:** Serving a simpler but still useful response instead of failing completely.
- **Q:** What does N+2 capacity mean? — **A:** Enough capacity to survive one planned outage and one unplanned failure at the same time.

Designing Test Infrastructure at Scale: CI, Distributed Test Execution, Flaky Tests, Analytics and Device Labs

In this chapter:

- Design a CI system with presubmit and postsubmit, end to end, the way you would in a Google SWE-Test interview
- Build a distributed test execution service that shards tests by historical duration (with Python code)
- Detect and quarantine flaky tests with clear, testable rules
- Model test results for analytics, and design a device and browser lab
- Explain how you would test the test infrastructure itself

Time: ~80 minutes | **Level:** Expert

This is the system design round that matters most for a SWE-Test or Test Engineer role. The earlier lesson "Test infrastructure as a design topic" gave you the main boxes. This chapter turns that into a full 45-minute answer with requirements, estimates, APIs, data model, components, scaling, failure handling and testing.

You already know CI pipelines, flaky tests and device clouds from your SDET work. The new skill is to talk about them **at scale**, with numbers and trade-offs.

What Google has said publicly

Use only public facts from *Software Engineering at Google* (Winters, Manshreck and Wright, O'Reilly, 2020), free at <https://abseil.io/resources/swe-book>. Say "according to the SWE at Google book" when you use them.

- **Test sizes** (Chapter 11, "Testing Overview", <https://abseil.io/resources/swe-book/html/ch11.html>): **small** tests run in a single process (often a single thread) and may not sleep, do I/O or use the network; **medium** tests run on a single machine and may only call `localhost`; **large** tests can span machines. Size is about resources, not lines of code. Some of these limits are enforced by test infrastructure.
- **Test scope mix** (Chapter 11): a rough guideline of about 80% unit, 15% integration and 5% end-to-end tests.
- **Flakiness** (Chapter 11): the book says that as you approach 1% flakiness, tests begin to lose value, and that Google's flaky rate hovers around 0.15%, which still means thousands of flakes every day.

- **Hermetic tests** (Chapters 11 and 23): a test should contain everything needed to set up, run and tear down its environment, with no shared databases or production backends.
- **Test doubles** (Chapter 13, <https://abseil.io/resources/swe-book/html/ch13.html>): fakes, stubs and mocks, with a preference for real implementations or fakes where possible.
- **Larger testing** (Chapter 14, <https://abseil.io/resources/swe-book/html/ch14.html>): types such as performance and load tests, probes and canary analysis, disaster recovery and chaos engineering, and A/B diff (regression) tests.
- **TAP** (Chapter 23, "Continuous Integration", <https://abseil.io/resources/swe-book/html/ch23.html>): the **Test Automation Platform** is Google's global continuous build. The book says TAP handles more than 50,000 unique changes and runs more than four billion individual test cases every day. Teams define a fast **presubmit** subset; the book says a change that passes presubmit has a very high likelihood (95%+) of passing the rest, and the average wait to submit is around 11 minutes. After submit, TAP runs all affected tests asynchronously (postsubmit). Because changes arrive faster than one per second, TAP **batches** changes, then splits failing batches and offers **culprit finding** tools; it can **automatically roll back** a change when it is highly confident it is the culprit. Each team has a **Build Cop** who keeps the build green. Tests mostly run on **Forge**, a distributed build-and-test system, and TAP uses the dependency graph from Forge and **Blaze** (open-sourced as Bazel) to run only affected tests.

Do not add internals beyond this. In the interview, design **your own** system and say "this is similar in spirit to what the SWE at Google book describes for TAP".

Step 1: Requirements

Ask first. A good set of assumptions:

Functional

- Run tests for every proposed change (**presubmit**) and block submission if they fail.
- Run a larger set after submission (**postsubmit**), find which change broke a test, and notify the owner.
- Shard tests across many machines; retry infrastructure failures.
- Detect flaky tests and quarantine them.
- Store results, logs and artifacts; offer dashboards and APIs.
- Run browser and mobile tests on a device lab.

Non-functional

- **Fast feedback:** presubmit p95 under 15 minutes.
- **Correct signal:** infrastructure failures must never be reported as test failures.
- **Scale:** 5,000 changes per day, growing 2x per year.
- **Reproducible:** hermetic runs; anyone can re-run the exact same test.
- **Cost aware:** only run what a change can affect.
- **Available:** if CI is down, nobody can submit, so target 99.9% for the presubmit path.

Step 2: Estimates

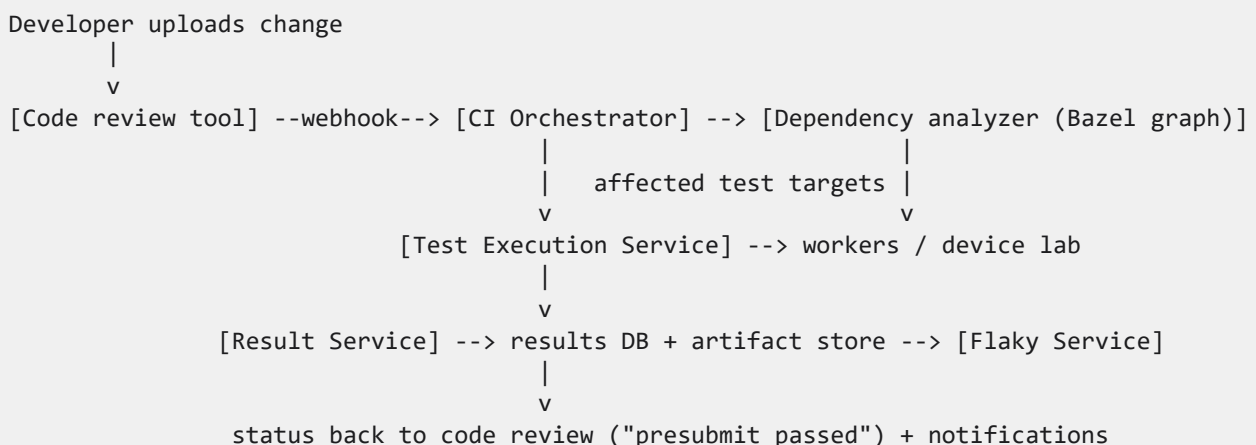
Assume a company with 3,000 engineers (not Google; these numbers are for practice).

- Changes per day: 5,000. Each change runs presubmit about 3 times (new revisions) = **15,000 presubmit runs per day**.
- Affected tests per run (after dependency analysis): about 2,000 on average.
- Test executions per day: $15,000 \times 2,000 = \mathbf{30 \text{ million}}$. Add postsubmit and nightly runs: about **40 million per day**.
- Rate: $40,000,000 / 86,400 = \text{about } \mathbf{460 \text{ per second}}$; peak (office hours, 3x) about **1,400 per second**.
- Compute: average test 5 seconds $\rightarrow 40M \times 5 \text{ s} = 200M \text{ CPU-seconds per day} = \text{about } 55,600 \text{ CPU-hours per day} \rightarrow \text{about } \mathbf{2,300 \text{ cores busy on average}}$, around **7,000 at peak**.
- Result rows: $40M \times 200 \text{ bytes} = \mathbf{8 \text{ GB per day}}$, about **3 TB per year**.
- Logs: $40M \times 20 \text{ KB} = \mathbf{800 \text{ GB per day}}$. Keep passing logs 14 days, failing logs 1 year: tens of TB in object storage.

These numbers drive decisions: a single SQL table will struggle with 3 TB per year of high-rate inserts plus analytics, so use a wide-column or time-partitioned store for raw results and a columnar warehouse for analytics.

Part A: The CI system (presubmit and postsubmit)

High-level flow



Presubmit

1. A change is uploaded. The orchestrator creates a **run**.
2. The **dependency analyzer** finds every test target that depends on changed files (with a build graph like Bazel). This is called **test selection** or **affected-test analysis**.
3. Presubmit runs **small and medium** tests, plus a few chosen large tests. Large, slow, non-hermetic tests go to postsubmit.
4. Quarantined flaky tests still run but **cannot block** the change.
5. The result is posted to the code review. Submission is blocked until presubmit is green.

Postsubmit

1. After submit, run **all affected tests**, including large ones.
2. At high change rates, **batch** several changes into one run to save compute.
3. If a batch fails, find the **culprit**: re-run the failing tests on each change, or **bisect** (binary search) between the last green and first red change.
4. Notify the change author and the team's build cop; offer or perform an **automatic rollback** if confidence is high.

```
def find_culprit(changes, test_passes):  
    """changes: ordered list, first is known good, last is known bad.  
    test_passes(change) runs the failing test at that change."""  
    lo, hi = 0, len(changes) - 1          # lo = good, hi = bad  
    while hi - lo > 1:  
        mid = (lo + hi) // 2  
        if test_passes(changes[mid]):  
            lo = mid  
        else:  
            hi = mid  
    return changes[hi]                    # first bad change  
  
history = ["c1", "c2", "c3", "c4", "c5", "c6"]  
broken_from = "c4"  
print(find_culprit(history, lambda c: c < broken_from)) # c4
```

Bisection needs about $\log_2(N)$ runs: 64 changes need about 6 runs. With flaky tests, run each step 2-3 times, or the search can point to the wrong change.

APIs

```
POST /v1/runs {change_id, revision, kind: PRESUBMIT|POSTSUBMIT, priority}  
GET /v1/runs/{run_id} -> status, counts, links  
GET /v1/runs/{run_id}/tests?status=FAILED  
POST /v1/runs/{run_id}/retry {test_ids} (author re-runs failures)  
GET /v1/changes/{change_id}/status -> PENDING | PASSED | FAILED | INFRA_ERROR  
GET /v1/tests/{test_id}/history?limit=200  
POST /v1/tests/{test_id}/quarantine {reason, bug_id}
```

All write APIs take an **idempotency key**, because webhooks are delivered at least once.

Part B: Distributed test execution service

Components

- **Scheduler:** turns a run into **shards** and puts them on a priority queue. Presubmit has higher priority than nightly runs.
- **Queue:** durable (for example Pub/Sub or Kafka), with per-team quotas so one team cannot take all workers.
- **Workers:** containers on a cluster manager (Kubernetes outside Google; Borg is Google's public example, see the papers chapter). Each worker pulls a shard, fetches build outputs from a **remote**

cache, runs tests in a sandbox and uploads results.

- **Lease and heartbeat**: a worker leases a shard for, say, 2 minutes and renews it. If it stops renewing, the shard goes back to the queue.

Sharding by historical duration

Splitting by test count is bad: one shard may get all the slow tests. Use the **median duration from history**, and pack shards using the **Longest Processing Time first (LPT)** rule: sort tests from slowest to fastest, and always give the next test to the currently lightest shard.

```
import heapq

def shard_by_duration(durations, num_shards):
    """durations: {test_id: median_seconds}. Returns list of (total, tests)."""
    heap = [(0.0, i, []) for i in range(num_shards)] # (total, id, tests)
    for test, secs in sorted(durations.items(), key=lambda x: -x[1]):
        total, i, tests = heapq.heappop(heap) # lightest shard
        tests.append(test)
        heapq.heappush(heap, (total + secs, i, tests))
    return sorted((t, tests) for t, _, tests in heap)

hist = {"login": 40, "checkout": 90, "search": 30, "profile": 20, "cart": 60, "api": 10}
for total, tests in shard_by_duration(hist, 3):
    print(total, tests)
# 80.0 ['cart', 'profile']
# 80.0 ['login', 'search', 'api']
# 90.0 ['checkout']
```

Run time is the slowest shard (90 s here), so balance matters more than count. Extra rules:

- **New tests** have no history: use the average for their size (small, medium, large).
- **Choose the shard count** from a target: total time / target time per shard, capped by available workers and startup overhead (if starting a worker costs 30 s, 2-second shards are wasteful).
- **Straggler handling**: when 95% of shards are done, start a **backup copy** of the slowest remaining shard and take whichever finishes first (the same idea as MapReduce backup tasks).
- **Test ordering inside a shard**: run historically failing tests first, so authors get a failure signal sooner.

Failure handling

Failure	Detection	Action
Worker dies	Lease expires	Re-queue shard on another worker
Bad worker (disk full, broken network)	Many tests fail with infra error signatures on one host	Drain host, retry shards elsewhere, mark INFRA_ERROR not FAIL
Test hangs	Per-test timeout by size (for example small 60 s, large 15 min)	Kill, record TIMEOUT, collect thread dump
Queue backlog	Queue age metric	Autoscale workers; shed nightly work before presubmit

Failure	Detection	Action
Remote cache down	Error rate	Fall back to building locally (slower but correct)

The key rule: **separate infrastructure failures from test failures**. Report them with different statuses, and never blame a developer for an infra problem.

Part C: Flaky test detection and quarantine

A **flaky test** passes and fails on the **same code and configuration**. Following the SWE at Google book, treat flakiness as a signal-quality problem: rerunning only trades CPU for engineering time and delays the fix.

Signals

1. **Pass on retry**: fails, then passes on the same commit. Strongest signal.
2. **Mixed results on the same commit** across different runs.
3. **Flip rate** in postsubmit history: pass, fail, pass, fail with no relevant change.
4. **Deliberate reruns**: a nightly job re-runs suspicious tests 50-100 times to measure the rate.

Scoring and rules

```
def flaky_score(runs):
    """runs: list of (commit, passed). Score = share of commits with mixed results."""
    by_commit = {}
    for commit, passed in runs:
        by_commit.setdefault(commit, set()).add(passed)
    mixed = sum(1 for results in by_commit.values() if len(results) == 2)
    return mixed / len(by_commit)

def decide(score, flaky_events, threshold=0.02, min_events=3):
    return "QUARANTINE" if score >= threshold and flaky_events >= min_events else "OK"

runs = [("a", True), ("a", False), ("b", True), ("c", True), ("c", False), ("d", True)]
s = flaky_score(runs)
print(round(s, 2), decide(s, flaky_events=2))    # 0.5 OK (not enough evidence yet)
print(decide(0.05, flaky_events=4))            # QUARANTINE
```

Quarantine workflow

1. Test crosses the threshold -> mark **quarantined**. It still runs (to keep collecting data) but **does not block** presubmit. The SWE at Google book mentions teams using a tool to temporarily remove flaky tests from presubmit while they are fixed.
2. **File a bug automatically** to the owning team with evidence: failing logs, error signature, failure rate, first-seen date.
3. **Un-quarantine** after the fix, once the test passes, say, 200 runs in a row.
4. **Limit quarantine size per team** and age; a test quarantined for 90 days escalates to the team lead. Otherwise quarantine becomes a graveyard of hidden bugs.

Root-cause helpers

Group failures by **error signature** (normalised stack trace or message hash). Correlate with worker host, time of day, test order and parallelism. Common causes you already know: timing waits, shared test data, order dependency, real race conditions in the product, and unstable external services (fix by making the test hermetic with fakes).

Part D: Test result storage and analytics

Data model

```
runs          (run_id PK, change_id, revision, kind, priority, status, created_at,
finished_at)
shards        (run_id, shard_id, worker_id, status, started_at, finished_at)
test_results  (test_id, run_id, attempt, status, duration_ms, error_signature,
               worker_id, commit, created_at)          -- one row per attempt
tests         (test_id PK, target, owner_team, size, median_duration_ms,
               flaky_score, quarantined, quarantine_bug)
artifacts     (run_id, test_id, attempt, kind: LOG|SCREENSHOT|VIDEO|COVERAGE, uri)
```

Status values: PASSED, FAILED, FLAKY (passed on retry), TIMEOUT, SKIPPED, INFRA_ERROR.

Storage choices

- **Hot path (results of current runs):** a wide-column store (for example Bigtable or Cassandra) with row key `test_id#reverse_timestamp`, so "last 200 results of this test" is one contiguous scan. A SQL database for runs and tests metadata, which are small.
- **Artifacts:** object storage (for example GCS or S3) with lifecycle rules: delete passing logs after 14 days, keep failing logs longer.
- **Analytics:** stream results into a columnar warehouse (for example BigQuery), partitioned by day, for questions like:
 - Slowest 100 tests by p95 duration.
 - Flakiest tests per team this week.
 - Percent of time the main branch is green.
 - Presubmit queue time and run time p50/p95.
 - Infra error rate per worker pool.
 - Cost per run, per team.

Scaling

Write results in **batches** from workers through a result service (or a queue) rather than one database insert per test. Use **idempotent writes** keyed by `(run_id, test_id, attempt)` so retried uploads do not double count. Dashboards read from pre-aggregated tables updated every few minutes, not from raw rows.

Part E: Device and browser lab

Requirements

Run Selenium, Playwright, Appium and native mobile tests on many browser versions and real devices. Say 300 real phones, 2,000 emulator or browser slots, 50,000 device-test sessions per day.

Components

```
Test shard --lease(model=Pixel 8, os=14)--> [Device Manager] --> [Host machine] --USB-->
[Phone]
                                     |
                                     inventory DB + health
reset
                                     |
Browser shard --> [Browser grid: containers with Chrome/Firefox/Edge versions]
                                     agent: install, run,
```

- **Device manager:** keeps an **inventory** (model, OS version, state: IDLE, LEASED, RESETTING, BROKEN) and gives **leases** by capability.
- **Host agent:** runs on machines with USB hubs; installs apps, runs tests, captures video and logs, and **resets** the device (uninstall apps, clear data, reboot when needed).
- **Browser grid:** short-lived containers, one per session, with pinned browser versions, so every session starts clean (hermetic).
- **Scheduling:** match by capability; queue by priority; avoid starving rare devices by reserving some for presubmit.

Failure handling

- **Health checks** before each lease: battery, storage, network, ADB connection. Failing devices go to BROKEN and a ticket is filed for the lab team.
- If a device dies mid-test, record **INFRA_ERROR**, retry on another device of the same model.
- Track **flaky rate per device**: if one phone causes most failures, it is the device, not the test.
- Leases expire automatically so a crashed test cannot hold a device forever.

Testing the test infrastructure itself

This is where you stand out. The CI system is a product, and its users are engineers. Test it like one:

1. **Canary repository:** a small repo with known tests - always pass, always fail, 10% flaky, hangs, crashes, uses too much memory. Run it every 10 minutes and assert each gets the right status. This is a **prober** for CI.
2. **Unit tests for the logic:** sharding (balance within 10% of ideal), culprit finding (with and without flakes), flaky scoring and quarantine rules, status mapping (infra vs test failure).
3. **Fault injection:** kill workers mid-shard, drop result uploads, slow the remote cache, fill a disk, expire leases. Assert no lost results, no duplicates and correct INFRA_ERROR statuses.
4. **Shadow mode for new algorithms:** run a new flaky detector or test selection in parallel without acting, and compare its decisions with the current one before switching.

5. **Test selection safety:** periodically run the **full** suite and check whether any failure was missed by affected-test analysis.
6. **Load tests:** replay a peak day of runs into a staging cluster to find scheduler and database bottlenecks.
7. **SLOs for CI:** presubmit p95 time, infra error rate under 0.5%, percent of runs with a correct status. Alert on burn rate, just like a production service.
8. **Canary releases of CI itself:** roll new worker images to 5% of the fleet first and compare infra error rates.

Trade-offs to say out loud

- **Presubmit coverage vs speed:** more tests before submit means fewer broken builds but slower developers. The SWE at Google book describes running a fast subset in presubmit and the rest in postsubmit.
- **Retries vs hiding bugs:** retries unblock developers but can hide real race conditions. Always record FLAKY separately.
- **Batching vs blame:** batching saves compute but makes culprit finding harder.
- **Hermetic vs realistic:** hermetic tests are stable; live backends catch integration problems. Use both, at different stages.
- **More shards vs overhead:** faster wall time, but more startup cost and scheduling load.

Tester's corner

- You have lived the pain points: flaky tests, slow pipelines, broken devices. Use **real stories** with numbers ("I cut suite time from 60 to 12 minutes by sharding on duration").
- Always split **infra failures** from **test failures**; it is the most common signal-quality bug in CI systems.
- Talk about **metrics**: feedback time, flaky rate, green rate, infra error rate, cost per run.
- Mention **test sizes** (small, medium, large) and hermeticity; they show you know Google's public vocabulary.
- Treat quarantine as **temporary** with owners, bugs and deadlines.
- Finish every design with "**how I would test this system**"; most candidates forget.

Key takeaways

- Presubmit runs a fast, affected subset and blocks submit; postsubmit runs everything, batches changes and finds culprits by bisection.
- Shard by historical duration with the LPT rule; wall time equals the slowest shard.
- Workers use leases and heartbeats; dead workers' shards are re-queued, and infra errors are never reported as test failures.
- Flaky tests are detected by pass-on-retry and mixed results on the same commit, then quarantined with an auto-filed bug and release criteria.

- Store one row per attempt, use a row key like `test_id#reverse_timestamp`, artifacts in object storage, analytics in a columnar warehouse.
- A device lab needs inventory, capability-based leases, health checks and automatic reset.
- Test the test infrastructure with a canary repo, fault injection, shadow mode and its own SLOs.
- Quote only public facts from the SWE at Google book (test sizes, 0.15% flake rate, TAP presubmit and postsubmit).

Quiz

1. According to the SWE at Google book, what defines a "medium" test? A) 100-1,000 lines of code B) Runs on a single machine and may only use the network via localhost C) Takes 1-5 minutes D) Tests two classes
2. Tests have durations 50, 40, 30, 30, 20 seconds and you have 2 shards. Using the LPT rule, what is the wall time?
3. True or false: sharding tests by equal count is usually as good as sharding by duration.
4. A worker stops sending heartbeats in the middle of a shard. What should the system do?
5. Which is the strongest signal that a test is flaky? A) It is slow B) It fails, then passes on retry on the same commit C) It was recently added D) It has many assertions
6. What does the SWE at Google book say about flakiness around 1%? A) It is ideal B) Tests begin to lose value C) It is impossible D) It only matters for unit tests
7. Postsubmit found a failure in a batch of 32 changes. About how many bisection steps are needed to find the culprit, assuming no flakiness?
8. True or false: a quarantined test should stop running completely.
9. Many unrelated tests fail with "connection refused" on one worker host. What is the most likely cause and action?
10. How would you verify that your test infrastructure reports correct statuses all the time?

Answer key

1. **B** - Medium tests may use multiple processes and blocking calls but must stay on one machine, calling only localhost.
2. **90 seconds** - Sorted: 50, 40, 30, 30, 20. 50 -> A, 40 -> B, 30 -> B (70), 30 -> A (80), 20 -> B (90). Shards are 80 and 90, so wall time is 90 s. (The best possible split of 170 s total is 80/90, so LPT is optimal here.)
3. **False** - Test durations vary a lot, so equal counts can give very unequal shards; wall time is set by the slowest shard.
4. **Re-queue the shard** - When the lease expires, put the shard back on the queue for another worker, and record the attempt as an infra issue, not a test failure.
5. **B** - Different results on the same code and config is the definition of flaky.
6. **B** - The book says that as you approach 1% flakiness, tests begin to lose value, and Google's rate hovers around 0.15%.
7. **About 5** - $\log_2(32) = 5$. With flaky tests, repeat each step to avoid blaming the wrong change.

8. **False** - It should keep running without blocking presubmit, so you collect data and know when it is fixed.
9. **A bad worker** - It is an infrastructure problem; drain the host, retry the affected shards elsewhere, and mark the results INFRA_ERROR.
10. **Canary repository** - Run a repo of known pass, fail, flaky, hang and crash tests on a schedule and alert if any status is wrong; add fault injection and SLOs.

Flashcards

- **Q:** What is presubmit? — **A:** Tests that run on a proposed change before it is submitted, blocking submission if they fail.
- **Q:** What is postsubmit? — **A:** Tests run after a change lands, usually the full affected set, including large slow tests.
- **Q:** What is TAP, according to the SWE at Google book? — **A:** The Test Automation Platform, Google's global continuous build that runs tests for almost all changes.
- **Q:** What are Google's three test sizes? — **A:** Small (single process), medium (single machine, localhost only), large (can span machines).
- **Q:** What is the LPT sharding rule? — **A:** Sort tests slowest first and always add the next test to the currently lightest shard.
- **Q:** How do you find the change that broke a test in a batch? — **A:** Bisect between the last green and first red change, repeating runs if flakiness is possible.
- **Q:** Why separate INFRA_ERROR from FAILED? — **A:** So developers are never blamed for infrastructure problems and the signal stays trustworthy.
- **Q:** What happens to a quarantined test? — **A:** It keeps running but cannot block presubmit, and a bug is filed to its owner.
- **Q:** What row key fits test history queries? — **A:** `test_id#reverse_timestamp`, so recent results for one test are a single contiguous scan.
- **Q:** What does a device manager track? — **A:** Inventory, capabilities, state (idle, leased, resetting, broken) and leases with expiry.
- **Q:** What is a CI canary repository? — **A:** A repo of tests with known outcomes, run regularly to check the CI system reports correct statuses.
- **Q:** What flaky rate does the SWE at Google book report for Google? — **A:** It says Google's flaky rate hovers around 0.15%, which still means thousands of flakes every day.